

ICPC Training

Greedy, Bitwise and Number Theory

63 problems from the ICPC Training tracker and past ICPC India preliminaries

Maruri Saikiran

September 25, 2026

Contents

How to Use This Book	v
1 Greedy Algorithms	1
1.1 Theory and Must-Know Patterns	1
1.2 Task Scheduling Problem	3
1.3 Good Contest	4
1.4 Boss Fight	5
1.5 Flee to Shelter	7
1.6 Ordering Teams	8
1.7 Stick Lengths	9
1.8 Movie Festival	11
1.9 Restaurant Customers	12
1.10 Tasks and Deadlines	13
1.11 Reading Books	15
1.12 Creating a Wormhole	16
1.13 Concert Tickets	17
1.14 Towers	19
1.15 Missing Coin Sum	20
1.16 Playlist	22
1.17 Traffic Lights	23
1.18 Room Allocation	24
1.19 Factory Machines	26
1.20 Tatar TV Show	27
1.21 Delete and Concatenate	29
1.22 Always Changing	30
1.23 MEX Multiset	32
1.24 Reduction Game	34
1.25 A Game of Robots	36
1.26 Cracker Shops	37
1.27 Jai Hanuman	40
1.28 Break, Merge and Sort	41
1.29 Minimum Weight Bi-Partition	44
1.30 Coins	46
2 Bitwise Operations and XOR	49
2.1 Theory and Must-Know Patterns	49
2.2 Bitwise Exclusive Or	51
2.3 Bit Strings	51
2.4 Gray Code	53
2.5 A New Number System	54
2.6 Bitwise Reversion	55
2.7 Deletion Sort	56
2.8 ORXOR	57
2.9 Jackpot	59
2.10 Express the Number	61
2.11 Beautiful XOR	63

2.12	The 67th XOR Problem	64
2.13	Maximize XOR, Minimize Operations	66
2.14	Xor Battle	67
2.15	XOR Matching	69
3	Number Theory	73
3.1	Theory and Must-Know Patterns	73
3.2	Calculate GCD	75
3.3	The Last Cracker	76
3.4	Sieve of Erato67henes	77
3.5	Exponentiation	78
3.6	Exponentiation II	79
3.7	Counting Divisors	80
3.8	Common Divisors	82
3.9	Next Prime	83
3.10	Modulo Equality	85
3.11	Obtain Desired Standard Deviation	86
3.12	Euclid, Sequence and Two Numbers	88
3.13	GCD on Blackboard	89
3.14	Prime Multiples	90
3.15	Sum of Divisors	91
3.16	Divisor Analysis	93
3.17	A Simple GCD Problem (Easy Version)	95
3.18	Maximum GCD on Whiteboard	96
3.19	Substring Problem	98
3.20	Steel Bar Segment Durability	99
3.21	ISRO Digit Sum	100
A	Contest Cheat Sheet	105

How to Use This Book

This book covers the first three topics of the *ICPC Training* tracker (prep for the ICPC India online preliminary): **Greedy**, **Bitwise** / **XOR** and **Number Theory**.

Where the problems come from

- **Notion tracker** (“ICPC Training” page): every CSES, Codeforces and AtCoder problem listed under the three topics.
- **Past ICPC India preliminaries** (the tracker’s *Historical Prelim Archive*, PDFs from github.com/m-e-r-l-i-n/icpc-india): every prelim problem the archive files under these three topics. Statements were taken from the original contest PDFs, or from CodeChef’s archive where the PDF only contains an editorial.
- Three 2020–21 Amritapuri problems survive only as solution slides; their statements are **reconstructed** and marked with a purple box. Their solutions are validated against exhaustive search under the reconstructed rules.

No problem appears in both lists, so nothing had to be de-duplicated: the book has 63 distinct problems, ordered from easy to hard inside each chapter.

Every section has the same shape

Problem statement → source and level → pattern recognition → approach with the correctness argument → pseudocode → a dry run whose numbers were checked by running the code → the C++17 solution → time and space complexity → pitfalls.

The code is the tested code

Every listing is pulled directly from `icpc-training/<topic>/*.cpp`: C++17, specific headers (no `bits/stdc++.h`), fast I/O, small helper functions. The repository contains

- `tests/run_tests.py`: compiles every solution with `-Wall -Wextra` and runs all official samples (checkers for problems with several valid outputs);
- `tests/stress/*.py`: randomised stress tests against exhaustive brute-force references — tens of thousands of cases, zero mismatches.

Passing samples and stress tests is strong evidence, not a judge verdict: submit to the original judges to be certain.

Suggested order before the prelim

Easy problems of all three chapters first (they are what decides Q1–Q3), then the PYQs, then the “Hard” tags. Re-read the pitfall boxes the evening before the contest.

Problem Index

#	Problem	Source	Level	List
Greedy				
1	Task Scheduling Problem (p. 3)	AtCoder ABC103 A	Easy	Notion
2	Good Contest (p. 4)	Codeforces 2266A	Easy	Notion
3	Boss Fight (p. 5)	Codeforces 2252A	Easy	Notion
4	Flee to Shelter (p. 7)	ICPC 2013–14 Amritapuri Online Round	Easy	PYQ
5	Ordering Teams (p. 8)	ICPC 2017–18 Online Prelim., Kharagpur	Easy	PYQ
6	Stick Lengths (p. 9)	CSES 1074	Easy	Notion
7	Movie Festival (p. 11)	CSES 1629	Easy	Notion
8	Restaurant Customers (p. 12)	CSES 1619	Easy	Notion
9	Tasks and Deadlines (p. 13)	CSES 1630	Easy	Notion
10	Reading Books (p. 15)	CSES 1631	Easy–Medium	Notion
11	Creating a Wormhole (p. 16)	ICPC 2013–14 Amritapuri Online Round	Easy–Medium	PYQ
12	Concert Tickets (p. 17)	CSES 1091	Easy–Medium	Notion
13	Towers (p. 19)	CSES 1073	Medium	Notion
14	Missing Coin Sum (p. 20)	CSES 2183	Medium	Notion
15	Playlist (p. 22)	CSES 1141	Easy–Medium	Notion
16	Traffic Lights (p. 23)	CSES 1163	Medium	Notion
17	Room Allocation (p. 24)	CSES 1164	Medium	Notion
18	Factory Machines (p. 26)	CSES 1620	Medium	Notion
19	Tatar TV Show (p. 27)	Codeforces 2236B	Easy–Medium	Notion
20	Delete and Concatenate (p. 29)	Codeforces 2245B	Medium	Notion
21	Always Changing (p. 30)	Codeforces 2252B	Medium	Notion
22	MEX Multiset (p. 32)	Codeforces 2259D	Medium	Notion
23	Reduction Game (p. 34)	ICPC 2018–19 Online Prelim.	Medium	PYQ
24	A Game of Robots (p. 36)	ICPC 2018–19 Online Prelim.	Medium	PYQ
25	Cracker Shops (p. 37)	ICPC 2014–15 Kharagpur Online Re-contest	Medium–Hard	PYQ
26	Jai Hanuman (p. 40)	ICPC 2014–15 Kharagpur Online Re-contest	Medium–Hard	PYQ
27	Break, Merge and Sort (p. 41)	ICPC 2020–21 Amritapuri Prelim., problem 3	Medium	PYQ
28	Minimum Weight Bi-Partition (p. 44)	ICPC 2020–21 Amritapuri Prelim., problem 4	Medium	PYQ
29	Coins (p. 46)	AtCoder AGC018 C	Hard	Notion
Bitwise				
30	Bitwise Exclusive Or (p. 51)	AtCoder ABC213 A	Easy	Notion
31	Bit Strings (p. 51)	CSES 1617	Easy	Notion
32	Gray Code (p. 53)	CSES 2205	Easy–Medium	Notion
33	A New Number System (p. 54)	ICPC 2010–11 Kanpur Online Round	Easy	PYQ
34	Bitwise Reversion (p. 55)	Codeforces 2153B	Easy	Notion
35	Deletion Sort (p. 56)	Codeforces 2200B	Easy	Notion
36	ORXOR (p. 57)	AtCoder ABC197 C	Easy–Medium	Notion
37	Jackpot (p. 59)	ICPC 2013–14 Kharagpur Online Round	Hard	PYQ
38	Express the Number (p. 61)	ICPC 2020–21 Amritapuri Prelim., problem 2	Easy–Medium	PYQ
39	Beautiful XOR (p. 63)	Codeforces 2162C	Medium	Notion

#	Problem	Source	Level	List
40	The 67th XOR Problem (p. 64)	Codeforces 2218E	Medium	Notion
41	Maximize XOR, Minimize Operations (p. 66)	Codeforces 2260C	Medium	Notion
42	Xor Battle (p. 67)	AtCoder AGC045 A	Hard	Notion
43	XOR Matching (p. 69)	AtCoder ABC126 F	Hard	Notion
Number Theory				
44	Calculate GCD (p. 75)	AtCoder Math & Algorithm 015	Easy	Notion
45	The Last Cracker (p. 76)	ICPC 2014–15 Kharagpur Online Re-contest	Easy	PYQ
46	Sieve of Erato67henes (p. 77)	Codeforces 2195A	Easy	Notion
47	Exponentiation (p. 78)	CSES 1095	Easy	Notion
48	Exponentiation II (p. 79)	CSES 1712	Easy–Medium	Notion
49	Counting Divisors (p. 80)	CSES 1713	Easy–Medium	Notion
50	Common Divisors (p. 82)	CSES 1081	Medium	Notion
51	Next Prime (p. 83)	CSES 3396	Medium	Notion
52	Modulo Equality (p. 85)	ICPC 2017–18 Online Prelim., Kharagpur	Medium	PYQ
53	Obtain Desired Standard Deviation (p. 86)	ICPC 2017–18 Online Prelim.	Easy–Medium	PYQ
54	Euclid, Sequence and Two Numbers (p. 88)	Codeforces 2234A	Easy	Notion
55	GCD on Blackboard (p. 89)	AtCoder ABC125 C	Medium	Notion
56	Prime Multiples (p. 90)	CSES 2185	Medium	Notion
57	Sum of Divisors (p. 91)	CSES 1082	Medium	Notion
58	Divisor Analysis (p. 93)	CSES 2182	Medium–Hard	Notion
59	A Simple GCD Problem (Easy Version) (p. 95)	Codeforces 2210C1	Medium	Notion
60	Maximum GCD on Whiteboard (p. 96)	Codeforces 2156C	Medium–Hard	Notion
61	Substring Problem (p. 98)	ICPC 2013–14 Kharagpur Online Round	Medium	PYQ
62	Steel Bar Segment Durability (p. 99)	ICPC 2014–15 Amritapuri Online Round	Medium	PYQ
63	ISRO Digit Sum (p. 100)	ICPC 2014–15 Amritapuri Online Round	Hard	PYQ

Chapter 1

Greedy Algorithms

A greedy algorithm commits to the choice that looks best right now and never looks back. That is only safe when you can prove that some optimal solution agrees with your choice. In a contest the proof is usually two lines long — but those two lines are the difference between an Accepted and a Wrong Answer on test 37.

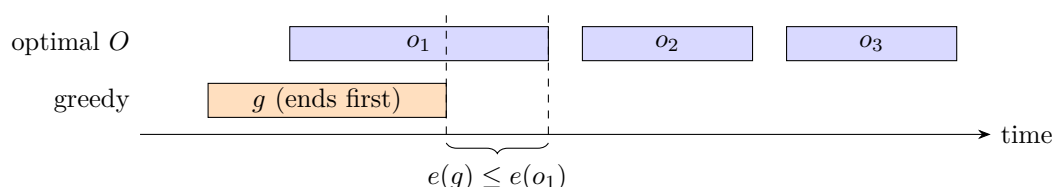
1.1 Theory and Must-Know Patterns

A greedy algorithm builds a solution one decision at a time, always taking the locally best option by some rule. Three questions decide whether a greedy is correct:

1. **What is the rule?** (“earliest end time first”, “smallest element first”, “largest coin that fits”, ...)
2. **Why does some optimal solution follow the rule?** (the proof — usually an exchange argument)
3. **How do I apply the rule fast?** (sorting, a heap, a multiset, two pointers)

Proof technique 1: the exchange argument

Take any optimal solution O that disagrees with the greedy choice g . Modify O by swapping in g and show the result is still valid and no worse. Repeating the swap turns O into the greedy solution without ever losing value, so greedy is optimal.



In interval scheduling (Section 1.8) the interval g with the earliest end time finishes no later than o_1 , so replacing o_1 by g cannot collide with $o_2, o_3, \dots$ — the solution keeps the same size.

Proof technique 2: greedy stays ahead

Show by induction that after every step the greedy partial solution is at least as good as any other partial solution by some measure (“has watched at least as many movies and is free at least as early”). Then at the end it is at least as good as the optimum.

Proof technique 3: a matching lower bound

Find a quantity every solution must pay (“each non-survivor unit must be removed”, “someone must miss problem i in $n - a_i$ places”) and show the greedy pays exactly that. Several prelim problems in this chapter are solved this way: the formula is a lower bound, and the greedy construction achieves it.

The ICPC greedy toolkit

Pattern	Typical rule	Problems here
Sort + scan	process in sorted order, keep a running value	Movie Festival, Tasks and Deadlines, Wormhole, Missing Coin Sum
Median / extremes	optimum sits at a median or at max/min	Stick Lengths, Task Scheduling, Reading Books
Multiset “best fit”	take the closest item $\leq$ or $>$ a key	Concert Tickets, Towers, Traffic Lights
Heap of resources	reuse the resource that frees earliest	Room Allocation, Coins (top- X sums)
Sweep line / difference array	+1 at start, -1 after end	Restaurant Customers, Game of Robots
Two pointers	window never needs to move left	Playlist, Cracker shops
Binary search on the answer	“can we finish by time T ?” is monotone	Factory Machines
Huffman / merge smallest	repeatedly combine the two cheapest	Break, Merge and Sort
Counting argument	answer = forced lower bound	Good Contest, Boss Fight, Reduction Game, Always Changing
Parity invariant	an operation preserves parity of something	Tatar TV Show

Contest Checklist Before Submitting a Greedy

1. Write the exchange (or lower-bound) argument in one sentence. If you cannot, brute-force small cases first.
2. Check ties: does the sort key need a secondary key?
3. Check overflow: sums of $2 \cdot 10^5$ values up to 10^9 need `long long`.
4. Check the empty / single-element / all-equal cases.

Problems are ordered roughly from easy to hard. Every C++ listing in this chapter is the exact file in `icpc-training/greedy/`, and every one of them passes the official samples plus randomised stress tests against an exhaustive brute force (`tests/`).

1.2 Task Scheduling Problem

Source: AtCoder ABC103 A (Notion: Greedy)

Level: Easy

Problem Statement. There are three tasks with values A_1, A_2, A_3 ($1 \leq A_i \leq 100$). The first task you complete costs 0; completing task j right after task i costs $|A_j - A_i|$. Find the minimum total cost to complete all three tasks.

Pattern Recognition

“Visit points on a line, pay the distance between consecutive visits” — the cost of any route is at least the length of the segment it covers.

Approach and intuition

Put the three values on a number line. Any order of visiting them must travel from the smallest to the largest value at some point, so every order costs at least $\max A - \min A$. Visiting them in sorted order walks monotonically from min to max and costs exactly that amount.

Why It Is Optimal

Lower bound: the route touches both $\min A$ and $\max A$; by the triangle inequality the total path length between those two visits is at least $\max A - \min A$. *Achievability:* sorted order has cost $(A_{(2)} - A_{(1)}) + (A_{(3)} - A_{(2)}) = A_{(3)} - A_{(1)}$.

Algorithm

- 1: read A_1, A_2, A_3
- 2: **return** $\max(A_1, A_2, A_3) - \min(A_1, A_2, A_3)$

Dry Run. $A = (1, 6, 3)$: sorted 1, 3, 6, cost $2 + 3 = 5 = 6 - 1$. ✓

C++17 implementation

```

1 // AtCoder ABC103 A - Task Scheduling Problem
2 // Greedy: visiting values in sorted order costs exactly max - min.
3 #include <algorithm>
4 #include <iostream>
5 using namespace std;
6
7 int main() {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10    int a, b, c;
11    cin >> a >> b >> c;
12    int hi = max({a, b, c}), lo = min({a, b, c});
13    cout << hi - lo << '\n'; // any walk must span [lo, hi]
14    return 0;
15 }
```

Complexity Analysis

Time: $O(1)$. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- All equal values (100 100 100) give 0 — the formula handles it.
- The same idea generalises to n points: answer $\max - \min$, not a sum of sorted gaps you must compute one by one.

1.3 Good Contest

Source: Codeforces 2266A (Notion: Greedy, 800)

Level: Easy

Problem Statement. A contest has three problems and n participants ($1 \leq n \leq 9$). Problem i was solved by exactly a_i participants ($0 \leq a_i \leq n$). A participant is *weak* if they did not solve all three problems. Over all scoreboards consistent with a , find the minimum possible number of weak participants. ($t \leq 3000$ test cases.)

Pattern Recognition

“Minimum number of people who miss *something*” — overlap the misses as much as possible. The answer is a lower bound that one construction meets.

Approach and intuition

Problem i is *missed* by $n - a_i$ participants, and each of them is weak. Hence at least $\max_i(n - a_i)$ participants are weak. To achieve this, number the participants $1..n$ and let the misses of every problem fall on the *first* participants: problem i is missed by participants $1, \dots, n - a_i$. Then exactly participants $1, \dots, \max_i(n - a_i)$ miss something.

Lower Bound Meets Construction

$$\text{answer} = \max(n - a_1, n - a_2, n - a_3).$$

Algorithm

```

1: for each test case do
2:   print max( $n - a_1, n - a_2, n - a_3$ )
3: end for

```

Dry Run. $n = 6, a = (4, 3, 2)$: misses are 2, 3, 4; stacking them on participants 1..4 gives 4 weak participants. $n = 5, a = (0, 5, 5)$: all 5 miss problem 1, answer 5. ✓

C++17 implementation

```

1 // Codeforces 2266A - Good Contest
2 // Stack every "miss" of every problem on the same people:
3 // the weak count is forced to be at least  $n - a_i$  for each problem  $i$ .
4 #include <algorithm>
5 #include <iostream>
6 using namespace std;
7
8 int minWeak(int n, int a1, int a2, int a3) {
9     return max({n - a1, n - a2, n - a3});
10 }
11
12 int main() {

```

```

13 ios::sync_with_stdio(false);
14 cin.tie(nullptr);
15 int t;
16 cin >> t;
17 while (t--) {
18     int n, a1, a2, a3;
19     cin >> n >> a1 >> a2 >> a3;
20     cout << minWeak(n, a1, a2, a3) << '\n';
21 }
22 return 0;
23 }

```

Complexity Analysis

Time: $O(1)$ per test. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- Do *not* add the misses: $\sum(n - a_i)$ counts a participant who misses two problems twice.
- $a_i = n$ for all i gives 0.

1.4 Boss Fight

Source: Codeforces 2252A (Notion: Greedy, 800)

Level: Easy

Problem Statement. You have $n \leq 50$ spell cards; card i deals a_i damage ($1 \leq a_i \leq 1000$). You may play them in any order. If you ever play two cards of *equal* damage consecutively, a shield activates: the triggering card still deals its damage, but every later card deals 0. Find the maximum total damage.

Pattern Recognition

“Arrange items so that no two equal items are adjacent” is possible iff the most frequent value v with frequency f satisfies $f \leq (n - f) + 1$. When it fails, the dominant value is the only obstacle.

Approach and intuition

Let v be the most frequent damage value, f its frequency, $r = n - f$ the number of other cards and R their total damage.

- If $f \leq r + 1$ we can interleave $(v x v x \dots)$ so no two equal cards touch: every card counts, $\text{answer} = \sum a_i$.
- Otherwise play $v, x_1, v, x_2, \dots, x_r, v$ (all r others and $r + 1$ copies of v , no equal neighbours) and then one more v as the trigger. $\text{Damage} = R + (r + 2)v$.

Why $r + 2$ Copies Is the Maximum

Before the trigger, the played cards have no equal neighbours, so among them at most $(\text{number of other cards}) + 1 \leq r + 1$ are copies of v . The trigger adds one more. No other

card is ever lost, so the construction is optimal:

$$\text{answer} = \begin{cases} \sum a_i & f \leq r + 1 \\ R + \min(f, r + 2) \cdot v & \text{otherwise.} \end{cases}$$

(If a different value triggered the shield we would only lose more copies of v .)

Algorithm

- 1: count frequencies; let v be a value of maximum frequency f
- 2: $r \leftarrow n - f$, $R \leftarrow \sum a - f \cdot v$
- 3: **if** $f \leq r + 1$ **then return** $\sum a$
- 4: **else return** $R + \min(f, r + 2) \cdot v$
- 5: **end if**

Dry Run. $a = [10, 5, 10, 10]$: $v = 10$, $f = 3$, $r = 1$, $3 > 2$, so answer $5 + 3 \cdot 10 = 35$ (order 10, 5, 10, 10). $a = [7^6]$: $r = 0$, answer $0 + 2 \cdot 7 = 14$. ✓

C++17 implementation

```

1 // Codeforces 2252A - Boss Fight
2 // If the most frequent value v fits (f <= others + 1) we can play everything
3 // with no equal neighbours. Otherwise the best we can do is
4 // v x v x ... v v -> all others plus (others + 2) copies of v.
5 #include <algorithm>
6 #include <iostream>
7 #include <map>
8 #include <vector>
9 using namespace std;
10
11 long long maxDamage(const vector<int>& a) {
12     map<int, int> freq;
13     long long total = 0;
14     for (int x : a) { freq[x]++; total += x; }
15     int v = 0, f = 0;
16     for (auto [val, cnt] : freq)
17         if (cnt > f) { f = cnt; v = val; }
18     int others = (int)a.size() - f;
19     if (f <= others + 1) return total; // perfect alternation
20     long long restSum = total - 1LL * v * f;
21     return restSum + 1LL * min(f, others + 2) * v; // alternate, then trigger
22 }
23
24 int main() {
25     ios::sync_with_stdio(false);
26     cin.tie(nullptr);
27     int t;
28     cin >> t;
29     while (t--) {
30         int n;
31         cin >> n;
32         vector<int> a(n);
33         for (int& x : a) cin >> x;
34         cout << maxDamage(a) << '\n';
35     }

```

```

36     return 0;
37 }

```

Complexity Analysis

Time: $O(n \log n)$ per test (map of frequencies). **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- $n = 1$: $f = 1 \leq 0 + 1$, the single card counts.
- The trigger card *does* deal damage — forgetting it gives $R + (r + 1)v$.

1.5 Flee to Shelter

Source: ICPC 2013–14 Amritapuri Online Round (PYQ)

Level: Easy

Problem Statement. Dr. Stone is at the shuttle and must bring N instruments from the telescope to the shuttle. She can carry at most M instruments per trip, and a one-way walk between the two takes T minutes. Find the minimum total time ($1 \leq N, M, T \leq 100$, up to 1000 tests).

Pattern Recognition

Trips of fixed cost with a capacity limit: always fill the capacity. Number of trips = $\lceil N/M \rceil$.

Approach and intuition

Every trip consists of walking to the telescope (T) and back loaded (T). A trip carries at most M instruments, so at least $\lceil N/M \rceil$ trips are needed; carrying M each time (and the remainder last) achieves that. She starts at the shuttle and must end there, so the answer is

$$2T \left\lceil \frac{N}{M} \right\rceil.$$

Algorithm

- 1: $\text{trips} \leftarrow (N + M - 1) \text{ div } M$ ▷ integer ceiling
- 2: **return** $2 \cdot \text{trips} \cdot T$

Dry Run. $N = 3, M = 2, T = 10$: $\lceil 3/2 \rceil = 2$ trips, $2 \cdot 2 \cdot 10 = 40$. ✓

C++17 implementation

```

1 // ICPC 2013-14 Amritapuri Online - Flee to Shelter
2 // Carry the maximum M each time: ceil(N / M) round trips of 2T minutes.
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9     int c;
10    cin >> c;
11    while (c--) {
12        long long n, m, t;

```

```

13     cin >> n >> m >> t;
14     long long trips = (n + m - 1) / m;    // ceil(n / m)
15     cout << 2 * trips * t << '\n';
16 }
17 return 0;
18 }

```

Complexity Analysis

Time: $O(1)$ per test. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- Use integer ceiling $(N + M - 1)/M$; floating `ceil` is unnecessary.
- She starts at the *shuttle*, so even the first trip costs $2T$ (walk out, walk back).

1.6 Ordering Teams

Source: ICPC 2017–18 Online Preliminary, Kharagpur (PYQ, ORDTEAMS)

Level: Easy

Problem Statement. Three team members have scores (s_1, s_2, s_3) in three skills. Person x is *better* than y if x scores no less than y in every skill and strictly more in at least one. Decide whether the three can be ordered so that each person is strictly better than the previous one.

Pattern Recognition

A dominance chain is increasing in *every* coordinate, hence in the coordinate sum. Sorting by the sum fixes the only possible order.

Approach and intuition

If x is better than y then $\text{sum}(x) > \text{sum}(y)$ (no coordinate drops, one rises). So in any valid chain the sums strictly increase, which means the chain must be the order by increasing sum. Sort the three people by their sum and check the two consecutive relations. With ties in the sum, the two tied people cannot be ordered, and the check fails automatically.

Algorithm

- 1: sort the three people by $s_1 + s_2 + s_3$
- 2: **return** `BETTER( $p_2, p_1$ ) and BETTER( $p_3, p_2$ )`

Dry Run. Case 1: sums 6, 9, 10; (2, 3, 4) beats (1, 2, 3) and (2, 3, 5) beats (2, 3, 4) $\Rightarrow$ yes. Case 2: (2, 3, 4) appears twice, not strictly better $\Rightarrow$ no. ✓

C++17 implementation

```

1 // ICPC 2017-18 Online Preliminary - Ordering teams (ORDTEAMS)
2 // Sort by total score; any valid chain must be in increasing-sum order,
3 // so we only need to check consecutive pairs of that order.
4 #include <algorithm>
5 #include <array>
6 #include <iostream>
7 using namespace std;
8

```

```

9  using Person = array<int, 3>;
10
11  bool strictlyBetter(const Person& x, const Person& y) {    // x better than y
12      bool some = false;
13      for (int k = 0; k < 3; k++) {
14          if (x[k] < y[k]) return false;
15          if (x[k] > y[k]) some = true;
16      }
17      return some;
18  }
19
20  int main() {
21      ios::sync_with_stdio(false);
22      cin.tie(nullptr);
23      int t;
24      cin >> t;
25      while (t--) {
26          array<Person, 3> p;
27          for (auto& q : p) cin >> q[0] >> q[1] >> q[2];
28          sort(p.begin(), p.end(), [](const Person& a, const Person& b) {
29              return a[0] + a[1] + a[2] < b[0] + b[1] + b[2];
30          });
31          bool ok = strictlyBetter(p[1], p[0]) && strictlyBetter(p[2], p[1]);
32          cout << (ok ? "yes" : "no") << '\n';
33      }
34      return 0;
35  }

```

Complexity Analysis

Time: $O(1)$ per test. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- “Better” needs a *strict* improvement somewhere: identical people cannot be chained.
- Output is lowercase yes/no.

1.7 Stick Lengths

Source: CSES 1074 (Notion: Greedy)

Level: Easy

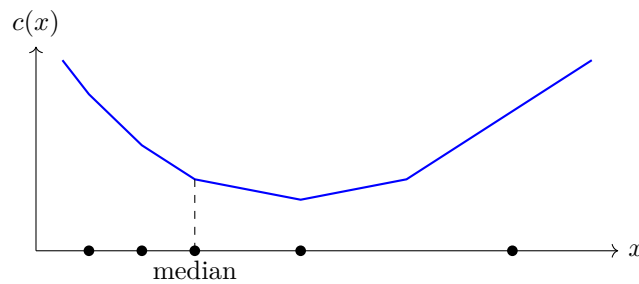
Problem Statement. There are $n \leq 2 \cdot 10^5$ sticks of lengths $p_i \leq 10^9$. Changing a stick's length by x costs x . Make all sticks equal with minimum total cost.

Pattern Recognition

Minimise $\sum |p_i - x|$ over x : the answer is at a **median**. (Minimising $\sum (p_i - x)^2$ would give the mean instead.)

Approach and intuition

Let $c(x) = \sum_i |p_i - x|$. Moving x one unit to the right changes the cost by (number of sticks $\leq x$) $-$ (number of sticks $> x$). While fewer than half of the sticks are to the left, moving right helps; once at least half are to the left, it hurts. The turning point is the median.



c is convex and piecewise linear with slope $\#\{p_i < x\} - \#\{p_i > x\}$; it is minimal where the slope changes sign.

Algorithm

- 1: $m \leftarrow$ the element of rank $\lfloor n/2 \rfloor$ (`nth_element`)
- 2: **return** $\sum_i |p_i - m|$

Dry Run. $p = [2, 3, 1, 5, 2]$, sorted $1, 2, 2, 3, 5$, median 2 . Cost $0 + 1 + 1 + 3 + 0 = 5$. ✓

C++17 implementation

```

1 // CSES 1074 - Stick Lengths
2 // sum |p_i - x| is minimised at a median of p.
3 #include <algorithm>
4 #include <iostream>
5 #include <vector>
6 using namespace std;
7
8 int main() {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11    int n;
12    cin >> n;
13    vector<long long> p(n);
14    for (auto& x : p) cin >> x;
15    nth_element(p.begin(), p.begin() + n / 2, p.end());
16    long long med = p[n / 2], cost = 0;
17    for (long long x : p) cost += (x > med ? x - med : med - x);
18    cout << cost << '\n';
19    return 0;
20 }
```

Complexity Analysis

Time: $O(n)$ expected (`nth_element`). **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- The cost can reach $2 \cdot 10^5 \cdot 10^9 = 2 \cdot 10^{14}$: use `long long`.
- For even n any value between the two middle elements is optimal; picking either middle element is fine.

1.8 Movie Festival

Source: CSES 1629 (Notion: Greedy)

Level: Easy

Problem Statement. $n \leq 2 \cdot 10^5$ movies are shown; movie i runs from a_i to b_i ($a_i < b_i \leq 10^9$). What is the maximum number of movies you can watch completely? (A movie may start exactly when the previous ends.)

Pattern Recognition

Interval scheduling / activity selection: **pick the interval that ends first**, discard everything overlapping it, repeat.

Approach and intuition

Sort by end time. Keep `freeAt`, the end of the last chosen movie. Scan the movies; take a movie whenever it starts at or after `freeAt`.

Exchange Argument

Let g be the movie that ends first and O an optimal schedule whose first movie is $o_1 \neq g$. Since $b_g \leq b_{o_1}$, replacing o_1 by g keeps O feasible (everything after o_1 starts at or after $b_{o_1} \geq b_g$) and of the same size. So some optimal schedule starts with g ; recurse on the movies starting after b_g . (See the picture in Section 1.1.)

Algorithm

```

1: sort movies by end time
2:  $cnt \leftarrow 0$ ,  $freeAt \leftarrow 0$ 
3: for each movie  $(a, b)$  in that order do
4:   if  $a \geq freeAt$  then
5:      $cnt \leftarrow cnt + 1$ ,  $freeAt \leftarrow b$ 
6:   end if
7: end for
8: return  $cnt$ 

```

Dry Run. Movies $(3, 5), (4, 9), (5, 8)$; by end: $(3, 5), (5, 8), (4, 9)$. Take $(3, 5)$; $(5, 8)$ starts at $5 \geq 5$, take; $(4, 9)$ starts at $4 < 8$, skip. Answer 2. ✓

C++17 implementation

```

1 // CSES 1629 - Movie Festival
2 // Classic interval scheduling: always take the movie that ends first.
3 #include <algorithm>
4 #include <iostream>
5 #include <utility>
6 #include <vector>
7 using namespace std;
8
9 int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12     int n;
13     cin >> n;
14     vector<pair<int, int>> mv(n); // (end, start)
15     for (auto& [e, s] : mv) cin >> s >> e;

```

```

16  sort(mv.begin(), mv.end());
17  int watched = 0, freeAt = 0;
18  for (auto [e, s] : mv)
19      if (s >= freeAt) { watched++; freeAt = e; }
20  cout << watched << '\n';
21  return 0;
22  }

```

Complexity Analysis

Time: $O(n \log n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- Sorting by *start* time or by *length* is wrong (a long early movie, or a short movie overlapping two others).
- Use $a \geq \text{freeAt}$, not $>$: back-to-back movies are allowed.

1.9 Restaurant Customers

Source: CSES 1619 (Notion: Greedy)

Level: Easy

Problem Statement. Customer i is in the restaurant from time a_i to b_i ($1 \leq a_i < b_i \leq 10^9$, all $2n$ times distinct, $n \leq 2 \cdot 10^5$). Find the maximum number of customers present at the same time.

Pattern Recognition

“Maximum overlap of intervals” $\Rightarrow$ **sweep line**: an arrival is a $+1$ event, a departure a -1 event; the answer is the maximum prefix sum in time order.

Approach and intuition

The number of people present only changes at event times. Sorting the $2n$ events and accumulating ± 1 reproduces the head count at every moment; its maximum is the answer. Because times are distinct we never have to decide whether a departure and an arrival at the same instant overlap.

Algorithm

- 1: events $\leftarrow \{(a_i, +1), (b_i, -1)\}$, sort by time
- 2: $cur \leftarrow 0$, $best \leftarrow 0$
- 3: **for** each event (t, d) **do**
- 4: $cur \leftarrow cur + d$, $best \leftarrow \max(best, cur)$
- 5: **end for**
- 6: **return** $best$

Dry Run. Intervals $(5, 8), (2, 4), (3, 9)$: events $2^+, 3^+, 4^-, 5^+, 8^-, 9^-$, counts $1, 2, 1, 2, 1, 0$. Maximum 2. ✓

C++17 implementation

```

1 // CSES 1619 - Restaurant Customers
2 // Sweep line over +1 (arrival) / -1 (leave) events.
3 #include <algorithm>
4 #include <iostream>
5 #include <utility>
6 #include <vector>
7 using namespace std;
8
9 int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12     int n;
13     cin >> n;
14     vector<pair<int, int>> ev;
15     ev.reserve(2 * n);
16     for (int i = 0; i < n; i++) {
17         int a, b;
18         cin >> a >> b;
19         ev.push_back({a, +1});
20         ev.push_back({b, -1});
21     }
22     sort(ev.begin(), ev.end());           // all times are distinct
23     int cur = 0, best = 0;
24     for (auto [t, d] : ev) best = max(best, cur += d);
25     cout << best << '\n';
26     return 0;
27 }

```

Complexity Analysis

Time: $O(n \log n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- If times could coincide you must decide the tie order (process -1 before $+1$ for half-open intervals). Here they are distinct.
- Coordinates up to 10^9 : never allocate an array indexed by time.

1.10 Tasks and Deadlines

Source: CSES 1630 (Notion: Greedy)

Level: Easy

Problem Statement. You must process all $n \leq 2 \cdot 10^5$ tasks one after another starting at time 0. Task i has duration a_i and deadline d_i ; finishing it at time f_i earns $d_i - f_i$ (possibly negative). Maximise the total reward.

Pattern Recognition

The objective splits into a fixed part ($\sum d_i$) and a part depending on the order ($-\sum f_i$). Minimising the sum of completion times is **Shortest Processing Time first**.

Approach and intuition

$\sum (d_i - f_i) = \sum d_i - \sum f_i$. The deadlines do not depend on the order, so we only need to minimise $\sum f_i$. If task x is right before task y and $a_x > a_y$, swapping them does not change

when anything else finishes, while the two completion times change from $(t + a_x) + (t + a_x + a_y)$ to $(t + a_y) + (t + a_x + a_y)$ — a decrease of $a_x - a_y > 0$. So in an optimal order durations are non-decreasing.

Algorithm

```

1: sort tasks by duration
2:  $now \leftarrow 0, rew \leftarrow 0$ 
3: for each  $(a, d)$  do
4:    $now \leftarrow now + a; rew \leftarrow rew + d - now$ 
5: end for
6: return  $rew$ 

```

Dry Run. Tasks (6, 10), (8, 15), (5, 12) sorted by a : (5, 12), (6, 10), (8, 15). Finish times 5, 11, 19; rewards 7, -1, -4; total 2. ✓

C++17 implementation

```

1 // CSES 1630 - Tasks and Deadlines
2 // Reward = sum d - sum f. sum d is fixed; minimise sum of finishing
3 // times with Shortest-Processing-Time-first (exchange argument).
4 #include <algorithm>
5 #include <iostream>
6 #include <utility>
7 #include <vector>
8 using namespace std;
9
10 int main() {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13     int n;
14     cin >> n;
15     vector<pair<long long, long long>> t(n); // (duration, deadline)
16     for (auto& [a, d] : t) cin >> a >> d;
17     sort(t.begin(), t.end());
18     long long now = 0, reward = 0;
19     for (auto [a, d] : t) {
20         now += a;
21         reward += d - now;
22     }
23     cout << reward << '\n';
24     return 0;
25 }

```

Complexity Analysis

Time: $O(n \log n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- Sorting by deadline is the classic wrong answer — deadlines do not influence the optimal order at all.
- $\sum f_i$ reaches about $2 \cdot 10^5 \cdot 2 \cdot 10^{11}$: use `long long`.

1.11 Reading Books

Source: CSES 1631 (Notion: Greedy)

Level: Easy–Medium

Problem Statement. Two people must each read all n books; book i takes t_i time. Nobody can read two books at once and a book cannot be read by both at the same time. Find the minimum time until both have read all books.

Pattern Recognition

Two lower bounds, one construction: each reader needs $S = \sum t_i$, and the longest book M must be read twice without overlap.

Approach and intuition

Clearly the answer is at least S (each person reads everything) and at least $2M$ (the two readings of the longest book cannot overlap, and each reader is busy with it for M).

- If $M \geq S - M$: reader 1 reads the long book while reader 2 reads all the others (they fit in time $S - M \leq M$); then they swap. Total $2M$.
- If $M < S - M$: arrange the books in a circle and let reader 2 start with the book “half a turn” ahead; since no book is longer than the rest, the two readers never meet on the same book. Total S .

So the answer is $\max(S, 2M)$.

Algorithm

- 1: $S \leftarrow \sum t_i$, $M \leftarrow \max t_i$
- 2: **return** $\max(S, 2M)$

Dry Run. $t = [2, 8, 3]$: $S = 13$, $M = 8$, $2M = 16 > 13$. Answer 16: reader 1 reads 8 while reader 2 reads 2, 3 (time 5), then they swap. ✓

C++17 implementation

```

1 // CSES 1631 - Reading Books
2 // Answer = max(sum, 2 * longest book).
3 #include <algorithm>
4 #include <iostream>
5 using namespace std;
6
7 int main() {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10    int n;
11    cin >> n;
12    long long sum = 0, mx = 0;
13    for (int i = 0; i < n; i++) {
14        long long t;
15        cin >> t;
16        sum += t;
17        mx = max(mx, t);
18    }
19    cout << max(sum, 2 * mx) << '\n';
20    return 0;
21 }
```

Complexity Analysis**Time:** $O(n)$. **Space:** $O(1)$.**Common Pitfalls and Edge Cases**

- $n = 1$: $S = M$, answer $2M$.
- S reaches $2 \cdot 10^{14}$: long long.

1.12 Creating a Wormhole

Source: ICPC 2013–14 Amritapuri Online Round (PYQ)**Level:** Easy–Medium

Problem Statement. You have N rods ($3 \leq N \leq 10^4$, lengths $a_i \leq 10^9$). Using some of them end to end, form a closed polygon of non-zero area. Output the maximum number of rods that can be used, or -1 if no polygon is possible.

Pattern Recognition

Polygon inequality: a set of sides forms a non-degenerate polygon iff **the longest side is strictly shorter than the sum of the others**. Fix the longest side, then take everything shorter.

Approach and intuition

Sort the rods: $a_0 \leq a_1 \leq \dots \leq a_{N-1}$. Suppose the longest rod used is a_j . Adding any rod shorter than (or equal in index below) a_j only *increases* the sum of the other sides and keeps a_j the longest, so the best set with maximum a_j is $\{a_0, \dots, a_j\}$, which is valid iff

$$a_0 + a_1 + \dots + a_{j-1} > a_j.$$

The answer is the largest $j + 1$ ($j \geq 2$) satisfying this, or -1 .

Algorithm

```

1: sort  $a$ ;  $pre \leftarrow 0$ ;  $best \leftarrow -1$ 
2: for  $j = 0$  to  $N - 1$  do
3:   if  $j \geq 2$  and  $pre > a_j$  then
4:      $best \leftarrow j + 1$ 
5:   end if
6:    $pre \leftarrow pre + a_j$ 
7: end for
8: return  $best$ 

```

Dry Run. $a = [1, 12, 3, 2, 5, 1, 50]$, sorted $1, 1, 2, 3, 5, 12, 50$. $j = 2$: $2 < 2$? no. $j = 3$: $4 > 3$ yes $\rightarrow 3$. $j = 4$: $7 > 5 \rightarrow 5$. $j = 5$: $12 > 12$? no. $j = 6$: $24 > 50$? no. Answer 5. $[1, 1, 2]$: $2 > 2$ fails, -1 . ✓

C++17 implementation

```

1 // ICPC 2013-14 Amritapuri Online - Creating a Wormhole
2 // Polygon exists iff longest side < sum of the others. For a fixed longest
3 // stick a[j] (sorted) it is best to take every shorter stick too.
4 #include <algorithm>

```

```

5 #include <iostream>
6 #include <vector>
7 using namespace std;
8
9 int maxSticks(vector<long long> a) {
10     sort(a.begin(), a.end());
11     long long prefix = 0;           // sum of a[0..j-1]
12     int best = -1;
13     for (int j = 0; j < (int)a.size(); j++) {
14         if (j >= 2 && prefix > a[j]) best = j + 1;
15         prefix += a[j];
16     }
17     return best;
18 }
19
20 int main() {
21     ios::sync_with_stdio(false);
22     cin.tie(nullptr);
23     int t;
24     cin >> t;
25     while (t--) {
26         int n;
27         cin >> n;
28         vector<long long> a(n);
29         for (auto& x : a) cin >> x;
30         cout << maxSticks(a) << '\n';
31     }
32     return 0;
33 }

```

Complexity Analysis

Time: $O(N \log N)$. **Space:** $O(N)$.

Common Pitfalls and Edge Cases

- Strict inequality: $1 + 1 = 2$ gives a degenerate (zero-area) triangle.
- Do not stop at the first failure — a later, longer prefix can become valid again (e.g. 1, 1, 2, 3, 5 fails at 2 but succeeds at 3).
- Prefix sums reach 10^{13} : `long long`.

1.13 Concert Tickets

Source: CSES 1091 (Notion: Greedy)

Level: Easy–Medium

Problem Statement. There are n tickets with prices h_i . Customers arrive one by one; customer j buys the most expensive remaining ticket whose price does not exceed t_j , or gets nothing (-1). Print what each customer pays ($n, m \leq 2 \cdot 10^5$).

Pattern Recognition

“Largest remaining element $\leq x$, then delete it” $\Rightarrow$ `multiset` with `upper_bound` and one step back.

Approach and intuition

The rule is given by the statement, so the whole task is choosing a data structure supporting *predecessor query* and *delete* in $O(\log n)$ with duplicates: a `std::multiset`. `upper_bound(t)` returns the first price $> t$; the element just before it (if any) is the largest price $\leq t$.

Algorithm

```

1: insert all prices into multiset  $S$ 
2: for each budget  $t$  do
3:    $it \leftarrow S.upper\_bound(t)$ 
4:   if  $it = S.begin()$  then
5:     print  $-1$ 
6:   else
7:      $it \leftarrow it - 1$ ; print  $*it$ ; erase  $it$ 
8:   end if
9: end for

```

Dry Run. Prices $\{3, 5, 5, 7, 8\}$. $t = 4$: pays 3. $t = 8$: pays 8. $t = 3$: nothing ≤ 3 left, -1 . ✓

C++17 implementation

```

1  // CSES 1091 - Concert Tickets
2  // Each customer takes the most expensive ticket <= their budget.
3  #include <iostream>
4  #include <iterator>
5  #include <set>
6  using namespace std;
7
8  int main() {
9      ios::sync_with_stdio(false);
10     cin.tie(nullptr);
11     int n, m;
12     cin >> n >> m;
13     multiset<int> tickets;
14     for (int i = 0; i < n; i++) {
15         int h;
16         cin >> h;
17         tickets.insert(h);
18     }
19     for (int i = 0; i < m; i++) {
20         int t;
21         cin >> t;
22         auto it = tickets.upper_bound(t);           // first price > t
23         if (it == tickets.begin()) {
24             cout << -1 << '\n';
25         } else {
26             --it;                                   // largest price <= t
27             cout << *it << '\n';
28             tickets.erase(it);
29         }
30     }
31     return 0;
32 }

```

Complexity Analysis**Time:** $O((n + m) \log n)$. **Space:** $O(n)$.**Common Pitfalls and Edge Cases**

- `S.erase(value)` would delete *all* copies of that price; erase the iterator.
- `lower_bound(t)` is off by one for equal prices.

1.14 Towers

Source: CSES 1073 (Notion: Greedy)**Level:** Medium

Problem Statement. Cubes of sizes $k_1, \dots, k_n$ arrive in order ($n \leq 2 \cdot 10^5$). Each cube is placed on top of an existing tower whose top cube is strictly larger, or starts a new tower. Minimise the number of towers.

Pattern Recognition

“Place each item on the pile whose top is the smallest value still larger than it” — patience sorting. The number of piles equals the length of the longest non-decreasing subsequence.

Approach and intuition

Keep the multiset of tower tops. For a new cube k , put it on the tower with the *smallest* top $> k$; if none exists, start a new tower.

Lower Bound Meets Greedy

Lower bound. If $i < j$ and $k_i \leq k_j$, cubes i and j can never share a tower (j would sit above i but is not smaller). So a non-decreasing subsequence of length L forces at least L towers.

Greedy matches it. The greedy keeps the tops sorted. When a cube k opens tower number m , every existing top is $\leq k$; link k to the current top of tower $m - 1$ (placed earlier, value $\leq k$). Following these links from the cube that opened the last tower yields a non-decreasing subsequence whose length equals the number of towers. Hence greedy = optimum.

Algorithm

```

1: tops ← empty multiset
2: for each cube  $k$  do
3:    $it \leftarrow tops.upper\_bound(k)$ 
4:   if  $it \neq tops.end()$  then erase  $it$ 
5:   end if
6:   insert  $k$ 
7: end for
8: return  $|tops|$ 

```

Dry Run. 3, 8, 2, 1, 5: tops $\{3\} \rightarrow \{3, 8\} \rightarrow \{2, 8\}$ (2 on 3) $\rightarrow \{1, 8\}$ (1 on 2) $\rightarrow \{1, 5\}$ (5 on 8). Two towers. ✓

C++17 implementation

```

1 // CSES 1073 - Towers
2 // Put each cube on the tower with the smallest top strictly greater than it.
3 #include <iostream>
4 #include <set>
5 using namespace std;
6
7 int main() {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10    int n;
11    cin >> n;
12    multiset<int> tops;
13    for (int i = 0; i < n; i++) {
14        int k;
15        cin >> k;
16        auto it = tops.upper_bound(k);
17        if (it != tops.end()) tops.erase(it); // place k on that tower
18        tops.insert(k);
19    }
20    cout << tops.size() << '\n';
21    return 0;
22 }

```

Complexity Analysis

Time: $O(n \log n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- The upper cube must be *strictly* smaller: equal sizes need a new tower, hence `upper_bound`, not `lower_bound`.
- The `tops` multiset is always sorted, so a plain `vector` with binary search also works (like LIS).

1.15 Missing Coin Sum

Source: CSES 2183 (Notion: Greedy)

Level: Medium

Problem Statement. You have $n \leq 2 \cdot 10^5$ coins with positive values $x_i \leq 10^9$. Find the smallest positive sum that *cannot* be formed by a subset of the coins.

Pattern Recognition

Maintain a reachable range $[0, \text{reach})$ while adding items in increasing order — a classic invariant greedy.

Approach and intuition

Sort the coins. **Invariant:** after processing some prefix, every sum in $[0, r)$ is makeable and r is not. Initially $r = 1$ (only 0 is makeable). Take the next coin x :

- If $x \leq r$: sums $[0, r)$ plus x give $[x, x+r)$, which overlaps $[0, r)$, so now $[0, r+x)$ is makeable: $r \leftarrow r + x$.

- If $x > r$: this coin and all later ones exceed r , and the coins so far cannot make r . So r is the answer.

Algorithm

```

1: sort  $x$ ;  $r \leftarrow 1$ 
2: for each  $v$  in  $x$  do
3:   if  $v > r$  then break
4:   end if
5:    $r \leftarrow r + v$ 
6: end for
7: return  $r$ 

```

Dry Run. Coins 2, 9, 1, 2, 7 sorted 1, 2, 2, 7, 9: $r = 1 \rightarrow 2 \rightarrow 4 \rightarrow 6$; next coin 7 > 6, stop. Answer 6. ✓

C++17 implementation

```

1 // CSES 2183 - Missing Coin Sum
2 // Invariant: after the first  $i$  sorted coins, every sum in  $[0, reach)$  is
3 // makeable. A coin  $x \leq reach$  extends it to  $reach + x$ ;  $x > reach$  leaves a gap.
4 #include <algorithm>
5 #include <iostream>
6 #include <vector>
7 using namespace std;
8
9 int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12     int n;
13     cin >> n;
14     vector<long long> x(n);
15     for (auto& v : x) cin >> v;
16     sort(x.begin(), x.end());
17     long long reach = 1; // smallest sum not yet makeable
18     for (long long v : x) {
19         if (v > reach) break;
20         reach += v;
21     }
22     cout << reach << '\n';
23     return 0;
24 }

```

Complexity Analysis

Time: $O(n \log n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- Start with $r = 1$, not 0.
- r can reach $2 \cdot 10^{14} + 1$: long long.
- If no coin is 1, the answer is 1 immediately.

1.16 Playlist

Source: CSES 1141 (Notion: Greedy)

Level: Easy–Medium

Problem Statement. Given a sequence of $n \leq 2 \cdot 10^5$ song ids, find the length of the longest contiguous block in which all songs are distinct.

Pattern Recognition

Longest subarray with a property that is preserved by shrinking $\Rightarrow$ **two pointers**: the left end never moves back.

Approach and intuition

For each right end r , the best left end $\ell(r)$ is the smallest index such that $k[\ell..r]$ has no repeats. When we extend to r , the only possible repeat is $k[r]$; if its previous occurrence is at $p \geq \ell$, the window must start at $p + 1$. Since ℓ only moves right, the total work is linear.

Algorithm

```

1:  $\ell \leftarrow 0$ ,  $best \leftarrow 0$ ,  $last \leftarrow$  empty map
2: for  $r = 0$  to  $n - 1$  do
3:   if  $k_r \in last$  and  $last[k_r] \geq \ell$  then
4:      $\ell \leftarrow last[k_r] + 1$ 
5:   end if
6:    $last[k_r] \leftarrow r$ ;  $best \leftarrow \max(best, r - \ell + 1)$ 
7: end for

```

Dry Run. 1, 2, 1, 3, 2, 7, 4, 2: windows end with lengths 1, 2, 2, 3, 3, 4, 5, 3 (at $r = 6$ the window is 1, 3, 2, 7, 4). Answer 5. ✓

C++17 implementation

```

1  // CSES 1141 - Playlist
2  // Two pointers: the window [l, r] never contains a repeated song.
3  #include <algorithm>
4  #include <iostream>
5  #include <unordered_map>
6  #include <vector>
7  using namespace std;
8
9  int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12     int n;
13     cin >> n;
14     vector<int> k(n);
15     for (auto& v : k) cin >> v;
16     unordered_map<int, int> last;           // song -> last index seen
17     last.reserve(2 * n);
18     int best = 0, l = 0;
19     for (int r = 0; r < n; r++) {
20         auto it = last.find(k[r]);
21         if (it != last.end() && it->second >= l) l = it->second + 1;
22         last[k[r]] = r;
23         best = max(best, r - l + 1);
24     }

```

```

25     cout << best << '\n';
26     return 0;
27 }

```

Complexity Analysis

Time: $O(n)$ expected (hash map). **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- Check $last[k_r] \geq \ell$: an occurrence left of the window is irrelevant; without the check ℓ could move *left*.
- Anti-hash tests exist on Codeforces; coordinate compression or a `map` is the safe alternative.

1.17 Traffic Lights

Source: CSES 1163 (Notion: Greedy)

Level: Medium

Problem Statement. A street spans positions $0..x$. Traffic lights are added at distinct positions $p_1, \dots, p_n$ ($0 < p_i < x \leq 10^9$, $n \leq 2 \cdot 10^5$). After each addition print the length of the longest segment without traffic lights.

Pattern Recognition

Online “split a segment and report the maximum” $\Rightarrow$ an ordered **set** of cut points to find the segment being split, and a **multiset** of lengths to report the maximum.

Approach and intuition

Adding a light at p splits exactly one segment $[lo, hi]$ with $lo < p < hi$: its neighbours in the set of cuts. Remove length $hi - lo$ from the multiset and insert $p - lo$ and $hi - p$. The maximum is the last element of the multiset.

Algorithm

- 1: $cuts \leftarrow \{0, x\}$, $lens \leftarrow \{x\}$
- 2: **for** each p **do**
- 3: $hi \leftarrow cuts.upper_bound(p)$, $lo \leftarrow$ predecessor of hi
- 4: remove one copy of $hi - lo$; insert $p - lo$, $hi - p$; insert p into $cuts$
- 5: print $\max(lens)$
- 6: **end for**

Dry Run. $x = 8$, lights 3, 6, 2: $\{8\} \rightarrow \{3, 5\}$ max 5; $\rightarrow \{3, 3, 2\}$ max 3; $\rightarrow \{2, 1, 3, 2\}$ max 3. Output 5 3 3. ✓

C++17 implementation

```

1 // CSES 1163 - Traffic Lights
2 // Keep the cut positions in a set and the segment lengths in a multiset.
3 #include <iostream>
4 #include <iterator>
5 #include <set>
6 using namespace std;

```

```

7
8 int main() {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11    int x, n;
12    cin >> x >> n;
13    set<int> cuts = {0, x};
14    multiset<int> lens = {x};
15    for (int i = 0; i < n; i++) {
16        int p;
17        cin >> p;
18        auto hi = cuts.upper_bound(p);
19        auto lo = prev(hi);
20        lens.erase(lens.find(*hi - *lo)); // split [lo, hi] at p
21        lens.insert(p - *lo);
22        lens.insert(*hi - p);
23        cuts.insert(p);
24        cout << *lens.rbegin() << (i + 1 < n ? ' ' : '\n');
25    }
26    return 0;
27 }

```

Complexity Analysis

Time: $O(n \log n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- `lens.erase(value)` removes *all* equal lengths; use `lens.erase(lens.find(value))`.
- Offline alternative: add all lights, then remove them in reverse order (merging segments only increases the maximum).

1.18 Room Allocation

Source: CSES 1164 (Notion: Greedy)

Level: Medium

Problem Statement. $n \leq 2 \cdot 10^5$ customers stay from day a_i to day b_i inclusive. Two customers can share a room if the first leaves strictly before the second arrives. Find the minimum number of rooms and a valid assignment.

Pattern Recognition

Interval partitioning: process by start time; reuse the resource that becomes free the earliest (min-heap keyed by end time).

Approach and intuition

Sort customers by arrival. Keep a min-heap of occupied rooms keyed by departure day. For the next customer, if the room with the earliest departure is free (departure < arrival) reuse it; otherwise open a new room.

Why This Is Optimal

When a new room is opened for a customer arriving on day a , every existing room is occupied by someone who arrived no later than a and leaves on or after a . So at day a

all these guests plus the new one are present simultaneously: *any* allocation needs at least that many rooms. The greedy therefore uses exactly the maximum overlap, which is a lower bound.

Algorithm

```

1: sort customers by  $(a, b)$ ;  $H \leftarrow$  empty min-heap of  $(b, \text{room})$ 
2: for each customer  $(a, b, id)$  do
3:   if  $H$  non-empty and  $H.\text{top}.b < a$  then
4:      $\text{room}[id] \leftarrow H.\text{top}.\text{room}$ ; pop
5:   else
6:      $\text{room}[id] \leftarrow$  new room number
7:   end if
8:   push  $(b, \text{room}[id])$ 
9: end for

```

Dry Run. $(1, 2), (2, 4), (4, 4)$: first gets room 1; second arrives on day 2, but room 1 frees after day 2 ($2 \not< 2$), so room 2; third arrives on day 4, room 1 ($2 < 4$). Output 2 rooms, 1 2 1. ✓

C++17 implementation

```

1  // CSES 1164 - Room Allocation
2  // Process by arrival; reuse the room that frees up earliest if it is free.
3  #include <algorithm>
4  #include <functional>
5  #include <iostream>
6  #include <queue>
7  #include <tuple>
8  #include <utility>
9  #include <vector>
10 using namespace std;
11
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15     int n;
16     cin >> n;
17     vector<tuple<int, int, int>> c(n);    // (arrival, departure, index)
18     for (int i = 0; i < n; i++) {
19         auto& [a, b, id] = c[i];
20         cin >> a >> b;
21         id = i;
22     }
23     sort(c.begin(), c.end());
24     // min-heap of (departure day, room number)
25     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> busy;
26     vector<int> room(n);
27     int rooms = 0;
28     for (auto [a, b, id] : c) {
29         if (!busy.empty() && busy.top().first < a) {
30             room[id] = busy.top().second;    // reuse the earliest-free room
31             busy.pop();
32         } else {
33             room[id] = ++rooms;    // every room is occupied
34         }
35     }
36 }

```

```

35     busy.push({b, room[id]});
36 }
37 cout << rooms << '\n';
38 for (int i = 0; i < n; i++) cout << room[i] << (i + 1 < n ? ' ' : '\n');
39 return 0;
40 }

```

Complexity Analysis

Time: $O(n \log n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- The comparison is *strict*: $b < a$ (a guest leaving on day 2 still occupies the room on day 2).
- Remember original indices — the output is in input order.

1.19 Factory Machines

Source: CSES 1620 (Notion: Greedy)

Level: Medium

Problem Statement. $n \leq 2 \cdot 10^5$ machines work in parallel; machine i needs k_i seconds per product. Find the minimum time to make $t \leq 10^9$ products.

Pattern Recognition

“Minimum time such that we can finish” with a monotone feasibility test $\Rightarrow$ **binary search on the answer**. The greedy part is the test: in time T each machine simply works non-stop.

Approach and intuition

In time T , machine i makes $\lfloor T/k_i \rfloor$ products; running every machine all the time is clearly best, so $f(T) = \sum_i \lfloor T/k_i \rfloor$ products are possible. f is non-decreasing, so find the smallest T with $f(T) \geq t$. An upper bound is $T = t \cdot \min k_i$ (the fastest machine alone).

Algorithm

```

1: lo ← 0, hi ← t · min k
2: while lo < hi do
3:   mid ← ⌊(lo + hi)/2⌋
4:   if  $\sum \lfloor \text{mid} / k_i \rfloor \geq t$  then hi ← mid
5:   else lo ← mid + 1
6:   end if
7: end while
8: return lo

```

Dry Run. $k = (3, 2, 5)$, $t = 7$: $f(7) = 2 + 3 + 1 = 6 < 7$, $f(8) = 2 + 4 + 1 = 7 \geq 7$. Answer 8. ✓

C++17 implementation

```

1 // CSES 1620 - Factory Machines
2 // Binary search the answer T: in T seconds machine i makes floor(T / k_i).
3 #include <algorithm>

```

```

4 #include <iostream>
5 #include <vector>
6 using namespace std;
7
8 bool enough(const vector<long long>& k, long long T, long long t) {
9     long long made = 0;
10    for (long long v : k) {
11        made += T / v;
12        if (made >= t) return true;        // stop early, avoids overflow
13    }
14    return false;
15 }
16
17 int main() {
18     ios::sync_with_stdio(false);
19     cin.tie(nullptr);
20     int n;
21     long long t;
22     cin >> n >> t;
23     vector<long long> k(n);
24     for (auto& v : k) cin >> v;
25     long long lo = 0, hi = *min_element(k.begin(), k.end()) * t; // hi works
26     while (lo < hi) {
27         long long mid = lo + (hi - lo) / 2;
28         if (enough(k, mid, t)) hi = mid;
29         else lo = mid + 1;
30     }
31     cout << lo << '\n';
32     return 0;
33 }

```

Complexity Analysis

Time: $O(n \log(t \cdot \min k)) \approx n \cdot 60$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- $\sum \lfloor T/k_i \rfloor$ can overflow even `long long` when $T \approx 10^{18}$ and $k_i = 1$: stop summing as soon as the total reaches t .
- $hi = t \cdot \min k \leq 10^{18}$ fits in `long long`.

1.20 Tatar TV Show

Source: Codeforces 2236B (Notion: Greedy, 800)

Level: Easy–Medium

Problem Statement. A binary string s of length n and an integer k ($1 \leq k \leq n \leq 2 \cdot 10^5$) are given. One move picks i with $1 \leq i \leq n - k$ and flips both s_i and s_{i+k} . Can s become all zeros?

Pattern Recognition

An operation touching positions i and $i + k$ never mixes residues modulo $k \Rightarrow$ split into k independent chains. Flipping two cells preserves the **parity** of the number of ones.

Approach and intuition

Positions $r, r+k, r+2k, \dots$ form a chain, and a move flips two *adjacent* cells of one chain. Each move changes the number of ones in its chain by $-2, 0$ or $+2$, so the parity of each chain is invariant: an odd chain can never reach zero ones. Conversely, if a chain has an even number of ones, sweep it left to right: whenever the current cell is 1, flip it together with its successor. This pushes the ones to the end of the chain where, by parity, the last cell ends as 0.

Criterion

The answer is YES iff for every residue $r \in [0, k)$ the chain $\{i : i \equiv r \pmod{k}\}$ contains an even number of ones.

Algorithm

```

1: ones[r] ← 0 for r < k
2: for i = 0 to n - 1 do
3:   if si = 1 then ones[i mod k]++
4:   end if
5: end for
6: return all ones[r] even

```

Dry Run. $n = 4, k = 2$, 1010: chain $\{1, 3\}$ has two ones, chain $\{2, 4\}$ none $\Rightarrow$ YES. $n = 3, k = 2$, 111: chain $\{2\}$ has one one $\Rightarrow$ NO. ✓

C++17 implementation

```

1 // Codeforces 2236B - Tatar TV Show
2 // Positions with equal index mod k form a chain; a move flips two adjacent
3 // chain cells, so a chain can be cleared iff it holds an even number of 1s.
4 #include <iostream>
5 #include <string>
6 #include <vector>
7 using namespace std;
8
9 bool canClear(int n, int k, const string& s) {
10     vector<int> ones(k, 0);
11     for (int i = 0; i < n; i++)
12         if (s[i] == '1') ones[i % k]++;
13     for (int c : ones)
14         if (c % 2) return false;
15     return true;
16 }
17
18 int main() {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21     int t;
22     cin >> t;
23     while (t--) {
24         int n, k;
25         string s;
26         cin >> n >> k >> s;
27         cout << (canClear(n, k, s) ? "YES" : "NO") << '\n';
28     }
29     return 0;
30 }

```

Complexity Analysis**Time:** $O(n)$. **Space:** $O(k)$.**Common Pitfalls and Edge Cases**

- When $k = n$ no move exists at all; the criterion still works (every chain has one cell, so each must already be 0).
- Allocate the counter array per test with size k (sum of n is bounded, sum of k is too).

1.21 Delete and Concatenate

Source: Codeforces 2245B (Notion: Greedy, 1100)**Level:** Medium

Problem Statement. Array a of $n \leq 2 \cdot 10^5$ integers ($|a_i| \leq 10^9$) and a cost $c \geq 0$. Until the array is empty, either remove one element (gain its value) or remove two *currently adjacent* elements (gain their maximum). Every operation costs c . Maximise the final score.

Pattern Recognition

Start from a baseline (remove everything singly) and ask what each pairing *changes*: pairing $x \leq y$ gains $c - x$ over two single removals. The best pairs use the smallest elements as their minima.

Approach and intuition

Removing everything singly gives $\sum a_i - nc$. Replacing two single removals of $x \leq y$ by one pair removal changes the score by $(\max(x, y) - c) - (x - c) - (y - c) = c - x$. So every pair contributes $c - \min$ of the pair.

Which Pairs Are Possible?

Nested pairs are fine (remove the inner pair first), so any *non-crossing* matching is achievable. Given k “small” elements (the k smallest) and k “partners” (the k largest), a non-crossing matching that pairs every small element with a partner always exists — repeatedly match an adjacent small/partner pair in the order of positions, exactly like balanced parentheses. Each small element is $\leq$ its partner, so the gain is $\sum_{i \leq k} (c - a_{(i)})$. Take pairs while $a_{(i)} < c$ and $i \leq n/2$.

Algorithm

```

1:  $score \leftarrow \sum a_i - n \cdot c$ ; sort  $a$ 
2: for  $i = 0$  while  $i < \lfloor n/2 \rfloor$  and  $a_i < c$  do
3:    $score \leftarrow score + (c - a_i)$ 
4: end for
5: return  $score$ 

```

Dry Run. $a = [3, 1, 4, 1, 5, 9]$, $c = 6$: baseline $23 - 36 = -13$; sorted $1, 1, 3, 4, 5, 9$; gains $5, 5, 3$ for the three smallest (all < 6). Score 0. $a = [1, 3, 1]$, $c = 2$: $5 - 6 = -1$, one pair gains 1, score 0. ✓

C++17 implementation

```

1 // Codeforces 2245B - Delete and Concatenate
2 // Base: remove everything singly -> sum(a) - n*c. Pairing x <= y instead
3 // gains c - x. Pair the k smallest with the k largest while a_(i) < c.
4 #include <algorithm>
5 #include <iostream>
6 #include <vector>
7 using namespace std;
8
9 long long bestScore(vector<long long> a, long long c) {
10     int n = a.size();
11     long long score = -1LL * n * c;
12     for (long long v : a) score += v;
13     sort(a.begin(), a.end());
14     for (int i = 0; i < n / 2 && a[i] < c; i++) score += c - a[i];
15     return score;
16 }
17
18 int main() {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21     int t;
22     cin >> t;
23     while (t--) {
24         int n;
25         long long c;
26         cin >> n >> c;
27         vector<long long> a(n);
28         for (auto& v : a) cin >> v;
29         cout << bestScore(a, c) << '\n';
30     }
31     return 0;
32 }

```

Complexity Analysis

Time: $O(n \log n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- $n \cdot c$ reaches $2 \cdot 10^{14}$ and the answer can be $-4 \cdot 10^9$ or lower: `long long` everywhere.
- At most $\lfloor n/2 \rfloor$ pairs exist.

1.22 Always Changing

Source: Codeforces 2252B (Notion: Greedy, 1100)

Level: Medium

Problem Statement. A binary string s ($n \leq 2 \cdot 10^5$) must be turned into an *alternating* string (no two equal neighbours) by deleting characters. The deleted characters must strictly alternate in value: after deleting a 0 the next deletion must be a 1 and vice versa (the first may be either). Find the minimum number of deletions, or -1 .

Pattern Recognition

Two separate questions: *which* characters must go (forced by runs), and *can the counts be balanced* ($|\#0 - \#1| \leq 1$ is exactly what an alternating deletion order needs).

Approach and intuition

Split s into maximal runs. A run of length L must lose at least $L - 1$ characters, otherwise two equal characters stay adjacent. Let z and o be these forced deletions of zeros and ones. Deletions can be performed in an alternating order iff their counts differ by at most one (order does not change the final string).

- If $|z - o| \leq 1$: answer $z + o$.
- Otherwise we need $need = |z - o| - 1$ extra deletions of the *deficient* character. Deleting the last character of an *interior* run of that type merges its two neighbours, forcing one more deletion of the excess type — the difference does not improve. Only a run at either *end* of the string can be removed cleanly. So there are at most $cap \in \{0, 1, 2\}$ useful extra deletions.

Formula

$$\text{answer} = \begin{cases} z + o & |z - o| \leq 1, \\ z + o + need & need \leq cap, \\ -1 & \text{otherwise,} \end{cases}$$

where cap counts the string ends (s_1, s_n) equal to the deficient character. Verified against an exhaustive search over all 2^n deletion sets for every string with $n \leq 10$.

Algorithm

- 1: $z, o \leftarrow$ number of indices i with $s_i = s_{i-1}$, split by character
- 2: **if** $|z - o| \leq 1$ **then return** $z + o$
- 3: **end if**
- 4: $need \leftarrow |z - o| - 1$; $d \leftarrow$ the character with fewer forced deletions
- 5: $cap \leftarrow [s_1 = d] + [s_n = d]$
- 6: **return** $need \leq cap ? z + o + need : -1$

Dry Run. 100010: runs 1|000|1|0, $z = 2, o = 0, need = 1$, the deficient type is 1 and $s_1 = 1$, so $cap \geq 1$: answer 3. 011110: $o = 3, z = 0, need = 2$, both ends are 0: answer 5. 111: $o = 2, need = 1$, no zero at an end: -1. ✓

C++17 implementation

```

1 // Codeforces 2252B - Always Changing
2 // Forced deletions: L-1 from every run. If the 0/1 counts differ by more
3 // than 1, the deficient type can only be topped up by deleting a whole
4 // run of that type sitting at an end of the string (at most two).
5 #include <cstdlib>
6 #include <iostream>
7 #include <string>
8 using namespace std;
9
10 long long minOps(const string& s) {
11     int n = s.size();
12     long long z = 0, o = 0; // forced 0- and 1-deletions
13     for (int i = 1; i < n; i++)
14         if (s[i] == s[i - 1]) (s[i] == '0' ? z : o)++;
15     if (llabs(z - o) <= 1) return z + o;
16     long long need = llabs(z - o) - 1;
17     char deficient = (z > o) ? '1' : '0';

```

```

18     int cap = (s[0] == deficient) + (s[n - 1] == deficient);
19     // s[0] and s[n-1] lie in different runs whenever both equal
20     // the deficient type (the excess type must appear somewhere).
21     return need <= cap ? z + o + need : -1;
22 }
23
24 int main() {
25     ios::sync_with_stdio(false);
26     cin.tie(nullptr);
27     int t;
28     cin >> t;
29     while (t--) {
30         int n;
31         string s;
32         cin >> n >> s;
33         cout << minOps(s) << '\n';
34     }
35     return 0;
36 }

```

Complexity Analysis

Time: $O(n)$. **Space:** $O(1)$ beyond the string.

Common Pitfalls and Edge Cases

- Deleting an interior single-character run never helps — this is the trap that makes “just delete more of the rare character” wrong.
- $n = 1$ or an already alternating string: 0.

1.23 MEX Multiset

Source: Codeforces 2259D (Notion: Greedy, 1200)

Level: Medium

Problem Statement. Distribute the $n \geq 3$ values a_i ($0 \leq a_i \leq 10^9$) into three multisets A, B, C so that $\text{MEX}(A) + \text{MEX}(B) + \text{MEX}(C) \geq 2 \max(\text{MEX}(A), \text{MEX}(B), \text{MEX}(C))$. Print a construction or NO.

Pattern Recognition

Rewrite the condition: with sorted MEX values $p \leq q \leq r$ it says $p + q \geq r$ (“triangle-like”). It holds *automatically* whenever the two largest are equal. Look for the cheapest such configuration.

Approach and intuition

A set has $\text{MEX} \geq 1$ only if it contains a 0. Let z be the number of zeros.

- $z = 0$: all three MEX values are 0 and $0 \geq 0$ holds. Put everything in A .
- $z = 1$: exactly one set has $\text{MEX} \geq 1$, the others 0, and $0 + 0 < r$. Impossible.
- $z \geq 2$: $B = \{0\}$, $C =$ all the ones, $A =$ everything else (the remaining zeros and all values ≥ 2). Then A and B contain 0 but no 1, so both have MEX 1, and C has no 0, so MEX 0: $1 + 1 + 0 \geq 2 \cdot 1$.

Only the Number of Zeros Matters

Larger values never have to be arranged carefully: once the two biggest MEX values are equal, the inequality is satisfied, and $\text{MEX} = 1$ for two sets costs exactly two zeros.

Algorithm

- 1: $z \leftarrow$ number of zeros
- 2: **if** $z = 1$ **then** **return** NO
- 3: **end if**
- 4: all letters A; **if** $z \geq 2$: first zero $\rightarrow$ B, every 1 $\rightarrow$ C

Dry Run. $[0, 2, 2]$: one zero, NO. $[1, 0, 0, 1, 2, 1]$: string CBACAC: $A = \{0, 2\}$ (MEX 1), $B = \{0\}$ (MEX 1), $C = \{1, 1, 1\}$ (MEX 0); $2 \geq 2$. $\checkmark$ (Any valid answer is accepted.)

C++17 implementation

```

1 // Codeforces 2259D - MEX Multiset
2 // Only the number of zeros matters:
3 //   no zero   -> every MEX is 0, 0 + 0 >= 0 holds;
4 //   one zero  -> exactly one set has MEX >= 1 while the other two are 0: NO;
5 //   >= 2 zeros -> A = {0, all values >= 2}, B = {0}, C = {all 1s}
6 //               gives MEX 1, 1, 0 and 1 + 0 >= 1.
7 #include <iostream>
8 #include <string>
9 #include <vector>
10 using namespace std;
11
12 bool build(const vector<long long>& a, string& out) {
13     int n = a.size(), zeros = 0;
14     for (long long v : a) zeros += (v == 0);
15     out.assign(n, 'A');
16     if (zeros == 0) return true;
17     if (zeros == 1) return false;
18     bool gaveB = false;
19     for (int i = 0; i < n; i++) {
20         if (a[i] == 0 && !gaveB) { out[i] = 'B'; gaveB = true; }
21         else if (a[i] == 1) out[i] = 'C';
22         // every other element (extra zeros, values >= 2) stays in A
23     }
24     return true;
25 }
26
27 int main() {
28     ios::sync_with_stdio(false);
29     cin.tie(nullptr);
30     int t;
31     cin >> t;
32     while (t--) {
33         int n;
34         cin >> n;
35         vector<long long> a(n);
36         for (auto& v : a) cin >> v;
37         string s;
38         if (build(a, s)) cout << "YES\n" << s << '\n';
39         else cout << "NO\n";
40     }

```

```

41     return 0;
42 }
```

Complexity Analysis

Time: $O(n)$. **Space:** $O(n)$ for the output string.

Common Pitfalls and Edge Cases

- Do not try to maximise MEX values — a large MEX in one set makes the inequality *harder*.
- The output checker accepts any valid distribution; the sample output is just one of them.

1.24 Reduction Game

Source: ICPC 2018–19 Online Preliminary (PYQ, REDCGAME)

Level: Medium

Problem Statement. An array a of $n \leq 50$ integers and an integer k ($1 \leq a_i, k \leq 5 \cdot 10^4$). While there exist $i \neq j$ with $\min(a_i, a_j) > k$ you *must* pick such a pair and decrease both by one. Choosing the pairs cleverly, maximise the final sum.

Pattern Recognition

Forced process + “maximise what is left” = *minimise the number of operations*. Think in terms of excesses $e_i = a_i - k$ over the threshold and a single survivor.

Approach and intuition

Only elements with $a_i > k$ ever change; let $e_i = a_i - k > 0$ be their excesses. The game stops exactly when at most one excess is positive. Each operation lowers the sum by 2.

Pick the element with the largest excess as the *survivor* s ; all other excesses (total R , largest m) must reach 0. Every one of those R units is removed, costing R from the sum. The question is how many units the survivor must lose as well:

- non-survivors can cancel in pairs among themselves, unless one of them dominates ($m > R - m$): then $2m - R$ of its units must be cancelled against the survivor;
- if R is odd, one unit is left over and must pair with the survivor.

Result

$$\text{answer} = \sum a_i - R - \max(2m - R, R \bmod 2),$$

and $2m - R \leq m \leq e_s$, so the survivor never runs out. Choosing a smaller survivor can only increase R by more than it saves. Verified against an exhaustive game search on 3000 random small arrays.

Algorithm

- 1: $E \leftarrow \{a_i - k : a_i > k\}$
- 2: **if** $|E| \leq 1$ **then return** $\sum a$
- 3: **end if**
- 4: remove the maximum of E (survivor); $R \leftarrow \sum E$, $m \leftarrow \max E$

5: **return** $\sum a - R - \max(2m - R, R \bmod 2)$

Dry Run. $a = [2, 3, 2]$, $k = 1$: excesses 1, 2, 1. Survivor 2; others 1, 1, $R = 2$, $m = 1$, loss $\max(0, 0) = 0$. Answer $7 - 2 = 5$ (final $[1, 3, 1]$). $a = [2, 2]$, $k = 1$: survivor 1, $R = 1$, loss $\max(1, 1) = 1$, answer $4 - 2 = 2$. ✓

C++17 implementation

```

1 // ICPC 2018-19 Online Preliminary - Reduction Game (REDCGAME)
2 // Work with excesses  $e_i = a_i - k$  ( $> 0$  only). The process stops when at
3 // most one excess is positive, so keep the largest as the survivor and
4 // cancel the others among themselves as much as possible.
5 #include <algorithm>
6 #include <iostream>
7 #include <vector>
8 using namespace std;
9
10 long long maxFinalSum(const vector<long long>& a, long long k) {
11     long long total = 0;
12     vector<long long> e;
13     for (long long x : a) {
14         total += x;
15         if (x > k) e.push_back(x - k);
16     }
17     if (e.size() <= 1) return total;           // no move is possible
18     sort(e.begin(), e.end());
19     e.pop_back();                             // survivor = largest excess
20     long long R = 0;
21     for (long long x : e) R += x;
22     long long m = e.back();                   // largest non-survivor
23     long long survivorLoss = max(2 * m - R, R % 2);
24     return total - R - survivorLoss;         // every unit removed costs 1
25 }
26
27 int main() {
28     ios::sync_with_stdio(false);
29     cin.tie(nullptr);
30     int t;
31     cin >> t;
32     while (t--) {
33         int n;
34         long long k;
35         cin >> n >> k;
36         vector<long long> a(n);
37         for (auto& x : a) cin >> x;
38         cout << maxFinalSum(a, k) << '\n';
39     }
40     return 0;
41 }

```

Complexity Analysis

Time: $O(n \log n)$ per test. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- Elements $\leq k$ are frozen forever; do not include them in E .
- Simulating with a priority queue is too slow: excesses reach $5 \cdot 10^4$ with 5000 tests.

1.25 A Game of Robots

Source: ICPC 2018–19 Online Preliminary (PYQ, ROBOGAME)

Level: Medium

Problem Statement. A 1-D grid of $n \leq 50$ cells is given as a string: `.` is empty, a digit x is a robot that moves back and forth inside the cells $[i - x, i + x]$ (clipped to the grid). You choose each robot's initial direction; the robots choose their own start times and *want* to collide. Output **safe** if you can prevent every collision, else **unsafe**.

Pattern Recognition

The robots control their start times, so direction choices cannot stop two robots whose ranges share a cell. Reduce to **interval overlap**, then a difference array.

Approach and intuition

If the ranges of two robots intersect in some cell c , each robot passes through c periodically; by delaying one robot's start by the right amount they are at c at the same moment, whatever directions we chose. If no two ranges intersect, the robots never share a cell at all. So:

$$\text{safe} \iff \text{no cell is covered by two ranges.}$$

A robot with $x = 0$ occupies just its own cell — it still counts.

Algorithm

- 1: $\text{diff}[\cdot] \leftarrow 0$
- 2: **for** each robot at i with range x **do**
- 3: $\text{diff}[\max(0, i - x)]++$; $\text{diff}[\min(n - 1, i + x) + 1]-$
- 4: **end for**
- 5: **return** **unsafe** if some prefix sum ≥ 2 , else **safe**

Dry Run. `.2...2..` (0-indexed): ranges $[0, 3]$ and $[3, 7]$ share cell 3 $\Rightarrow$ unsafe. `.2.....`: one robot $\Rightarrow$ safe. `1.1.1.1`: ranges $[0, 1]$, $[1, 3]$ overlap $\Rightarrow$ unsafe. ✓

C++17 implementation

```

1 // ICPC 2018-19 Online Preliminary - A Game of Robots (ROBOGAME)
2 // Two robots can be made to collide iff their movement ranges share a cell;
3 // so the grid is safe iff no cell is covered by two ranges.
4 #include <algorithm>
5 #include <iostream>
6 #include <string>
7 #include <vector>
8 using namespace std;
9
10 bool isSafe(const string& s) {
11     int n = s.size();
12     vector<int> diff(n + 1, 0);

```

```

13  for (int i = 0; i < n; i++) {
14      if (s[i] == '.') continue;
15      int x = s[i] - '0';
16      diff[max(0, i - x)]++;
17      diff[min(n - 1, i + x) + 1]--;
18  }
19  int cover = 0;
20  for (int i = 0; i < n; i++) {
21      cover += diff[i];
22      if (cover >= 2) return false;
23  }
24  return true;
25 }
26
27 int main() {
28     ios::sync_with_stdio(false);
29     cin.tie(nullptr);
30     int t;
31     cin >> t;
32     while (t--) {
33         string s;
34         cin >> s;
35         cout << (isSafe(s) ? "safe" : "unsafe") << '\n';
36     }
37     return 0;
38 }

```

Complexity Analysis

Time: $O(n)$ per test. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- Clip ranges to the grid *before* checking overlap.
- The long story about directions is a red herring; the key sentence is “they can coordinate ... and decide when to start”.

1.26 Cracker Shops

Source: ICPC 2014–15 Kharagpur Online Re-contest (PYQ, ACM14KG1) **Level:** Medium–Hard

Problem Statement. Shops $1..N$ each sell a set of cracker types from $1..M$ (all M types appear somewhere). For every shop i find $j \leq i \leq k$ such that shops $j..k$ together sell all M types, minimising $k - j$ and then j . ($\sum N, \sum M, \sum X \leq 5 \cdot 10^5$.)

Pattern Recognition

Two layers: **two pointers** give, for each left end j , the shortest covering window $[j, e_j]$; then every query window is one of these, possibly *stretched* to reach i . Monotone boundaries $\Rightarrow$ a sliding ordered set.

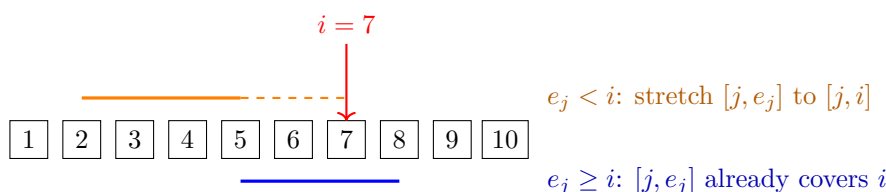
Approach and intuition

Step 1. For each j let e_j be the smallest k such that $[j, k]$ covers all types (∞ if none). As j grows, e_j never decreases, so a standard two-pointer sweep with a counter per type computes

all e_j in linear time.

Step 2. For shop i and a left end $j \leq i$, the best window starting at j that contains i is $[j, \max(e_j, i)]$. Split the candidates:

- $e_j < i$: the window is $[j, i]$, length $i - j$, best for the *largest* such j . Because e is non-decreasing, these j form a prefix $1..p - 1$, so the candidate is $j = p - 1$.
- $e_j \geq i$: the window is $[j, e_j]$, scored by $(e_j - j, j)$. These j lie in $[p, i]$ — a range whose both ends only move right as i grows. Keep them in an ordered set and read its minimum.



Algorithm

```

1: compute  $e_j$  by two pointers
2:  $open \leftarrow \emptyset$  (pairs  $(e_j - j, j)$ ),  $p \leftarrow 1$ 
3: for  $i = 1$  to  $N$  do
4:   if  $e_i < \infty$ : insert  $(e_i - i, i)$ 
5:   while  $p \leq i$  and  $e_p < i$  do erase  $(e_p - p, p)$ ;  $p \leftarrow p + 1$ 
6:   end while
7:    $best \leftarrow \min(open)$  as window  $[j, e_j]$ 
8:   if  $p > 1$ : compare with window  $[p - 1, i]$  (length  $i - p + 1$ , tie on smaller  $j$ )
9:   print  $best$ 
10: end for

```

Dry Run. Test 2: shops $\{1\}, \{2, 3\}, \{1\}$, $M = 3$. $e_1 = 2$, $e_2 = 3$, $e_3 = \infty$. $i = 1$: $open = \{(1, 1)\} \Rightarrow (1, 2)$. $i = 2$: $open \{(1, 1), (1, 2)\}$, tie on length, smaller $j \Rightarrow (1, 2)$. $i = 3$: $e_1 = 2 < 3$ so $j = 1$ leaves, $p = 2$; $open \{(1, 2)\} \Rightarrow [2, 3]$; the stretched candidate $[1, 3]$ has length $2 > 1$. Output 1 2, 1 2, 2 3. ✓

C++17 implementation

```

1 // ICPC 2014-15 Kharagpur Online Re-contest - Cracker shops (ACM14KG1)
2 // Step 1 (two pointers):  $e[j]$  = least  $k$  such that shops  $j..k$  cover all  $M$  types.
3 // Step 2: for shop  $i$  the best window starting at  $j$  is  $[j, \max(e[j], i)]$ .
4 // * starts  $j$  with  $e[j] < i$  form a prefix; the best of them is the last one.
5 // * starts  $j$  in  $[p, i]$  ( $e[j] \geq i$ ) are scored by  $(e[j] - j, j)$ : keep them in
6 //   an ordered set that slides right as  $i$  grows.
7 #include <iostream>
8 #include <set>
9 #include <string>
10 #include <utility>
11 #include <vector>
12 using namespace std;
13
14 const int INF = 1e9;
15
16 int main() {
17     ios::sync_with_stdio(false);
18     cin.tie(nullptr);
19     int T;

```

```

20  cin >> T;
21  while (T--) {
22      int n, m;
23      cin >> n >> m;
24      vector<vector<int>> shop(n + 1);
25      for (int i = 1; i <= n; i++) {
26          int x;
27          cin >> x;
28          shop[i].resize(x);
29          for (int& c : shop[i]) cin >> c;
30      }
31      // ---- step 1: minimal right end for every left end ----
32      vector<int> e(n + 2, INF), have(m + 1, 0);
33      int distinct = 0, r = 0;
34      for (int j = 1; j <= n; j++) {
35          while (r < n && distinct < m) {
36              ++r;
37              for (int c : shop[r]) if (have[c]++ == 0) distinct++;
38          }
39          if (distinct == m) e[j] = r;
40          for (int c : shop[j]) if (--have[c] == 0) distinct--;
41      }
42      // ---- step 2: sweep i = 1..n ----
43      set<pair<int, int>> open;           // (e[j] - j, j) with e[j] >= i
44      int p = 1;                        // first j with e[j] >= i
45      string out;
46      for (int i = 1; i <= n; i++) {
47          if (e[i] != INF) open.insert({e[i] - i, i});
48          while (p <= i && e[p] < i) { // e[p] < i: window no longer spans i
49              open.erase({e[p] - p, p});
50              p++;
51          }
52          int bestLen = INF, bestJ = 0, bestK = 0;
53          if (!open.empty()) {
54              auto [len, j] = *open.begin();
55              bestLen = len; bestJ = j; bestK = e[j];
56          }
57          if (p > 1) { // candidate: j = p - 1, window [j, i]
58              int j = p - 1, len = i - j;
59              if (len < bestLen || (len == bestLen && j < bestJ)) {
60                  bestLen = len; bestJ = j; bestK = i;
61              }
62          }
63          out += to_string(bestJ) + ' ' + to_string(bestK) + '\n';
64      }
65      cout << out;
66  }
67  return 0;
68 }

```

Complexity Analysis

Time: $O((N + \sum X) + N \log N)$. **Space:** $O(N + M + \sum X)$.

Common Pitfalls and Edge Cases

- The editorial's "corner case": near the end every minimal window ends before i , so the answer is a *stretched* window — the $j = p - 1$ candidate.
- Ties: minimise length first, then j .
- Up to 10^4 tests: allocate per test only $O(N + M)$ memory, never a global array of fixed maximum size reset each time.

1.27 Jai Hanuman

Source: ICPC 2014–15 Kharagpur Online Re-contest (PYQ, ACM14KG4) **Level:** Medium–Hard

Problem Statement. N people and $5N$ crackers numbered $1..5N$; person i starts with crackers $3i - 2, 3i - 1, 3i$. In order $1, 2, \dots, N$ each person may make up to 3 swaps with anyone (swaps cannot be refused, Hanuman's spare crackers included). Everyone wants three *consecutive* crackers; among those, the least one should be $\geq P_i$ and as small as possible; if that is impossible, the least one should be as small as possible. Everybody plays optimally and knows everything. Print each person's least cracker.

Pattern Recognition

Sequential moves where the *last* mover cannot be disturbed $\Rightarrow$ **backward induction**: fix the last player's outcome, then the second last's among what is left, and so on.

Approach and intuition

Three swaps are enough to collect any three crackers, so person N , who moves last, always ends with exactly the triple they want. Person $N - 1$ knows this, so anything they take from N 's target set will be taken away again; their best achievable triple is the best one *avoiding* N 's triple. Inductively, person i gets their preferred triple among crackers not wanted by $i + 1, \dots, N$.

Implementation: a start s is *available* while $s, s + 1, s + 2$ are all free. Keep all available starts in a **set**. For person i (from N down to 1) take **lower_bound**(P_i), or the smallest start if none exists. Taking $[s, s + 2]$ kills starts $s - 2, \dots, s + 2$. At most 5 starts die per person and there are $5N - 2$ initially, so a start always remains.

Algorithm

- 1: $S \leftarrow \{1, \dots, 5N - 2\}$
- 2: **for** $i = N$ **down to** 1 **do**
- 3: $s \leftarrow$ smallest element of S that is $\geq P_i$, else $\min S$
- 4: $ans_i \leftarrow s$; remove $s - 2..s + 2$ from S
- 5: **end for**

Dry Run. $N = 2$, $P = (4, 9)$, crackers $1..10$. Person 2: starts ≥ 9 need $s \leq 8$, none, so $s = 1$ (crackers 1, 2, 3). Person 1: smallest free start ≥ 4 is 4. Output 4 1. With $P = (1, 5)$: person 2 takes 5, person 1 takes 1: 1 5. ✓

C++17 implementation

```
1 // ICPC 2014-15 Kharagpur Online Re-contest - Jai Hanuman (ACM14KG4)
2 // Person N moves last, so nobody can take anything from them: they get their
3 // ideal triple. Person N-1 gets the ideal triple among what N does not want,
```

```

4 // and so on -> assign greedily from N down to 1.
5 // A "start" s is available while s, s+1, s+2 are all untaken.
6 #include <iostream>
7 #include <set>
8 #include <string>
9 #include <vector>
10 using namespace std;
11
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15     int T;
16     cin >> T;
17     while (T--) {
18         int n;
19         cin >> n;
20         vector<int> P(n);
21         for (int& x : P) cin >> x;
22         int total = 5 * n;
23         set<int> starts; // valid triple starts
24         for (int s = 1; s + 2 <= total; s++) starts.insert(starts.end(), s);
25         vector<int> ans(n);
26         for (int i = n - 1; i >= 0; i--) {
27             auto it = starts.lower_bound(P[i]); // least start >= P[i]
28             if (it == starts.end()) it = starts.begin(); // fall back: least
29             int s = *it;
30             ans[i] = s;
31             for (int d = -2; d <= 2; d++) starts.erase(s + d); // overlap s..s+2
32         }
33         string out;
34         for (int i = 0; i < n; i++) out += to_string(ans[i]) + (i + 1 < n ? " " : "\n");
35         cout << out;
36     }
37     return 0;
38 }

```

Complexity Analysis

Time: $O(N \log N)$ per test. **Space:** $O(N)$.

Common Pitfalls and Edge Cases

- Processing people from 1 to N (forward) is the natural but wrong reading — earlier players do not get to keep contested crackers.
- Consecutive means *any* three consecutive numbers, not aligned blocks $\{3i-2, 3i-1, 3i\}$.

1.28 Break, Merge and Sort

Source: ICPC 2020–21 Amritapuri Preliminary, problem 3 (PYQ)

Level: Medium

Reconstructed Statement

The original statement is no longer online (it was hosted on CodeDrills, which is offline, and the Notion archive stores only the official *solution slides*). The statement below is

reconstructed from those slides. The cost model was checked by an exhaustive Dijkstra search over all split/merge sequences on 300 random arrays: it reproduces the editorial's formula exactly. Verify against the original if you find it.

Problem Statement. An array A of length n . You may **split** an array into two contiguous non-empty parts, paying the length of the shorter part, and **merge** two sorted arrays into one sorted array, paying the sum of their lengths. Starting from A , obtain a single sorted array with minimum total cost.

Pattern Recognition

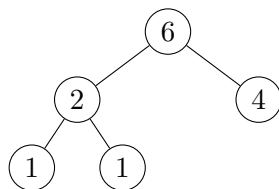
Four greedy insights from the editorial: (1) all splits can be done before all merges; (2) cut exactly at the descents $A_i > A_{i+1}$; (3) peel the smaller end piece each time; (4) merge the two shortest arrays (Huffman).

Approach and intuition

Cutting at every descent leaves the maximal sorted runs $r_1, \dots, r_t$; further cuts only create more merging work.

Splitting. Repeatedly cut off the smaller of the two end runs. Each cut costs at most that run's length and every run except the largest is cut off once, so splitting costs $n - \max_i r_i$ — and this is also a lower bound.

Merging. Merging behaves like building a Huffman tree: a run merged c_i times contributes $c_i \cdot r_i$, where c_i is its depth in the merge tree. As in Huffman coding, there is an optimal tree in which the two shortest runs are siblings, so “always merge the two shortest” is optimal.



Runs 1, 1, 4: merge 1 + 1 (cost 2), then 2 + 4 (cost 6): merging cost $8 = \sum$ internal nodes.

Total Cost

$$\text{answer} = \left(n - \max_i r_i\right) + \text{Huffman}(r_1, \dots, r_t).$$

Algorithm

- 1: split A into maximal non-decreasing runs $r_1..r_t$
- 2: $\text{cost} \leftarrow n - \max r_i$; push all r_i into a min-heap
- 3: **while** heap size > 1 **do**
- 4: pop x, y ; $\text{cost} \leftarrow \text{cost} + x + y$; push $x + y$
- 5: **end while**
- 6: **return** cost

Dry Run. $A = [3, 1, 4, 2]$: runs $[3], [1, 4], [2]$, lengths 1, 2, 1. Split cost $4 - 2 = 2$. Huffman: $1 + 1 = 2$ (cost 2), $2 + 2 = 4$ (cost 4). Total 8. A sorted array has one run and costs 0. ✓

C++17 implementation

```

1 // ICPC 2020-21 Amritapuri Preliminary - Break, Merge and Sort
2 // (statement reconstructed from the official solution slides; see the book)
3 // Cost model: splitting an array costs the length of the shorter part,
4 // merging two sorted arrays costs the sum of their lengths.
5 // Answer = (n - longest sorted run) + Huffman cost of the run lengths.
6 #include <algorithm>
7 #include <functional>
8 #include <iostream>
9 #include <queue>
10 #include <vector>
11 using namespace std;
12
13 long long minCost(const vector<long long>& a) {
14     vector<long long> runs;
15     long long cur = 1;
16     for (size_t i = 1; i < a.size(); i++) {
17         if (a[i] < a[i - 1]) { runs.push_back(cur); cur = 1; }
18         else cur++;
19     }
20     runs.push_back(cur);
21     long long cost = (long long)a.size() - *max_element(runs.begin(), runs.end());
22     priority_queue<long long, vector<long long>, greater<>> pq(runs.begin(), runs.end());
23     while (pq.size() > 1) { // Huffman merging
24         long long x = pq.top(); pq.pop();
25         long long y = pq.top(); pq.pop();
26         cost += x + y;
27         pq.push(x + y);
28     }
29     return cost;
30 }
31
32 int main() {
33     ios::sync_with_stdio(false);
34     cin.tie(nullptr);
35     int t;
36     cin >> t;
37     while (t--) {
38         int n;
39         cin >> n;
40         vector<long long> a(n);
41         for (auto& x : a) cin >> x;
42         cout << minCost(a) << '\n';
43     }
44     return 0;
45 }

```

Complexity Analysis

Time: $O(n \log n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- Equal neighbours are *not* a descent (non-decreasing runs).
- Merging runs left-to-right is the classic wrong answer: lengths 1, 1, 1, 100 merged left to right cost $2 + 3 + 103 = 108$; Huffman also gives 108 here, but for 100, 1, 1, 1 left to right

costs $101 + 102 + 103 = 306$ vs Huffman $2 + 3 + 103 = 108$.

1.29 Minimum Weight Bi-Partition

Source: ICPC 2020–21 Amritapuri Preliminary, problem 4 (PYQ)

Level: Medium

Reconstructed Statement

Only the official solution slides survive. They reduce the problem to: “a tree is a connected bipartite graph, so after adding new edges it is enough to find an MST; sorting vertices by A_i and joining consecutive ones gives the only $n - 1$ new edges that matter; existing edges have weight 0.” This section solves that core task. The full original statement may add conditions that this reduction already handles; check the original if you find it.

Problem Statement. A graph has n vertices with values A_i and m existing edges, which are free. You may add an edge (u, v) at cost $|A_u - A_v|$. Find the minimum total cost to make the graph connected (equivalently, the weight of a minimum spanning tree of the complete graph where existing edges weigh 0).

Pattern Recognition

MST on a complete graph with weights $|A_u - A_v| \Rightarrow$ only **adjacent-in-sorted-order** edges can be useful: $O(n)$ candidate edges instead of $O(n^2)$.

Approach and intuition

Sort the vertices by value: $A_{v_1} \leq \dots \leq A_{v_n}$. For $i < j$ the edge (v_i, v_j) has weight $A_{v_j} - A_{v_i}$, which equals the total weight of the path $v_i, v_{i+1}, \dots, v_j$, and each edge on that path is no heavier. By the cycle property of MSTs, a heaviest edge on a cycle is never needed, so the long edge can be dropped. The candidate set is: the m free edges plus the $n - 1$ sorted-neighbour edges. Run Kruskal with a DSU.

Algorithm

```

1:  $E \leftarrow \{(0, u, v) \text{ for existing edges}\} \cup \{(A_{v_{i+1}} - A_{v_i}, v_i, v_{i+1})\}$ 
2: sort  $E$ ; DSU on  $n$  vertices;  $cost \leftarrow 0$ 
3: for  $(w, u, v) \in E$  do
4:   if UNITE( $u, v$ ) then  $cost \leftarrow cost + w$ 
5:   end if
6: end for
7: return  $cost$ 
```

Dry Run. $A = [5, 1, 9, 4]$, existing edge $(1, 3)$. Sorted: $v = 2, 4, 1, 3$ (values 1, 4, 5, 9). Edges: $(1, 3)$ free; $(2, 4) = 3$, $(4, 1) = 1$, $(1, 3) = 4$. Kruskal: free $(1, 3)$, then $(4, 1)$ cost 1, then $(2, 4)$ cost 3; $(1, 3)$ now redundant. Total 4. ✓ (checked against Prim on the full graph)

C++17 implementation

```

1 // ICPC 2020-21 Amritapuri Preliminary - Minimum Weight Bi-Partition
2 // (core task reconstructed from the official solution slides; see the book)
3 // Existing edges are free; a new edge (u, v) costs |A_u - A_v|. A minimum
4 // spanning tree only ever needs edges between neighbours in sorted-A order.
5 #include <algorithm>
```

```

6 #include <iostream>
7 #include <numeric>
8 #include <tuple>
9 #include <vector>
10 using namespace std;
11
12 struct DSU {
13     vector<int> p, sz;
14     explicit DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
15     int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
16     bool unite(int a, int b) {
17         a = find(a); b = find(b);
18         if (a == b) return false;
19         if (sz[a] < sz[b]) swap(a, b);
20         p[b] = a; sz[a] += sz[b];
21         return true;
22     }
23 };
24
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28     int t;
29     cin >> t;
30     while (t--) {
31         int n, m;
32         cin >> n >> m;
33         vector<long long> A(n);
34         for (auto& x : A) cin >> x;
35         vector<tuple<long long, int, int>> edges;
36         for (int i = 0; i < m; i++) {
37             int u, v;
38             cin >> u >> v;
39             edges.push_back({0, u - 1, v - 1});
40         }
41         vector<int> ord(n);
42         iota(ord.begin(), ord.end(), 0);
43         sort(ord.begin(), ord.end(), [&](int x, int y) { return A[x] < A[y]; });
44         for (int i = 0; i + 1 < n; i++)
45             edges.push_back({A[ord[i + 1]] - A[ord[i]], ord[i], ord[i + 1]});
46         sort(edges.begin(), edges.end());
47         DSU d(n);
48         long long cost = 0;
49         for (auto [w, u, v] : edges)
50             if (d.unite(u, v)) cost += w;
51         cout << cost << '\n';
52     }
53     return 0;
54 }

```

Complexity Analysis

Time: $O((n + m) \log(n + m))$. **Space:** $O(n + m)$.

Common Pitfalls and Edge Cases

- Building all $\binom{n}{2}$ edges is $O(n^2)$ — TLE for $n = 10^5$.
- Self-loops and multi-edges among the free edges are harmless in Kruskal.

1.30 Coins

Source: AtCoder AGC018 C (Notion: Greedy, stretch goal)**Level:** Hard

Problem Statement. $X + Y + Z \leq 10^5$ people; person i has A_i gold, B_i silver and C_i bronze coins. Take gold from exactly X people, silver from Y others and bronze from the remaining Z . Maximise the total number of coins.

Pattern Recognition

Three-way assignment $\rightarrow$ subtract one option to get a two-way problem, then an **exchange argument** shows the two groups are separated by one sort order. Try every split point with prefix/suffix **top- k heaps**.

Approach and intuition

Everyone gives bronze by default: start from $\sum C_i$ and let $a_i = A_i - C_i$, $b_i = B_i - C_i$. Now choose X people for a and Y other people for b .

Exchange Argument

If p is gold and q is silver with $a_p - b_p < a_q - b_q$, swapping their roles changes the total by $(a_q + b_p) - (a_p + b_q) = (a_q - b_q) - (a_p - b_p) > 0$. So in an optimal solution, after sorting by $a_i - b_i$ descending, every gold person comes before every silver person: there is a split k with gold chosen among the first k people and silver among the last $n - k$.

For each k , the best gold choice is the X largest a in the prefix and the best silver choice the Y largest b in the suffix. Both are maintained incrementally with a min-heap of size X (resp. Y).

Algorithm

- 1: $base \leftarrow \sum C_i$; sort people by $a_i - b_i$ descending
- 2: $L[k] \leftarrow$ sum of X largest a among the first k (min-heap)
- 3: $R[k] \leftarrow$ sum of Y largest b among people $k..n - 1$ (min-heap)
- 4: **return** $base + \max_{X \leq k \leq n - Y} (L[k] + R[k])$

Dry Run. Sample 1 ($X = 1, Y = 2, Z = 1$): $\sum C = 15$; $(a, b) = (-2, 0), (2, 1), (0, -1), (2, -1)$. Sorted by $a - b$: $(2, -1), (0, -1), (2, 1), (-2, 0)$. At $k = 1$: gold $a = 2$, best two b among the rest $= 1 + 0$; total $15 + 3 = 18$. ✓

C++17 implementation

```

1 // AtCoder AGC018 C - Coins
2 // Subtract  $C_i$ : everyone gives bronze by default, then pick  $X$  people for
3 //  $a = A - C$  and  $Y$  people for  $b = B - C$ . Sorted by  $a - b$  (desc), all gold
4 // people come before all silver people, so try every split point.
5 #include <algorithm>
6 #include <functional>
7 #include <iostream>

```

```

8  #include <queue>
9  #include <vector>
10 using namespace std;
11
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15     int X, Y, Z;
16     cin >> X >> Y >> Z;
17     int n = X + Y + Z;
18     vector<long long> a(n), b(n);
19     long long base = 0;
20     for (int i = 0; i < n; i++) {
21         long long A, B, C;
22         cin >> A >> B >> C;
23         a[i] = A - C; b[i] = B - C; base += C;
24     }
25     vector<int> id(n);
26     for (int i = 0; i < n; i++) id[i] = i;
27     sort(id.begin(), id.end(), [&](int p, int q) { return a[p] - b[p] > a[q] - b[q]; });
28
29     // left[k] = best sum of X values of a among the first k people
30     vector<long long> left(n + 1, 0), right(n + 1, 0);
31     priority_queue<long long, vector<long long>, greater<>> pq;
32     long long s = 0;
33     for (int k = 1; k <= n; k++) {
34         pq.push(a[id[k - 1]]); s += a[id[k - 1]];
35         if ((int)pq.size() > X) { s -= pq.top(); pq.pop(); }
36         left[k] = s;
37     }
38     // right[k] = best sum of Y values of b among people k..n-1
39     priority_queue<long long, vector<long long>, greater<>> qq;
40     s = 0;
41     for (int k = n - 1; k >= 0; k--) {
42         qq.push(b[id[k]]); s += b[id[k]];
43         if ((int)qq.size() > Y) { s -= qq.top(); qq.pop(); }
44         right[k] = s;
45     }
46     long long best = -(1LL << 62);
47     for (int k = X; k <= n - Y; k++) best = max(best, left[k] + right[k]);
48     cout << base + best << '\n';
49     return 0;
50 }

```

Complexity Analysis

Time: $O(n \log n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- Greedy “give each person their best colour” ignores the exact quotas X, Y, Z .
- Sums reach $3 \cdot 10^{14}$: long long.

Chapter 2

Bitwise Operations and XOR

Bits are independent until a carry says otherwise. Almost every bitwise problem is solved by one of two moves: look at each bit separately, or build the answer from the highest bit down.

2.1 Theory and Must-Know Patterns

The operators

x	y	$x \& y$	$x y$	$x \oplus y$	C++
0	0	0	0	0	$\&, , ^$
0	1	0	1	1	$x \ll k = x \cdot 2^k$
1	0	0	1	1	$x \gg k = \lfloor x/2^k \rfloor$
1	1	1	1	0	<code>__builtin_popcountll(x)</code>

Identities you will use every contest

- $x \oplus x = 0$, $x \oplus 0 = x$, $\oplus$ is associative and commutative: **xor is its own inverse**. Hence $a \oplus c = b \iff c = a \oplus b$.
- $x + y = (x \oplus y) + 2(x \& y)$. Therefore $x \oplus y \leq x + y$ with equality iff $x \& y = 0$ (no carries).
- $x \& (x - 1)$ clears the lowest set bit; $x \& (-x)$ isolates it.
- Bit b of x : $(x \gg b) \& 1$. Highest set bit: $63 - \text{__builtin_clzll}(x)$.
- **Prefix xor:** with $P_0 = 0$, $P_i = a_1 \oplus \dots \oplus a_i$, the xor of $a_\ell \dots a_r$ is $P_r \oplus P_{\ell-1}$.
- $0 \oplus 1 \oplus \dots \oplus (2^M - 1) = 0$ for $M \geq 2$.
- A number v has at most $2^{\text{popcount}(v)}$ submasks; iterate them with `for (s = v; s; s = (s - 1) & v)`.

Pattern 1: bits are independent

AND, OR and XOR act on each bit separately. If a condition is a combination of these operations, decide it *bit by bit* (Bitwise Reversion, Tatar-style parity arguments). Addition breaks independence through carries — that is when you need a digit DP over bits (Jackpot).

Pattern 2: greedy from the highest bit

Bit b is worth more than all lower bits together ($2^b > 2^b - 1$). So to maximise a value, fix bits from the top: take bit b whenever some completion still exists (largest submask $\leq x$, maximum xor pair in a trie).

Pattern 3: xor linear algebra

Treat numbers as vectors over $\mathbb{F}_2$. A **xor basis** (Gaussian elimination on bits) answers “can v be written as a xor of a subset?” in $O(\log V)$ and has at most $\lceil \log_2 V \rceil$ elements.

$$\begin{array}{l} \text{basis } b_3 \\ b_1 \\ b_0 \end{array} \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

each basis vector owns a distinct leading bit;
to test v , scan from the top bit and xor with
the owner whenever the bit is set — v is repre-
sentable iff it becomes 0.

Pattern 4: tries on bits

A binary trie over the 30 bits of each number answers “maximum $v \oplus w$ over stored w ” greedily from the top bit in $O(30)$.

Bitwise Checklist

1. Parenthesise: `a & b == c` parses as `a & (b == c)`.
2. Use `1LL << b` when $b \geq 31$.
3. Values up to 10^{18} need 60 bits — loops over 0..30 silently miss the top.
4. Try $n \leq 3$ by hand and brute-force small ranges: bit characterisations are easy to get almost right.

2.2 Bitwise Exclusive Or

Source: AtCoder ABC213 A (Notion: Bitwise)

Level: Easy

Problem Statement. Given $0 \leq A, B \leq 255$, find the unique C with $A \oplus C = B$.

Pattern Recognition

Solve an xor “equation” by xoring both sides: xor is its own inverse.

Approach and intuition

Xor both sides of $A \oplus C = B$ with A : $A \oplus A \oplus C = A \oplus B$, and $A \oplus A = 0$, so $C = A \oplus B$. It is unique because $C \mapsto A \oplus C$ is a bijection (it is its own inverse).

Algorithm

1: **return** $A \oplus B$

Dry Run. $A = 3 = 011_2$, $B = 6 = 110_2$: $C = 101_2 = 5$, and $3 \oplus 5 = 6$. ✓

C++17 implementation

```

1 // AtCoder ABC213 A - Bitwise Exclusive Or
2 // A xor C = B <=> C = A xor B (xor both sides with A; A xor A = 0).
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9     int a, b;
10    cin >> a >> b;
11    cout << (a ^ b) << '\n';
12    return 0;
13 }
```

Complexity Analysis

Time: $O(1)$. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- In C++ $\wedge$ is xor, not power. $\text{pow}(2,3)$ and 2^3 are different things (8 vs 1).

2.3 Bit Strings

Source: CSES 1617 (Notion: Bitwise)

Level: Easy

Problem Statement. Count the bit strings of length $n \leq 10^6$, modulo $10^9 + 7$.

Pattern Recognition

Independent binary choices $\Rightarrow 2^n$; compute it with **fast modular exponentiation**.

Approach and intuition

Each of the n positions is 0 or 1 independently, so there are 2^n strings. Binary exponentiation computes $2^n \bmod p$ with $O(\log n)$ multiplications: write n in binary and multiply together the squares 2^{2^k} for the set bits k (see Section 3.1).

Algorithm

```

1:  $r \leftarrow 1, b \leftarrow 2$ 
2: while  $n > 0$  do
3:   if  $n$  odd then  $r \leftarrow rb \bmod p$ 
4:   end if
5:    $b \leftarrow b^2 \bmod p; n \leftarrow \lfloor n/2 \rfloor$ 
6: end while
7: return  $r$ 

```

Dry Run. $n = 3 = 11_2$: $r = 2$, then $r = 2 \cdot 4 = 8$. ✓

C++17 implementation

```

1 // CSES 1617 - Bit Strings
2 // Each of the n positions is independently 0 or 1: 2^n strings.
3 #include <iostream>
4 using namespace std;
5
6 const long long MOD = 1'000'000'007LL;
7
8 long long power(long long b, long long e, long long m) {
9     long long r = 1 % m;
10    b %= m;
11    while (e > 0) {
12        if (e & 1) r = r * b % m;
13        b = b * b % m;
14        e >>= 1;
15    }
16    return r;
17 }
18
19 int main() {
20     ios::sync_with_stdio(false);
21     cin.tie(nullptr);
22     long long n;
23     cin >> n;
24     cout << power(2, n, MOD) << '\n';
25     return 0;
26 }

```

Complexity Analysis

Time: $O(\log n)$. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- $1 \ll n$ overflows for $n \geq 31$ (and $n \geq 63$ for 1LL).
- Multiply in long long: $(10^9)^2$ does not fit in int.

2.4 Gray Code

Source: CSES 2205 (Notion: Bitwise)

Level: Easy–Medium

Problem Statement. List all 2^n bit strings of length $n \leq 16$ so that consecutive strings differ in exactly one bit.

Pattern Recognition

Reflected Gray code: $g(i) = i \oplus (i \gg 1)$.

Approach and intuition

Going from i to $i + 1$ flips a block of trailing bits: $i = \dots 011 \dots 1$ becomes $\dots 100 \dots 0$ (the lowest 0 and all ones below it). In $g(i) = i \oplus (i \gg 1)$ every bit is the xor of two neighbouring bits of i ; all pairs inside the flipped block change on both sides and stay equal, so only the pair at the top of the block changes. Hence $g(i)$ and $g(i + 1)$ differ in exactly one bit. The map is a bijection (it can be inverted by prefix xor), so all 2^n strings appear.

$n = 3$: 000 001 011 010 110 111 101 100
 ↘ ↘ ↘ ↘ ↘ ↘ ↘

Algorithm

- 1: for $i = 0$ to $2^n - 1$ do print $i \oplus (i \gg 1)$ as n bits
- 2: end for

Dry Run. $n = 2$: 0, 1, 3, 2 $\rightarrow$ 00, 01, 11, 10. ✓

C++17 implementation

```

1 // CSES 2205 - Gray Code
2 // g(i) = i xor (i >> 1): consecutive values differ in exactly one bit.
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 int main() {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10    int n;
11    cin >> n;
12    string out;
13    out.reserve((size_t)(n + 1) << n);
14    for (int i = 0; i < (1 << n); i++) {
15        int g = i ^ (i >> 1);
16        for (int b = n - 1; b >= 0; b--) out += char('0' + ((g >> b) & 1));
17        out += '\n';
18    }
19    cout << out;
20    return 0;
21 }
```

Complexity Analysis

Time: $O(n 2^n)$ (output size). **Space:** $O(n 2^n)$ for the buffered output.

Common Pitfalls and Edge Cases

- Print with leading zeros, most significant bit first.
- 2^{16} lines $\times$ 17 characters: build one string and print once instead of using `endl` per line.

2.5 A New Number System

Source: ICPC 2010–11 Kanpur Online Round (PYQ)**Level:** Easy

Problem Statement. A “1-based” number is a sequence of blocks of the digit 0. A block 0 sets a flag to 1; a block 00 sets it to 0; a block of $n > 2$ zeros appends $n - 2$ copies of the flag to a binary number. Each number ends with a token #; the input ends with `~`. Print each binary number in decimal (at most 30 bits).

Pattern Recognition

Parsing + building a binary number bit by bit: $v \leftarrow 2v + \text{bit}$.

Approach and intuition

Read whitespace-separated tokens; numbers may span several lines, which token reading handles automatically. Keep the current value and flag, and apply the rule for each block length. Appending a bit to the right of v is $v \leftarrow 2v + \text{bit}$ (a left shift by one).

Algorithm

```

1: for each token  $t$  do
2:   if  $t = \sim$  then stop
3:   else if  $t = \#$  then print  $v$ ; reset
4:   else if  $|t| = 1$  then  $flag \leftarrow 1$ 
5:   else if  $|t| = 2$  then  $flag \leftarrow 0$ 
6:   else repeat  $|t| - 2$  times:  $v \leftarrow 2v + flag$ 
7:   end if
8: end for

```

Dry Run. 0 0000 00 000 0 0000 #: flag 1, append 11; flag 0, append 0; flag 1, append 11.
Binary 11011 = 27. ✓

C++17 implementation

```

1 // ICPC 2010-11 Kanpur Online - A New Number System
2 // Blocks of '0': length 1 -> flag = 1, length 2 -> flag = 0,
3 // length n > 2 -> append (n - 2) copies of flag. Building a binary number
4 // bit by bit is value = 2 * value + bit.
5 #include <iostream>
6 #include <string>
7 using namespace std;
8
9 int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12     string tok;
13     long long value = 0;
14     int flag = 0;

```

```

15 while (cin >> tok) {
16     if (tok[0] == '~') break;           // end of input
17     if (tok == "#") {                   // end of one number
18         cout << value << '\n';
19         value = 0;
20         flag = 0;
21         continue;
22     }
23     int len = tok.size();
24     if (len == 1) flag = 1;
25     else if (len == 2) flag = 0;
26     else
27         for (int k = 0; k < len - 2; k++) value = 2 * value + flag;
28 }
29 return 0;
30 }

```

Complexity Analysis

Time: linear in the input size. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- The PDF prints “ n^2 ” — a lost subscript for $n - 2$.
- Reset both the value and the flag between numbers.

2.6 Bitwise Reversion

Source: Codeforces 2153B (Notion: Bitwise, 800)

Level: Easy

Problem Statement. Given $x, y, z \leq 10^9$, do non-negative a, b, c exist with $a \& b = x$, $b \& c = y$, $a \& c = z$?

Pattern Recognition

AND conditions only $\Rightarrow$ decide every bit independently.

Approach and intuition

Look at one bit. If it is set in two of the targets, e.g. in $x = a \& b$ and $y = b \& c$, then a , b and c all have it, so $a \& c$ has it too: the bit must be set in all three. Every other pattern is achievable:

- in none: set the bit nowhere;
- in exactly one, say x : set it in a and b only;
- in all three: set it in a , b , c .

So the answer is NO iff some bit is set in exactly two of x, y, z :

$$(x \& y \& \neg z) \mid (x \& z \& \neg y) \mid (y \& z \& \neg x) \neq 0.$$

Algorithm

```
1:  $t \leftarrow (x \& y \& \neg z) \mid (x \& z \& \neg y) \mid (y \& z \& \neg x)$ 
2: return  $t = 0$ 
```

Dry Run. $x = 4 = 0100_2$, $y = 8 = 1000_2$, $z = 12 = 1100_2$: bit 2 is in x and z but not $y \Rightarrow$ NO. $x = 3$, $y = 2$, $z = 6$: bit 1 in all three, bit 0 only in x , bit 2 only in $z \Rightarrow$ YES. ✓ (Verified on all 16^3 triples against brute force.)

C++17 implementation

```
1 // Codeforces 2153B - Bitwise Reversion
2 // Bits are independent. If a bit is set in two of x, y, z (say x = a&b and
3 // y = b&c), then a, b, c all have it, so the third AND has it too.
4 // => impossible exactly when some bit is set in exactly two of x, y, z.
5 #include <iostream>
6 using namespace std;
7
8 bool possible(long long x, long long y, long long z) {
9     long long exactlyTwo = (x & y & ~z) | (x & z & ~y) | (y & z & ~x);
10    return exactlyTwo == 0;
11 }
12
13 int main() {
14     ios::sync_with_stdio(false);
15     cin.tie(nullptr);
16     int t;
17     cin >> t;
18     while (t--) {
19         long long x, y, z;
20         cin >> x >> y >> z;
21         cout << (possible(x, y, z) ? "YES" : "NO") << '\n';
22     }
23     return 0;
24 }
```

Complexity Analysis

Time: $O(1)$ per test. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- $\neg z$ on a signed type sets the high bits too; harmless here because it is ANDed with non-negative values.

2.7 Deletion Sort

Source: Codeforces 2200B (Notion: Bitwise, 800)

Level: Easy

Problem Statement. An array of $n \leq 10$ positive integers. While it is not non-decreasing, you must delete one element of your choice. Minimise the number of elements left when the game ends.

Pattern Recognition

The tags say bitmask brute force (2^{10} subsets), but a one-line invariant solves it: *keep an inversion alive*.

Approach and intuition

If the array is already sorted, the game ends at once: answer n . Otherwise there is an inversion $a_i > a_j$ with $i < j$. As long as more than two elements remain, delete an element other than a_i and a_j : the inversion survives, so the array is still unsorted and the game continues. With two elements left (exactly the inversion), delete one; a single element is sorted. Answer 1.

Algorithm

1: **return** (a non-decreasing) ? n : 1

Dry Run. [1, 4, 2, 3]: unsorted, answer 1. [6, 7]: sorted, answer 2. ✓

C++17 implementation

```

1 // Codeforces 2200B - Deletion Sort
2 // A sorted array ends the game at once (answer n). Otherwise keep some
3 // inversion pair alive while deleting everything else: answer 1.
4 #include <iostream>
5 #include <vector>
6 using namespace std;
7
8 int main() {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11    int t;
12    cin >> t;
13    while (t--) {
14        int n;
15        cin >> n;
16        vector<int> a(n);
17        for (int& x : a) cin >> x;
18        bool sorted = true;
19        for (int i = 0; i + 1 < n; i++)
20            if (a[i] > a[i + 1]) sorted = false;
21        cout << (sorted ? n : 1) << '\n';
22    }
23    return 0;
24 }
```

Complexity Analysis

Time: $O(n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- Equal neighbours are allowed (“non-decreasing”).
- If you do write the bitmask brute force, remember the game stops as soon as the array is sorted — you cannot keep deleting.

2.8 ORXOR

Source: AtCoder ABC197 C (Notion: Bitwise)

Level: Easy–Medium

Problem Statement. Split an array of $N \leq 20$ numbers ($< 2^{30}$) into contiguous non-empty blocks; take the OR of each block and the XOR of those ORs. Minimise the result.

Pattern Recognition

$N \leq 20$ and a “cut or not” decision between neighbours $\Rightarrow$ enumerate the 2^{N-1} cut masks.

Approach and intuition

There are $N - 1$ gaps; each is cut or not, so a bitmask of $N - 1$ bits describes a partition. For each mask sweep the array, OR-ing into the current block and closing the block at a cut (or at the end) by xoring its OR into the total. $2^{19} \cdot 20 \approx 10^7$ steps.

Algorithm

```

1: for  $mask = 0$  to  $2^{N-1} - 1$  do
2:    $x \leftarrow 0, o \leftarrow 0$ 
3:   for  $i = 0$  to  $N - 1$  do
4:      $o \leftarrow o \mid a_i$ 
5:     if  $i = N - 1$  or bit  $i$  of  $mask$  then  $x \leftarrow x \oplus o; o \leftarrow 0$ 
6:     end if
7:   end for
8:    $best \leftarrow \min(best, x)$ 
9: end for

```

Dry Run. $[1, 5, 7]$: $[1, 5][7]$ gives $5 \oplus 7 = 2$; $[1][5][7]$ gives 3; $[1][5, 7]$ gives $1 \oplus 7 = 6$; $[1, 5, 7]$ gives 7. Minimum 2. ✓

C++17 implementation

```

1  // AtCoder ABC197 C - ORXOR
2  // N <= 20: enumerate the 2^(N-1) ways to cut between neighbours.
3  // Bit i of mask set <=> cut after position i.
4  #include <algorithm>
5  #include <iostream>
6  #include <vector>
7  using namespace std;
8
9  int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12     int n;
13     cin >> n;
14     vector<int> a(n);
15     for (int& x : a) cin >> x;
16     int best = (1 << 30) + 5;
17     for (int mask = 0; mask < (1 << (n - 1)); mask++) {
18         int xr = 0, orv = 0;
19         for (int i = 0; i < n; i++) {
20             orv |= a[i];
21             if (i == n - 1 || (mask >> i & 1)) { // close the current block
22                 xr ^= orv;
23                 orv = 0;
24             }
25         }
26         best = min(best, xr);
27     }
28     cout << best << '\n';

```

```

29     return 0;
30 }

```

Complexity Analysis

Time: $O(N 2^{N-1})$. **Space:** $O(N)$.

Common Pitfalls and Edge Cases

- $N = 1$: there is one mask (0) and the answer is a_1 .
- Initialise the best with something $\geq 2^{30}$, not 0.

2.9 Jackpot

Source: ICPC 2013–14 Kharagpur Online Round (PYQ)

Level: Hard

Problem Statement. A loyalty account starts with 1 point; once per week the points either increase by 1 or double. Alice and Bob each choose a sub-target, reach it in parallel as fast as possible, and then merge their points to hit exactly N ($2 \leq N \leq 10^9$, $T \leq 1000$). Find the minimum number of weeks.

Pattern Recognition

Reaching v from 1 with $+1/\times 2$ is reading v in binary. Splitting $N = a + b$ with carries between bits $\Rightarrow$ a **digit DP over the bits**.

Approach and intuition

One person. Work backwards from v : if v is even, the last step could have been a doubling; if odd, it was $+1$. Greedy reversal is optimal, giving

$$f(v) = (\text{number of bits} - 1) + (\text{popcount} - 1) = hb(v) + \text{popcount}(v) - 1,$$

where $hb(v)$ is the index of the highest set bit ($f(5) = f(101_2) = 2 + 2 - 1 = 3$: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$). This formula matches a BFS for all $v < 300$.

Two people. They work in parallel, so the answer is $\min_{a+b=N, a,b \geq 1} \max(f(a), f(b))$. N is up to 10^9 , so we cannot try every a .

Digit DP. Decide the bits of a and b from the top bit 30 down to 0. At bit p we know which carry c must leave this bit upward; we choose bits x, y and an incoming carry c' with $x + y + c' = N_p + 2c$. The first set bit of a (from the top) costs $p + 1$ (it fixes $hb = p$ and is one set bit); every later set bit costs 1. State: (carry c , a started?, b started?, cost of a so far) $\mapsto$ minimum cost of b . At the end no carry may enter bit 0 and both numbers must be non-zero; the answer is $\min \max(\text{cost}_a, \text{cost}_b) - 1$.

Algorithm

- 1: $dp[c = 0][0][0][0] \leftarrow 0$, all else ∞
- 2: **for** $p = 30$ **down to** 0 **do**
- 3: **for** each state $(c, s_a, s_b, c_a) \mapsto c_b$ and $x, y, c' \in \{0, 1\}$ with $x + y + c' = N_p + 2c$ **do**
- 4: $c'_a \leftarrow c_a + [x](s_a ? 1 : p + 1)$, $c'_b \leftarrow c_b + [y](s_b ? 1 : p + 1)$
- 5: relax $nd[c'][s_a \vee x][s_b \vee y][c'_a]$ with c'_b
- 6: **end for**
- 7: $dp \leftarrow nd$

8: end for

9: return $\min_{c_a} \max(c_a, dp[0][1][1][c_a]) - 1$

Dry Run. $N = 10$: $a = b = 5$, $f(5) = 3$, answer 3. $N = 55$: e.g. $15 + 40$ with $f(15) = 3 + 4 - 1 = 6$, $f(40) = 5 + 2 - 1 = 6$; no split does better. Answer 6. $N = 2$: $1 + 1$, answer 0. ✓ (DP checked against the brute-force split for every $N \leq 3000$.)

C++17 implementation

```

1 // ICPC 2013-14 Kharagpur Online - Jackpot
2 // Reaching v from 1 with (+1, x2) takes f(v) = hb(v) + popcount(v) - 1 steps
3 // (hb = index of the highest set bit). Alice and Bob work in parallel, so
4 // answer = min over a + b = N of max(f(a), f(b)).
5 // Digit DP over bits from the top, adding a + b = N with carries:
6 // state (carry that this bit must send upward, a started?, b started?, costA)
7 // value = minimum costB.
8 #include <algorithm>
9 #include <cstring>
10 #include <iostream>
11 using namespace std;
12
13 const int INF = 1e9;
14 const int BITS = 31; // N <= 1e9 < 2^30
15 const int MAXC = 2 * BITS + 2;
16
17 int solve(long long N) {
18     // dp[c][sa][sb][ca] = min cb ; c = carry out of the current bit
19     static int dp[2][2][2][MAXC], nd[2][2][2][MAXC];
20     auto fill = [](int (*d)[2][2][MAXC]) {
21         for (int c = 0; c < 2; c++) for (int x = 0; x < 2; x++)
22             for (int y = 0; y < 2; y++) for (int k = 0; k < MAXC; k++) d[c][x][y][k] =
23                 INF;
24     };
25     fill(dp);
26     dp[0][0][0][0] = 0; // nothing may carry out of the top bit
27     for (int p = BITS - 1; p >= 0; p--) {
28         fill(nd);
29         int bit = (N >> p) & 1;
30         for (int c = 0; c < 2; c++) for (int sa = 0; sa < 2; sa++)
31             for (int sb = 0; sb < 2; sb++) for (int ca = 0; ca < MAXC; ca++) {
32                 int cb = dp[c][sa][sb][ca];
33                 if (cb == INF) continue;
34                 for (int x = 0; x < 2; x++) for (int y = 0; y < 2; y++) for (int cin_ = 0;
35                     cin_ < 2; cin_++) {
36                     if (x + y + cin_ != bit + 2 * c) continue;
37                     // setting a bit costs 1; the first (highest) one also costs p
38                     int na = ca + (x ? (sa ? 1 : p + 1) : 0);
39                     int nb = cb + (y ? (sb ? 1 : p + 1) : 0);
40                     if (na >= MAXC) continue;
41                     int& ref = nd[cin_][sa | x][sb | y][na];
42                     ref = min(ref, nb);
43                 }
44             }
45         memcpy(dp, nd, sizeof dp);
46     }
47     int best = INF;
48     for (int ca = 0; ca < MAXC; ca++) // no carry into bit 0, both >= 1
49         if (dp[0][1][1][ca] != INF) best = min(best, max(ca, dp[0][1][1][ca]) - 1);
50 }

```

```

48     return best;
49 }
50
51 int main() {
52     ios::sync_with_stdio(false);
53     cin.tie(nullptr);
54     int t;
55     cin >> t;
56     while (t--) {
57         long long n;
58         cin >> n;
59         cout << solve(n) << '\n';
60     }
61     return 0;
62 }

```

Complexity Analysis

Time: $31 \text{ bits} \times 512 \text{ states} \times 8 \text{ choices} \approx 1.3 \cdot 10^5$ steps per test, $\approx 1.3 \cdot 10^8$ for $T = 1000$.
Space: $O(64)$ states per layer.

Common Pitfalls and Edge Cases

- Weeks count *in parallel*: it is max, not +.
- Both sub-targets must be ≥ 1 (everyone starts with one point).
- Trying only $a = \lfloor N/2 \rfloor$ fails: $55 = 27 + 28$ gives $\max(7, 6) = 7$.

2.10 Express the Number

Source: ICPC 2020–21 Amritapuri Preliminary, problem 2 (PYQ)

Level: Easy–Medium

Reconstructed Statement

Only the official solution slides survive (CodeDrills is offline). They say: “we can use only odd powers; an even power can be written as a sum of odd powers; if the top power $2^k > x$ has k odd subtract it, if k is even subtract 2^{k-1} twice; once $n \leq x$ write it as $A[1]$; impossible for odd n with $x = 0$; complexity $O(\log n)$.” The statement below is the reconstruction that matches every line of the slides. The greedy was checked against an exhaustive DP for all $n \leq 1500$ and nine values of x .

Problem Statement. Given n and x , write $n = A_1 + A_2 + \dots + A_m$ where $0 \leq A_1 \leq x$ and every other term is an *odd* power of two ($2, 8, 32, 128, \dots$). Minimise m , or print -1 if impossible.

Pattern Recognition

Representations with a restricted set of powers of two: process the binary expansion from the top bit, and rewrite forbidden powers with allowed ones ($2^{2j} = 2^{2j-1} + 2^{2j-1}$).

Approach and intuition

While $n > x$ we must spend terms. Let 2^k be the top bit of n .

- k odd: subtract 2^k (one term). Using smaller odd powers instead would need at least two terms for the same amount.
- k even: 2^k is not allowed; the largest allowed power below n is 2^{k-1} . Subtract it (one term); after that the top bit is at most $k-1$ (odd) and the loop continues.

When $n \leq x$, the rest becomes A_1 . If $x = 0$ and n is odd we can never finish, since all odd powers of two are even.

Algorithm

```

1: if  $x = 0$  and  $n$  odd then return  $-1$ 
2: end if
3:  $m \leftarrow 0$ 
4: while  $n > x$  do
5:    $k \leftarrow \text{hb}(n)$ ;  $n \leftarrow n - (k \text{ odd} ? 2^k : 2^{k-1})$ ;  $m \leftarrow m + 1$ 
6: end while
7: return  $m + 1$ 

```

Dry Run. $n = 10, x = 3$: top bit $8 = 2^3$ (odd), $n \rightarrow 2 \leq 3$: $m = 1$, answer 2 ($10 = 2 + 8$).
 $n = 10, x = 0$: $10 \rightarrow 2 \rightarrow 0$, answer 3 ($A_1 = 0, 8, 2$). $n = 7, x = 0$: odd, -1 . ✓

C++17 implementation

```

1 // ICPC 2020-21 Amritapuri Preliminary - Express the Number
2 // (statement reconstructed from the official solution slides; see the book)
3 // Write  $n = A_1 + (\text{odd powers of two})$ ,  $0 \leq A_1 \leq x$ , using the fewest terms.
4 // While  $n > x$  remove the top bit  $2^k$  if  $k$  is odd, else remove  $2^{(k-1)}$ 
5 // ( $2^k = 2^{(k-1)} + 2^{(k-1)}$  with  $k-1$  odd). Odd  $n$  with  $x = 0$  is impossible.
6 #include <iostream>
7 using namespace std;
8
9 int highBit(unsigned long long v) { return 63 - __builtin_clzll(v); }
10
11 long long minTerms(unsigned long long n, unsigned long long x) {
12     if (x == 0 && (n & 1)) return -1;
13     long long terms = 0;
14     while (n > x) {
15         int k = highBit(n);
16         n -= (k & 1) ? (1ULL << k) : (1ULL << (k - 1));
17         terms++;
18     }
19     return terms + 1; // + the free part  $A_1 = n$ 
20 }
21
22 int main() {
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr);
25     int t;
26     cin >> t;
27     while (t--) {
28         unsigned long long n, x;
29         cin >> n >> x;
30         cout << minTerms(n, x) << '\n';
31     }
32     return 0;
33 }

```

Complexity Analysis**Time:** $O(\log n)$. **Space:** $O(1)$.**Common Pitfalls and Edge Cases**

- $2^0 = 1$ is an *even* power; it is never allowed.
- Use `unsigned long long` and `1ULL << k` for large n .

2.11 Beautiful XOR

Source: Codeforces 2162C (Notion: Bitwise, 1100)**Level:** Medium

Problem Statement. Given $a, b \leq 10^9$. One operation picks $0 \leq x \leq a$ (current a) and sets $a \leftarrow a \oplus x$. Transform a into b with at most 100 operations, or print -1 .

Pattern Recognition

Constructive xor: first find the invariant (the top bit can never rise), then use a universal intermediate state — **all ones** below the top bit.

Approach and intuition

Impossible case. If $x \leq a$ then $x < 2^{hb(a)+1}$, so $a \oplus x$ also stays below $2^{hb(a)+1}$: the top bit never increases. If $hb(b) > hb(a)$ the answer is -1 .

Construction. Let $h = hb(a)$ and $\text{ones} = 2^{h+1} - 1$.

- $a = b$: zero operations.
- $a \oplus b \leq a$: one operation with $x = a \oplus b$.
- otherwise two operations: $x_1 = a \oplus \text{ones}$ (its top bit h is 0, so $x_1 < 2^h \leq a$) turns a into ones ; then $x_2 = \text{ones} \oplus b \leq \text{ones}$ turns it into b .

Algorithm

```

1: if  $hb(b) > hb(a)$  then return  $-1$ 
2: end if
3: if  $a = b$  then return  $[]$ 
4: end if
5: if  $a \oplus b \leq a$  then return  $[a \oplus b]$ 
6: end if
7: return  $[a \oplus \text{ones}, \text{ones} \oplus b]$ 

```

Dry Run. $a = 9 = 1001_2$, $b = 6 = 0110_2$: $a \oplus b = 15 > 9$, so two steps: $x_1 = 9 \oplus 15 = 6 \leq 9$, $a = 15$; $x_2 = 15 \oplus 6 = 9 \leq 15$, $a = 6$. $a = 292$, $b = 929$: $hb(929) = 9 > 8$, -1 . ✓ (Any valid sequence is accepted; checked for all $1 \leq a, b < 64$ against BFS reachability.)

C++17 implementation

```

1 // Codeforces 2162C - Beautiful XOR
2 // Every reachable value stays below  $2^{(hb(a)+1)}$ , so  $b$  with a higher top bit
3 // is impossible. Otherwise: 0 moves if  $a = b$ , 1 move if  $a \oplus b \leq a$ , else two:
4 //  $x1 = a \oplus \text{ones}$  ( $a$  becomes all-ones below its top bit),  $x2 = \text{ones} \oplus b$ .

```

```

5 #include <iostream>
6 using namespace std;
7
8 int highBit(long long v) { return 63 - __builtin_clzll(v); }
9
10 int main() {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13     int t;
14     cin >> t;
15     while (t--) {
16         long long a, b;
17         cin >> a >> b;
18         if (highBit(b) > highBit(a)) { cout << -1 << '\n'; continue; }
19         if (a == b) { cout << 0 << "\n\n"; continue; }
20         if ((a ^ b) <= a) { cout << 1 << '\n' << (a ^ b) << '\n'; continue; }
21         long long ones = (1LL << (highBit(a) + 1)) - 1;
22         cout << 2 << '\n' << (a ^ ones) << ' ' << (ones ^ b) << '\n';
23     }
24     return 0;
25 }

```

Complexity Analysis

Time: $O(1)$ per test. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- The bound is the *current* a , which changes after each operation.
- For 0 operations print 0 and an (empty) second line.

2.12 The 67th XOR Problem

Source: Codeforces 2218E (Notion: Bitwise, 1200)

Level: Medium

Problem Statement. Array a of n values ($\sum n \leq 3105$, $a_i \leq 10^9$). Exactly $n - 1$ times: choose an element, let x be its current value, xor *every* element with x , and delete the chosen one. Maximise the value that remains.

Pattern Recognition

Simulate a few steps symbolically: repeated “xor everything with x ” **telescopes**. What looks like a game collapses to “max xor of a pair”.

Approach and intuition

Write the current array as $a_j \oplus S$, where S is the total mask applied so far ($S = 0$ initially). Removing element i uses $x = a_i \oplus S$, so the new mask is $S \oplus x = S \oplus a_i \oplus S = a_i$: after every operation the mask equals the *original* value of the element just removed. Hence the survivor j ends as $a_j \oplus a_i$, where i is the last removed element. Any ordered pair (i, j) is achievable, so

$$\text{answer} = \max_{i \neq j} a_i \oplus a_j.$$

Use a binary trie: insert numbers one by one and query the best partner from the top bit down.

Algorithm

- 1: insert a_1 into the trie; $best \leftarrow 0$
- 2: **for** $i = 2$ **to** n **do**
- 3: $best \leftarrow \max(best, \text{QUERY}(a_i))$; insert a_i
- 4: **end for**

Dry Run. [1, 2, 3]: pairs give 3, 2, 1, answer 3 (remove 3: [2, 1], then remove 2: [3]). [67, 67]: 0. Sample 3: the best pair is $767 \oplus 267 = 1012$. ✓ (Verified by simulating every removal order on random arrays.)

C++17 implementation

```

1 // Codeforces 2218E - The 67th XOR Problem
2 // After removing element i the whole array has been xored with a_i's
3 // ORIGINAL value (the running masks telescope). So the survivor j ends as
4 // a_j xor a_last, where "last" is the final removed element:
5 // answer = max over pairs i != j of a_i xor a_j (binary trie).
6 #include <algorithm>
7 #include <array>
8 #include <iostream>
9 #include <vector>
10 using namespace std;
11
12 struct Trie {
13     vector<array<int, 2>> nxt{{0, 0}};
14     void insert(int v) {
15         int u = 0;
16         for (int b = 29; b >= 0; b--) {
17             int c = v >> b & 1;
18             if (!nxt[u][c]) { nxt[u][c] = nxt.size(); nxt.push_back({0, 0}); }
19             u = nxt[u][c];
20         }
21     }
22     int bestXor(int v) const { // max v ^ w over inserted w
23         int u = 0, r = 0;
24         for (int b = 29; b >= 0; b--) {
25             int want = (v >> b & 1) ^ 1;
26             if (nxt[u][want]) { r |= 1 << b; u = nxt[u][want]; }
27             else u = nxt[u][want ^ 1];
28         }
29         return r;
30     }
31 };
32
33 int main() {
34     ios::sync_with_stdio(false);
35     cin.tie(nullptr);
36     int t;
37     cin >> t;
38     while (t--) {
39         int n;
40         cin >> n;
41         vector<int> a(n);
42         for (int& x : a) cin >> x;
43         Trie tr;
44         tr.insert(a[0]);
45         int best = 0;

```

```

46     for (int i = 1; i < n; i++) {
47         best = max(best, tr.bestXor(a[i]));
48         tr.insert(a[i]);
49     }
50     cout << best << '\n';
51 }
52 return 0;
53 }
```

Complexity Analysis

Time: $O(30n)$. **Space:** $O(30n)$ trie nodes.

Common Pitfalls and Edge Cases

- The survivor is xored with the last removed *original* value, not with the xor of all removed values.
- Here $\sum n \leq 3105$, so an $O(n^2)$ double loop also passes; the trie is the version that scales.

2.13 Maximize XOR, Minimize Operations

Source: Codeforces 2260C (Notion: Bitwise, 1300)

Level: Medium

Problem Statement. Given $x, y < 2^{29}$. One operation decreases x by 1 and increases y by 1 (only if $x > 0$). Find the maximum possible $x \oplus y$ and the minimum number of operations reaching it.

Pattern Recognition

An invariant ($x + y = s$) plus the identity $u + v = (u \oplus v) + 2(u \& v) \Rightarrow$ the maximum is s , reached exactly when u is a **submask** of s .

Approach and intuition

After k operations the pair is $(u, v) = (x - k, y + k)$ with $u + v = s = x + y$. Since $u \oplus v = s - 2(u \& v) \leq s$, the maximum possible value is s , and $u = 0$ (i.e. $k = x$) already achieves it. Equality needs $u \& v = 0$, i.e. u and $v = s - u$ share no bits, i.e. u is a submask of s . We need $u \leq x$ and want $k = x - u$ minimal, so we want the **largest submask of s that is $\leq x$** . Build it greedily from the top: take bit b of s whenever the value stays $\leq x$ (a higher bit outweighs all lower bits together).

Algorithm

```

1:  $s \leftarrow x + y$ ;  $u \leftarrow 0$ 
2: for  $b = 30$  down to  $0$  do
3:   if bit  $b$  of  $s$  and  $u \mid 2^b \leq x$  then  $u \leftarrow u \mid 2^b$ 
4:   end if
5: end for
6: return  $(s, x - u)$ 
```

Dry Run. $(6, 4)$: $s = 10 = 1010_2$. Bit 3: $8 > 6$, skip; bit 1: $2 \leq 6$, take. $u = 2$, answer 10 with $6 - 2 = 4$ operations ($2 \oplus 8 = 10$). $(3, 1)$: $s = 4$, $u = 0$, answer 4 3. ✓ (Checked for all $x, y < 64$.)

C++17 implementation

```

1 // Codeforces 2260C - Maximize XOR, Minimize Operations
2 //  $x + y = s$  never changes and  $u \text{ xor } v \leq u + v$ , with equality iff  $u \& v = 0$ .
3 //  $u = 0$  already gives  $s$ , so the maximum is  $s$ . Equality needs  $u$  to be a
4 // submask of  $s$ ; the fewest moves use the LARGEST submask of  $s$  that is  $\leq x$ .
5 #include <iostream>
6 using namespace std;
7
8 long long largestSubmaskAtMost(long long s, long long x) {
9     long long u = 0;
10    for (int b = 30; b >= 0; b--)
11        if ((s >> b & 1) && (u | (1LL << b)) <= x) u |= 1LL << b;
12    return u;
13 }
14
15 int main() {
16     ios::sync_with_stdio(false);
17     cin.tie(nullptr);
18     int t;
19     cin >> t;
20     while (t--) {
21         long long x, y;
22         cin >> x >> y;
23         long long s = x + y;
24         cout << s << ' ' << x - largestSubmaskAtMost(s, x) << '\n';
25     }
26     return 0;
27 }

```

Complexity Analysis

Time: $O(\log s)$ per test. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- s can be up to 2^{30} : loop from bit 30.
- “Largest submask” greedy must compare the full partial value with x , not just the single bit.

2.14 Xor Battle

Source: AtCoder AGC045 A (Notion: Bitwise, harder)

Level: Hard

Problem Statement. $x = 0$. In round $i = 1..N$ ($N \leq 200$) person S_i may replace x by $x \oplus A_i$ ($A_i \leq 10^{18}$) or do nothing. Person 0 wants $x = 0$ at the end, person 1 wants $x \neq 0$. Who wins with optimal play?

Pattern Recognition

Games decided at the end $\Rightarrow$ reason **backwards**. The future power of person 0 is exactly the **xor span** of their later numbers.

Approach and intuition

Scan rounds from last to first and let V be the span of person 0's numbers in the rounds already scanned (the future). Claim: person 0 wins from round i with current value x iff $x \in V$.

- Round of person 0 with A_i : they win iff $x \in V$ or $x \oplus A_i \in V$, i.e. $x \in V + \{0, A_i\} = \text{span}(V \cup \{A_i\})$. So add A_i to the basis.
- Round of person 1: they win if $x \notin V$ or $x \oplus A_i \notin V$. If $A_i \notin V$, one of x and $x \oplus A_i$ is outside V , so person 1 wins no matter what happened before. If $A_i \in V$, their move does not matter.

So the answer is 1 iff some person-1 number is not representable by person 0's *later* numbers.

Algorithm

```

1: basis  $\leftarrow \emptyset$ 
2: for  $i = N$  down to 1 do
3:   if  $S_i = 0$  then insert  $A_i$ 
4:   else if  $A_i \notin \text{span}(\text{basis})$  then return 1
5:   end if
6: end for
7: return 0

```

Dry Run. $A = (1, 2)$, $S = 10$: basis $\{2\}$; person 1's $1 \notin \text{span}\{2\}$, answer 1. $A = (1, 1)$: $1 \in \text{span}\{1\}$, answer 0. ✓ (Checked against a full minimax on random games.)

C++17 implementation

```

1 // AtCoder AGC045 A - Xor Battle
2 // Scan from the last round. Keep a linear (xor) basis of person 0's later
3 // numbers. If person 1 owns an  $A_i$  that the basis cannot produce, person 1
4 // wins: whatever  $x$  is later, 0 cannot undo that bit pattern.
5 #include <iostream>
6 #include <string>
7 #include <vector>
8 using namespace std;
9
10 struct XorBasis {
11     unsigned long long b[61] = {};
12     // returns true if v was independent (and inserts it)
13     bool insert(unsigned long long v) {
14         for (int i = 60; i >= 0; i--) {
15             if (!(v >> i & 1)) continue;
16             if (!b[i]) { b[i] = v; return true; }
17             v ^= b[i];
18         }
19         return false;
20     }
21     bool representable(unsigned long long v) const {
22         for (int i = 60; i >= 0; i--)
23             if (v >> i & 1) {
24                 if (!b[i]) return false;
25                 v ^= b[i];
26             }
27         return true;
28     }
29 }

```

```

29 };
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34     int T;
35     cin >> T;
36     while (T--) {
37         int n;
38         cin >> n;
39         vector<unsigned long long> a(n);
40         for (auto& v : a) cin >> v;
41         string s;
42         cin >> s;
43         XorBasis basis;
44         int result = 0;
45         for (int i = n - 1; i >= 0; i--) {
46             if (s[i] == '0') basis.insert(a[i]);
47             else if (!basis.representable(a[i])) { result = 1; break; }
48         }
49         cout << result << '\n';
50     }
51     return 0;
52 }

```

Complexity Analysis

Time: $O(N \log A)$ per test. **Space:** $O(\log A)$.

Common Pitfalls and Edge Cases

- Only person 0's *later* numbers help; earlier ones are already spent.
- $A_i \leq 10^{18} < 2^{60}$: the basis needs bits 0..59 (we keep 61).

2.15 XOR Matching

Source: AtCoder ABC126 F (Notion: Bitwise, stretch goal)

Level: Hard

Problem Statement. Build a sequence of length 2^{M+1} ($M \leq 17$) in which each of $0..2^M - 1$ appears exactly twice and, for every value, the xor of the segment between (and including) its two copies equals K . Print -1 if impossible.

Pattern Recognition

Constructive xor with a symmetric (palindromic) layout; the key fact is $0 \oplus 1 \oplus \dots \oplus (2^M - 1) = 0$ for $M \geq 2$.

Approach and intuition

Impossible: $K \geq 2^M$ (no segment can have a bit that no element has); $M = 1, K = 1$ (only 0011-type sequences exist and they give 0). **Small cases:** $M = 0$: 0 0 iff $K = 0$; $M = 1, K = 0$: 0 0 1 1.

$M \geq 2, K < 2^M$: print

$$\underbrace{0, 1, \dots, 2^M - 1}_{\text{without } K}, K, \underbrace{2^M - 1, \dots, 1, 0}_{\text{without } K}, K.$$

Case $v \neq K$. The first copy of v is in the ascending half and the second in the descending half. Between them lie every value $w > v$ (with $w \neq K$) twice — once on each side — and K once. Pairs cancel, the two copies of v cancel, and the segment xor is K .

Case $v = K$. The segment is K , the whole descending half, K . The descending half is the xor of all values except K , which is $0 \oplus K = K$ (the xor of all of $0..2^M - 1$ is 0). With the two copies of K cancelling, the segment xor is K .

Algorithm

- 1: **if** $M = 0$ **then return** $K = 0 ? 0 \ 0 : -1$
- 2: **end if**
- 3: **if** $M = 1$ **then return** $K = 0 ? 0 \ 0 \ 1 \ 1 : -1$
- 4: **end if**
- 5: **if** $K \geq 2^M$ **then return** -1
- 6: **end if**
- 7: **print** ascending (skip K), K , descending (skip K), K

Dry Run. $M = 2, K = 1$: 0 2 3 1 3 2 0 1. For 3: segment 3,1,3, xor 1. For 0: 0,2,3,1,3,2,0, xor 1. For 1: 1,3,2,0,1, xor 1. ✓ (All $M \leq 4, K < 40$ checked by a verifier.)

C++17 implementation

```

1 // AtCoder ABC126 F - XOR Matching
2 // For  $M \geq 2$  and  $K < 2^M$ : 0 1 .. ( $2^M-1$  without  $K$ )  $K$  (reverse)  $K$ 
3 // Between the two copies of  $v \neq K$  lie all numbers except  $v$ , plus  $K$ :
4 // xor of all of  $0..2^M-1$  is 0, so that block xors to  $K$ .
5 #include <iostream>
6 #include <string>
7 using namespace std;
8
9 int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12     int M;
13     long long K;
14     cin >> M >> K;
15     if (M == 0) { cout << (K == 0 ? "0 0" : "-1") << '\n'; return 0; }
16     if (M == 1) { cout << (K == 0 ? "0 0 1 1" : "-1") << '\n'; return 0; }
17     if (K >= (1LL << M)) { cout << -1 << '\n'; return 0; }
18     int top = (1 << M) - 1;
19     string out;
20     for (int v = 0; v <= top; v++) if (v != K) out += to_string(v) + ' ';
21     out += to_string(K) + ' ';
22     for (int v = top; v >= 0; v--) if (v != K) out += to_string(v) + ' ';
23     out += to_string(K) + '\n';
24     cout << out;
25     return 0;
26 }

```

Complexity Analysis

Time: $O(2^M)$. **Space:** $O(2^M)$ output.

Common Pitfalls and Edge Cases

- K can be up to 10^9 while $2^M \leq 131072$: check $K \geq 2^M$ first.
- $M = 1$ is special because $0 \oplus 1 \neq 0$.

Chapter 3

Number Theory

Number theory problems in a prelim are rarely deep — they are about knowing five tools cold (Euclid, fast power, Fermat, the sieve and floor-division blocks) and never overflowing a 64-bit integer.

3.1 Theory and Must-Know Patterns

GCD and LCM

- **Euclid:** $\gcd(a, b) = \gcd(b, a \bmod b)$, $\gcd(a, 0) = a$. It takes $O(\log \min(a, b))$ steps — the worst case is consecutive Fibonacci numbers.
- $\text{lcm}(a, b) = a / \gcd(a, b) \cdot b$ (divide first to avoid overflow). C++17 has `std::gcd` and `std::lcm` in `<numeric>`.
- $\gcd$ is associative, $\gcd(0, x) = x$, so prefix/suffix $\gcd$ arrays answer “gcd of everything except a_i ” in $O(1)$.
- $\gcd(a_\ell, \dots, a_r) = \gcd$ of all adjacent pair gcds in the range.

Modular arithmetic

Work modulo $p = 10^9 + 7$ (prime). Addition, subtraction and multiplication commute with mod; keep every intermediate below p and multiply in `long long` (or `__int128` for 64-bit moduli).

- **Fast power:** a^e in $O(\log e)$ by squaring: $a^e = (a^{\lfloor e/2 \rfloor})^2 \cdot a^{e \bmod 2}$.
- **Fermat’s little theorem:** for prime $p \nmid a$, $a^{p-1} \equiv 1$, so $a^{-1} \equiv a^{p-2}$ and exponents can be reduced modulo $p-1$: $a^e \equiv a^{e \bmod (p-1)}$.
- Division by $d =$ multiplication by d^{p-2} (only if $p \nmid d$).

Primes and divisors

- **Sieve of Eratosthenes** up to V : $O(V \log \log V)$. A “divisor sieve” (add d to every multiple) is $O(V \log V)$ because $\sum_{d \leq V} V/d \approx V \ln V$.
- For $n = \prod p_i^{k_i}$:

$$\tau(n) = \prod (k_i + 1), \quad \sigma(n) = \prod \frac{p_i^{k_i+1} - 1}{p_i - 1}, \quad \prod_{d|n} d = n^{\tau(n)/2}.$$

- Trial division up to $\sqrt{n}$ factorises $n \leq 10^{12}$ with 10^6 steps; **Miller–Rabin** with bases $2, 3, 5, \dots, 37$ is exact for all 64-bit integers.

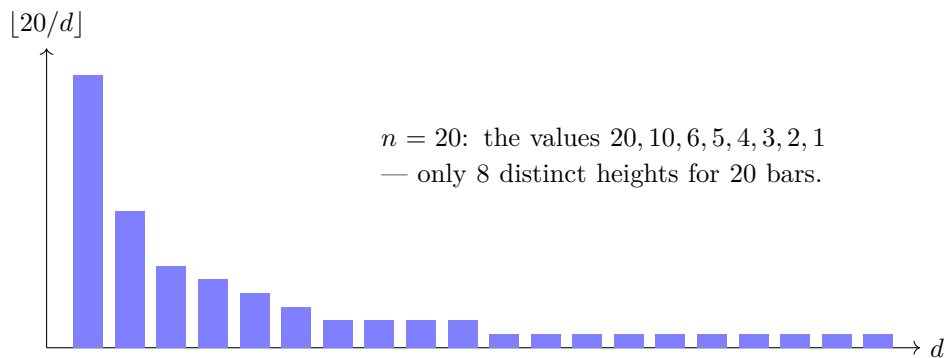
Counting tricks

- **Floor blocks:** $\lfloor n/d \rfloor$ takes at most $2\sqrt{n}$ distinct values; for a block starting at ℓ with value $q = \lfloor n/\ell \rfloor$ the block ends at $r = \lfloor n/q \rfloor$.

- **Inclusion–exclusion:** the count of $x \leq n$ divisible by at least one of $a_1, \dots, a_k$ is

$$\sum_{\emptyset \neq S \subseteq \{1..k\}} (-1)^{|S|+1} \left\lfloor \frac{n}{\text{lcm}(a_i : i \in S)} \right\rfloor.$$

- **Counting multiples:** “is there a pair with $\text{gcd} \geq d$?” — count the multiples of d with a frequency array: $\sum_d V/d = O(V \log V)$.



Number Theory Checklist

1. Every product of two values below 10^9 goes in `long long`; every product of two values below 10^{18} goes in `__int128`.
2. After subtraction modulo p add p before taking mod.
3. Exponents are reduced modulo $p - 1$, *not* p ; watch the base 0.
4. Special-case $n = 1$, 0^0 , and “no answer” outputs.

3.2 Calculate GCD

Source: AtCoder Math & Algorithm 015 (Notion: Number Theory)

Level: Easy

Problem Statement. Given $1 \leq A, B \leq 10^9$, print $\gcd(A, B)$.

Pattern Recognition

The Euclidean algorithm — the first tool of every number theory problem.

Approach and intuition

Any common divisor of a and b also divides $a - qb = a \bmod b$, and vice versa, so $\gcd(a, b) = \gcd(b, a \bmod b)$. The second argument strictly decreases and reaches 0, where $\gcd(a, 0) = a$. Every two steps the larger number at least halves, so there are $O(\log \min(a, b))$ steps.

Algorithm

```

1: while  $b \neq 0$  do
2:    $(a, b) \leftarrow (b, a \bmod b)$ 
3: end while
4: return  $a$ 

```

Dry Run. $(33, 88) \rightarrow (88, 33) \rightarrow (33, 22) \rightarrow (22, 11) \rightarrow (11, 0)$: 11. $(123, 777) \rightarrow (777, 123) \rightarrow (123, 39) \rightarrow (39, 6) \rightarrow (6, 3) \rightarrow (3, 0)$: 3. ✓

C++17 implementation

```

1 // AtCoder Math & Algorithm 015 - Calculate GCD
2 // Euclid:  $\gcd(a, b) = \gcd(b, a \bmod b)$ ,  $\gcd(a, 0) = a$ .
3 #include <iostream>
4 using namespace std;
5
6 long long gcdEuclid(long long a, long long b) {
7     while (b != 0) {
8         long long r = a % b;
9         a = b;
10        b = r;
11    }
12    return a;
13 }
14
15 int main() {
16     ios::sync_with_stdio(false);
17     cin.tie(nullptr);
18     long long a, b;
19     cin >> a >> b;
20     cout << gcdEuclid(a, b) << '\n';
21     return 0;
22 }

```

Complexity Analysis

Time: $O(\log \min(A, B))$. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- In contests just use `std::gcd` (C++17, <numeric>).
- The subtraction-only version (`a -= b`) is $O(\max)$ — TLE for $(10^9, 1)$.

3.3 The Last Cracker

Source: ICPC 2014–15 Kharagpur Online Re-contest (PYQ, ACM14KG2)

Level: Easy

Problem Statement. N crackers stand in a circle. The spark starts at cracker a ; each cracker fires and passes the spark to the next ($N \rightarrow 1$). The spark dies during pass $M + 1$. Which cracker fires last? ($N, M \leq 1000$.)

Pattern Recognition

Circular indexing $\Rightarrow$ convert to 0-based, add, take mod N , convert back.

Approach and intuition

After M successful passes the spark is at the cracker M steps after a ; that cracker fires and the next pass fails. In 0-based terms the position is $(a - 1 + M) \bmod N$, so the answer is $(a - 1 + M) \bmod N + 1$.

Algorithm

1: **return** $(a - 1 + M) \bmod N + 1$

Dry Run. $N = 5, a = 1$: $M = 3 \rightarrow 4, M = 5 \rightarrow 1, M = 19 \rightarrow (19 \bmod 5) + 1 = 5$. ✓

C++17 implementation

```

1 // ICPC 2014-15 Kharagpur Online Re-contest - The last cracker (ACM14KG2)
2 // After M passes the spark sits M steps clockwise of a: work 0-indexed mod N.
3 #include <iostream>
4 using namespace std;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9     int t;
10    cin >> t;
11    while (t--) {
12        long long n, a, m;
13        cin >> n >> a >> m;
14        cout << (a - 1 + m) % n + 1 << '\n';
15    }
16    return 0;
17 }
```

Complexity Analysis

Time: $O(1)$. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- $(a + M) \bmod N$ gives 0 instead of N — always shift to 0-based first.
- $M = 0$: the starting cracker fires and the spark dies; answer a .

3.4 Sieve of Eratosthenes

Source: Codeforces 2195A (Notion: Number Theory, 800)**Level:** Easy

Problem Statement. Given $n \leq 5$ integers $1 \leq a_i \leq 67$, can you choose a non-empty subset whose product is exactly 67?

Pattern Recognition

Product equal to a **prime** p : exactly one factor is p and all others are 1.

Approach and intuition

67 is prime. If a product of positive integers equals 67, one factor is divisible by 67; since every $a_i \leq 67$, that factor is 67 itself, and the remaining factors multiply to 1, so they are all 1. Conversely $\{67\}$ alone works. Answer: YES iff 67 occurs in the array.

Algorithm

1: **return** $(67 \in a)$

Dry Run. $[1, 7, 6, 7, 67]$: contains 67, YES. $[1, 3, 5, 7, 8]$: NO (the products $7 \cdot 8$, $3 \cdot 5 \cdot 7$, ... never equal a prime they do not contain). ✓

C++17 implementation

```

1 // Codeforces 2195A - Sieve of Eratosthenes
2 // 67 is prime and every a_i <= 67, so a product equal to 67 must use a 67
3 // (all other chosen factors must be 1). Answer: is 67 present?
4 #include <iostream>
5 using namespace std;
6
7 int main() {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10    int t;
11    cin >> t;
12    while (t--) {
13        int n;
14        cin >> n;
15        bool has67 = false;
16        for (int i = 0; i < n; i++) {
17            int x;
18            cin >> x;
19            if (x == 67) has67 = true;
20        }
21        cout << (has67 ? "YES" : "NO") << '\n';
22    }
23    return 0;
24 }
```

Complexity Analysis**Time:** $O(n)$. **Space:** $O(1)$.**Common Pitfalls and Edge Cases**

- The bound $a_i \leq 67$ matters: without it, 134 would not help either, but a subset search would be the safe general method.

3.5 Exponentiation

Source: CSES 1095 (Notion: Number Theory)**Level:** Easy**Problem Statement.** Answer $n \leq 2 \cdot 10^5$ queries $a^b \bmod (10^9 + 7)$ with $0 \leq a, b \leq 10^9$ and $0^0 = 1$.**Pattern Recognition**Large exponent $\Rightarrow$ **binary exponentiation**: square the base, multiply it in for every set bit of the exponent.**Approach and intuition**Write $b = \sum_k b_k 2^k$. Then $a^b = \prod_{k:b_k=1} a^{2^k}$, and $a^{2^{k+1}} = (a^{2^k})^2$. Scanning the bits of b from the low end we keep a^{2^k} and multiply it into the result when bit k is set: $O(\log b)$ multiplications.**Algorithm**

```

1: function POWER( $a, e, m$ )
2:    $r \leftarrow 1$ ;  $a \leftarrow a \bmod m$ 
3:   while  $e > 0$  do
4:     if  $e$  odd then  $r \leftarrow r \cdot a \bmod m$ 
5:     end if
6:      $a \leftarrow a^2 \bmod m$ ;  $e \leftarrow \lfloor e/2 \rfloor$ 
7:   end while
8:   return  $r$ 
9: end function

```

Dry Run. 3^4 : $e = 4$ (100_2): squares 3, 9, 81, only bit 2 set: $r = 81$. ✓**C++17 implementation**

```

1 // CSES 1095 - Exponentiation
2 // Binary exponentiation:  $a^b$  in  $O(\log b)$  multiplications.
3 #include <iostream>
4 using namespace std;
5
6 const long long MOD = 1'000'000'007LL;
7
8 long long power(long long b, long long e, long long m) {
9     long long r = 1 % m;
10    b %= m;
11    while (e > 0) {
12        if (e & 1) r = r * b % m;
13        b = b * b % m;

```

```

14     e >>= 1;
15 }
16 return r;
17 }
18
19 int main() {
20     ios::sync_with_stdio(false);
21     cin.tie(nullptr);
22     int n;
23     cin >> n;
24     while (n--) {
25         long long a, b;
26         cin >> a >> b;
27         cout << power(a, b, MOD) << '\n';    // 0^0 = 1 by the loop
28     }
29     return 0;
30 }

```

Complexity Analysis

Time: $O(\log b)$ per query. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- $0^0 = 1$ falls out naturally (r starts at 1, loop never runs).
- Use `long long`: $(10^9 + 6)^2 \approx 10^{18}$ fits, but not in `int`.

3.6 Exponentiation II

Source: CSES 1712 (Notion: Number Theory)

Level: Easy–Medium

Problem Statement. Answer $n \leq 10^5$ queries $a^{b^c} \bmod p$, $p = 10^9 + 7$, $0 \leq a, b, c \leq 10^9$, with $0^0 = 1$.

Pattern Recognition

A tower of exponents $\Rightarrow$ reduce the inner exponent modulo $p - 1$ (Fermat), **not** modulo p .

Approach and intuition

For $p \nmid a$, Fermat gives $a^{p-1} \equiv 1$, so $a^E \equiv a^{E \bmod (p-1)}$. Compute $E' = b^c \bmod (p-1)$ with fast power and return $a^{E'} \bmod p$. Because $0 \leq a < p$, the only base with $p \mid a$ is $a = 0$: then the answer is 0^{b^c} , which is 1 exactly when the exponent b^c equals 0 (that is, $b = 0$ and $c \geq 1$) and 0 otherwise.

Algorithm

- 1: **if** $a = 0$ **then return** $(b = 0 \text{ and } c > 0) ? 1 : 0$
- 2: **end if**
- 3: **return** `POWER(a, POWER(b, c, p - 1), p)`

Dry Run. $(3, 7, 1)$: $3^7 = 2187$. $(15, 2, 2)$: $15^4 = 50625$. $(0, 0, 5)$: $0^0 = 1$. $(0, 3, 0)$: $0^1 = 0$. ✓

C++17 implementation

```

1 // CSES 1712 - Exponentiation II
2 // Fermat: for p prime and p ∤ a, a^E = a^(E mod (p-1)) (mod p).
3 // Here 0 ≤ a < p, so only a = 0 needs care: 0^E = 1 iff E = b^c = 0.
4 #include <iostream>
5 using namespace std;
6
7 const long long MOD = 1'000'000'007LL;
8
9 long long power(long long b, long long e, long long m) {
10     long long r = 1 % m;
11     b %= m;
12     while (e > 0) {
13         if (e & 1) r = r * b % m;
14         b = b * b % m;
15         e >>= 1;
16     }
17     return r;
18 }
19
20 int main() {
21     ios::sync_with_stdio(false);
22     cin.tie(nullptr);
23     int n;
24     cin >> n;
25     while (n--) {
26         long long a, b, c;
27         cin >> a >> b >> c;
28         long long ans;
29         if (a == 0) {
30             bool expIsZero = (b == 0 && c > 0); // b^c = 0 only for b = 0, c ≥ 1
31             ans = expIsZero ? 1 : 0;
32         } else {
33             ans = power(a, power(b, c, MOD - 1), MOD);
34         }
35         cout << ans << '\n';
36     }
37     return 0;
38 }

```

Complexity Analysis

Time: $O(\log c + \log p)$ per query. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- Reducing the exponent modulo p instead of $p - 1$ is the classic bug.
- If b^c is a positive multiple of $p - 1$, the reduced exponent is 0; this is only wrong for $a = 0$, which we handle separately.

3.7 Counting Divisors

Source: CSES 1713 (Notion: Number Theory)

Level: Easy–Medium

Problem Statement. For each of $n \leq 10^5$ integers $x \leq 10^6$ print the number of divisors of x .

Pattern Recognition

Many queries over a small value range $\Rightarrow$ **precompute for every value** with a sieve, then answer in $O(1)$.

Approach and intuition

Instead of factorising each query, let every $d \leq V = 10^6$ add 1 to each of its multiples. Afterwards $\tau[x]$ is the number of divisors of x . The work is $\sum_{d \leq V} V/d \approx V \ln V \approx 1.4 \cdot 10^7$.

Algorithm

```

1: for  $d = 1$  to  $V$  do
2:   for  $m = d, 2d, \dots \leq V$  do  $\tau[m] \leftarrow \tau[m] + 1$ 
3:   end for
4: end for
5: answer each query with  $\tau[x]$ 

```

Dry Run. $16 = 2^4$: 5 divisors. 17: prime, 2. $18 = 2 \cdot 3^2$: $2 \cdot 3 = 6$. ✓

C++17 implementation

```

1 // CSES 1713 - Counting Divisors
2 // Divisor-count sieve: every d contributes 1 to each of its multiples.
3 // Total work  $\sum_{d \leq V} V/d = O(V \log V)$ .
4 #include <iostream>
5 #include <vector>
6 using namespace std;
7
8 int main() {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11    const int V = 1'000'000;
12    vector<int> tau(V + 1, 0);
13    for (int d = 1; d <= V; d++)
14        for (int m = d; m <= V; m += d) tau[m]++;
15    int n;
16    cin >> n;
17    while (n--) {
18        int x;
19        cin >> x;
20        cout << tau[x] << '\n';
21    }
22    return 0;
23 }

```

Complexity Analysis

Time: $O(V \log V + n)$. **Space:** $O(V)$.

Common Pitfalls and Edge Cases

- Trial division to $\sqrt{x}$ per query is $10^5 \cdot 10^3 = 10^8$ — also fine here, but the sieve is the pattern to remember.
- $\tau(n) = \prod (k_i + 1)$ with a smallest-prime-factor sieve is the $O(\log x)$ -per-query alternative.

3.8 Common Divisors

Source: CSES 1081 (Notion: Number Theory)

Level: Medium

Problem Statement. Given $n \leq 2 \cdot 10^5$ integers $x_i \leq 10^6$, find the maximum $\gcd(x_i, x_j)$ over pairs $i \neq j$.

Pattern Recognition

“Maximum gcd of a pair” $\Rightarrow$ flip the question: for each candidate d , count how many elements are **multiples of d** .

Approach and intuition

$\gcd(x_i, x_j) \geq d$ for some pair iff some $d' \geq d$ divides two elements. So the answer is the largest d that divides at least two elements. With a frequency array, check candidates from $V = 10^6$ downwards and count multiples of d ; stop at the first d with at least two. Total work $\leq \sum_d V/d$.

Algorithm

```

1:  $cnt[v] \leftarrow$  frequency of  $v$ 
2: for  $d = V$  down to 1 do
3:    $c \leftarrow \sum_{k \geq 1} cnt[kd]$ 
4:   if  $c \geq 2$  then return  $d$ 
5:   end if
6: end for

```

Dry Run. $[3, 14, 15, 7, 9]$: $d = 15, 14, \dots, 8$ each divide at most one element; $d = 7$ divides 14 and 7. Answer 7. ✓

C++17 implementation

```

1 // CSES 1081 - Common Divisors
2 // The answer is the largest d that divides at least two array elements:
3 // such a pair has gcd >= d, and the best pair's gcd itself is such a d.
4 #include <iostream>
5 #include <vector>
6 using namespace std;
7
8 int main() {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11    const int V = 1'000'000;
12    int n;
13    cin >> n;
14    vector<int> cnt(V + 1, 0);
15    for (int i = 0; i < n; i++) {
16        int x;
17        cin >> x;
18        cnt[x]++;
19    }
20    for (int d = V; d >= 1; d--) {
21        int multiples = 0;
22        for (int m = d; m <= V && multiples < 2; m += d) multiples += cnt[m];
23        if (multiples >= 2) { cout << d << '\n'; break; }
24    }
25    return 0;

```

26 }

Complexity Analysis**Time:** $O(V \log V)$. **Space:** $O(V)$.**Common Pitfalls and Edge Cases**

- Duplicates count: two equal values x give answer $\geq x$ (the frequency array handles it).
- The $O(n^2)$ pair loop is $4 \cdot 10^{10}$ — TLE.

3.9 Next Prime

Source: CSES 3396 (Notion: Number Theory)**Level:** Medium**Problem Statement.** For $t \leq 20$ values $n \leq 10^{12}$ print the smallest prime greater than n .**Pattern Recognition**

Prime gaps are tiny (below 10^{12} they are < 600), so just test $n+1, n+2, \dots$ — the whole problem is a fast primality test.

Approach and intuition

Trial division by all numbers up to $\sqrt{n} = 10^6$ per candidate would cost $\sim 10^6$ per test times hundreds of candidates. Instead use **Miller–Rabin**: write $n-1 = d \cdot 2^s$ with d odd; for a base a , a prime n must satisfy $a^d \equiv 1$ or $a^{d2^r} \equiv -1$ for some $r < s$. Testing the bases $2, 3, 5, \dots, 37$ is *deterministic* for all 64-bit n . Products of two 40-bit numbers overflow `long long`, so multiply in unsigned `__int128`.

Algorithm

- 1: $p \leftarrow n + 1$
- 2: **while not** MILLERRABIN(p) **do** $p \leftarrow p + 1$
- 3: **end while**
- 4: **return** p

Dry Run. $n = 1 \rightarrow 2$, $n = 2 \rightarrow 3$, $n = 3 \rightarrow 5$, $n = 42 \rightarrow 43$, $n = 1337 \rightarrow 1361$ (1338..1360 are all composite). ✓

C++17 implementation

```

1 // CSES 3396 - Next Prime
2 // Prime gaps below 1e12 are tiny (< 600), so test n+1, n+2, ... with a
3 // deterministic Miller-Rabin (bases 2..37 are exact for 64-bit integers).
4 #include <cstdint>
5 #include <iostream>
6 using namespace std;
7
8 using u64 = uint64_t;
9 using u128 = unsigned __int128;
10
11 u64 mulmod(u64 a, u64 b, u64 m) { return (u128)a * b % m; }
12

```

```

13 u64 powmod(u64 b, u64 e, u64 m) {
14     u64 r = 1;
15     b %= m;
16     while (e) {
17         if (e & 1) r = mulmod(r, b, m);
18         b = mulmod(b, b, m);
19         e >>= 1;
20     }
21     return r;
22 }
23
24 bool isPrime(u64 n) {
25     if (n < 2) return false;
26     for (u64 p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
27         if (n % p == 0) return n == p;
28     u64 d = n - 1;
29     int s = 0;
30     while (d % 2 == 0) { d /= 2; s++; }
31     for (u64 a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
32         u64 x = powmod(a, d, n);
33         if (x == 1 || x == n - 1) continue;
34         bool composite = true;
35         for (int r = 1; r < s; r++) {
36             x = mulmod(x, x, n);
37             if (x == n - 1) { composite = false; break; }
38         }
39         if (composite) return false;
40     }
41     return true;
42 }
43
44 int main() {
45     ios::sync_with_stdio(false);
46     cin.tie(nullptr);
47     int t;
48     cin >> t;
49     while (t--) {
50         u64 n;
51         cin >> n;
52         u64 p = n + 1;
53         while (!isPrime(p)) p++;
54         cout << p << '\n';
55     }
56     return 0;
57 }

```

Complexity Analysis

Time: $O(g \cdot 12 \log n)$ per test, g = prime gap. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- $n = 1$: the answer is 2 (the test must reject 1).
- $a * b \% m$ with $a, b \approx 10^{12}$ overflows 64 bits.

3.10 Modulo Equality

Source: ICPC 2017–18 Online Preliminary, Kharagpur (PYQ, EQUALMOD)

Level: Medium

Problem Statement. Arrays A and B of length N ($\sum N \leq 5 \cdot 10^5$, $A_i \leq 10^9$, $1 \leq B_i \leq 10^9$). One operation increments some A_i by 1. Find the minimum number of operations so that all $A_i \bmod B_i$ are equal.

Pattern Recognition

Choose a common target r and pay a piecewise-linear cost; the minimum of a function that is increasing between breakpoints lies **at a breakpoint**.

Approach and intuition

The common remainder r must satisfy $r < \min B$. With $c_i = A_i \bmod B_i$, reaching remainder r costs $r - c_i$ if $c_i \leq r$ and $r - c_i + B_i$ if $c_i > r$ (wrap around). So

$$\text{cost}(r) = Nr - \sum_i c_i + \sum_{c_i > r} B_i.$$

Between consecutive values of c the last sum is constant and the cost increases with r ; so the optimum is $r = 0$ or $r = c_i$ for some $c_i < \min B$. Sort the pairs by c and precompute suffix sums of B . Sweeping the candidates in increasing order, the index of the first $c_i > r$ only moves right, so every candidate is evaluated in amortised $O(1)$.

Algorithm

- 1: $c_i \leftarrow A_i \bmod B_i$; sort pairs (c_i, B_i) ; suffix sums suf
- 2: $best \leftarrow \text{cost}(0)$; $idx \leftarrow$ first index with $c > 0$
- 3: **for** each $c_i < \min B$ in sorted order **do**
- 4: advance idx past all $c \leq c_i$
- 5: $best \leftarrow \min(best, Nc_i - \sum c + suf[idx])$
- 6: **end for**

Dry Run. $A = (4, 2, 2)$, $B = (5, 3, 4)$: $c = (4, 2, 2)$, $\min B = 3$, candidates $r \in \{0, 2\}$. $\text{cost}(2) = 6 - 8 + 5 = 3$ (only $c = 4 > 2$ wraps, $B = 5$); $\text{cost}(0) = 0 - 8 + 12 = 4$. Answer 3. ✓

C++17 implementation

```

1 // ICPC 2017-18 Online Preliminary - Modulo Equality (EQUALMOD)
2 // Target remainder  $r < \min B$ .  $\text{Cost}(r) = \sum (r - c_i) + \sum_{\{c_i > r\}} B_i$ 
3 // with  $c_i = A_i \bmod B_i$ . Between two  $c$ -values the cost only grows with  $r$ ,
4 // so the optimum is  $r = 0$  or  $r = \text{some } c_i (< \min B)$ . Sweeping the sorted
5 // candidates keeps the "first  $c_i > r$ " pointer monotone.
6 #include <algorithm>
7 #include <iostream>
8 #include <utility>
9 #include <vector>
10 using namespace std;
11
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15     int T;
16     cin >> T;
17     while (T--) {

```

```

18     int n;
19     cin >> n;
20     vector<long long> A(n), B(n);
21     for (auto& x : A) cin >> x;
22     for (auto& x : B) cin >> x;
23     vector<pair<long long, long long>> cb(n);    // (c_i, B_i)
24     long long minB = B[0], sumC = 0;
25     for (int i = 0; i < n; i++) {
26         cb[i] = {A[i] % B[i], B[i]};
27         minB = min(minB, B[i]);
28         sumC += cb[i].first;
29     }
30     sort(cb.begin(), cb.end());
31     vector<long long> sufB(n + 1, 0);           // sum of B over cb[i..]
32     for (int i = n - 1; i >= 0; i--) sufB[i] = sufB[i + 1] + cb[i].second;
33     // cost(r) = n*r - sumC + (sum of B_i with c_i > r)
34     auto cost = [&](long long r, int firstAbove) {
35         return 1LL * n * r - sumC + sufB[firstAbove];
36     };
37     int idx = 0;                               // first index with c > 0
38     while (idx < n && cb[idx].first <= 0) idx++;
39     long long best = cost(0, idx);
40     for (int i = 0; i < n; i++) {               // candidates in sorted order
41         long long r = cb[i].first;
42         if (r >= minB) break;
43         while (idx < n && cb[idx].first <= r) idx++;    // idx only moves right
44         best = min(best, cost(r, idx));
45     }
46     cout << best << '\n';
47 }
48 return 0;
49 }

```

Complexity Analysis

Time: $O(N \log N)$. **Space:** $O(N)$.

Common Pitfalls and Edge Cases

- Forgetting $r < \min B$: a remainder must be smaller than *every* modulus.
- $N \cdot r$ and the B sums reach $5 \cdot 10^{14}$: `long long`.
- Trying every $r < \min B$ is up to 10^9 candidates — TLE.

3.11 Obtain Desired Standard Deviation

Source: ICPC 2017–18 Online Preliminary (PYQ, STDDEV)

Level: Easy–Medium

Problem Statement. Given $N \leq 10^5$ and $0 \leq \sigma \leq 10^5$, output N real numbers ($|a_i| \leq 10^8$) whose population standard deviation $\sqrt{\frac{1}{N} \sum (a_i - \mu)^2}$ equals σ (within 10^{-2}), or -1 .

Pattern Recognition

Constructive math: pick a **symmetric** array so the mean is 0, then solve one equation for the magnitude.

Approach and intuition

- $N = 1$: the deviation is always 0; answer 0 if $\sigma = 0$, else -1 .
- N even: $N/2$ copies of σ and $N/2$ of $-\sigma$. Mean 0, every squared deviation σ^2 , so the deviation is σ .
- N odd (≥ 3): $(N-1)/2$ pairs $\pm c$ and one 0. Mean 0 and variance $\frac{(N-1)c^2}{N} = \sigma^2$, so $c = \sigma\sqrt{N/(N-1)} \leq \sigma\sqrt{3/2} < 1.3 \cdot 10^5$.

Algorithm

- 1: if $N = 1$ then return $\sigma = 0 ? 0 : -1$
- 2: end if
- 3: $c \leftarrow \sigma$ if N even, else $\sigma\sqrt{N/(N-1)}$
- 4: print $c, -c, c, -c, \dots$ (and a final 0 if N is odd)

Dry Run. $N = 3, \sigma = 3$: $c = 3\sqrt{1.5} \approx 3.674235$; array 3.674235, $-3.674235, 0$: variance $2 \cdot 13.5/3 = 9$, deviation 3. ✓ (A checker recomputes the deviation for every test.)

C++17 implementation

```

1 // ICPC 2017-18 Online Preliminary - Obtain Desired Standard Deviation (STDDEV)
2 // N even : N/2 copies of +s and N/2 of -s -> mean 0, every deviation s.
3 // N odd : one 0 and (N-1)/2 pairs of +-c with (N-1) c^2 / N = s^2.
4 // N = 1 : the deviation is always 0.
5 #include <cmath>
6 #include <cstdio>
7 using namespace std;
8
9 int main() {
10     int T;
11     if (scanf("%d", &T) != 1) return 0;
12     while (T--) {
13         long long n, s;
14         scanf("%lld %lld", &n, &s);
15         if (n == 1) {
16             puts(s == 0 ? "0" : "-1");
17             continue;
18         }
19         double c = (n % 2 == 0) ? (double)s : s * sqrt((double)n / (n - 1));
20         for (long long i = 0; i < n; i++) {
21             double v;
22             if (n % 2 == 1 && i == n - 1) v = 0.0;           // the extra zero
23             else v = (i % 2 == 0) ? c : -c;
24             printf("%.9f%c", v, i + 1 < n ? ' ' : '\n');
25         }
26     }
27     return 0;
28 }

```

Complexity Analysis

Time: $O(N)$. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- The formula divides by N , not $N - 1$.
- Up to 10^6 numbers in total: `printf` with a fixed format is much faster than `cout` « `setprecision`.

3.12 Euclid, Sequence and Two Numbers

Source: Codeforces 2234A (Notion: Number Theory, 800)**Level:** Easy

Problem Statement. A Euclid sequence for $x \geq y$ is $a_1 = x$, $a_2 = y$, $a_{i+2} = a_i \bmod a_{i+1}$ (all terms positive). Given $n \leq 100$ numbers $b_i \leq 10^9$, can they be permuted into such a sequence? Print x y or -1 .

Pattern Recognition

Remainders are strictly smaller than the divisor $\Rightarrow$ the sequence is **forced** to be sorted in decreasing order. Construct, then verify.

Approach and intuition

For $i \geq 1$, $a_{i+2} = a_i \bmod a_{i+1} < a_{i+1}$, so $a_2 > a_3 > \dots > a_n$, and $a_1 \geq a_2$ by definition. The only possible order is descending; if two equal values exist they must be $a_1 = a_2$, which forces $a_3 = 0$ — allowed only when $n = 2$ (and descending order places them there anyway). So sort descending and check every triple.

Algorithm

- 1: sort b descending
- 2: **for** $i = 1$ **to** $n - 2$ **do**
- 3: **if** $b_i \bmod b_{i+1} \neq b_{i+2}$ **then return** -1
- 4: **end if**
- 5: **end for**
- 6: **return** (b_1, b_2)

Dry Run. $\{3, 8, 13, 5\} \rightarrow 13, 8, 5, 3$: $13 \bmod 8 = 5$, $8 \bmod 5 = 3$. Answer 13 8. $\{1, 2, 3, 4\} \rightarrow 4, 3, 2, 1$: $4 \bmod 3 = 1 \neq 2$, -1 . $\{1, 1, 1\}$: $1 \bmod 1 = 0 \neq 1$, -1 . ✓

C++17 implementation

```

1 // Codeforces 2234A - Euclid, Sequence and Two Numbers
2 // a_1 >= a_2 > a_3 > ... (each remainder is below its divisor and must stay
3 // positive), so the only candidate order is "sorted descending". Verify it.
4 #include <algorithm>
5 #include <functional>
6 #include <iostream>
7 #include <vector>
8 using namespace std;
9
10 int main() {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13     int t;
14     cin >> t;

```

```

15 while (t--) {
16     int n;
17     cin >> n;
18     vector<long long> b(n);
19     for (auto& x : b) cin >> x;
20     sort(b.begin(), b.end(), greater<>());
21     bool ok = true;
22     for (int i = 0; i + 2 < n && ok; i++)
23         if (b[i] % b[i + 1] != b[i + 2]) ok = false;
24     if (ok) cout << b[0] << ' ' << b[1] << '\n';
25     else cout << -1 << '\n';
26 }
27 return 0;
28 }

```

Complexity Analysis

Time: $O(n \log n)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- $n = 2$: always valid ($x = \max$, $y = \min$).
- Terms must be positive, so the sequence cannot contain the final 0 of Euclid's algorithm.

3.13 GCD on Blackboard

Source: AtCoder ABC125 C (Notion: Number Theory)

Level: Medium

Problem Statement. Replace one of $N \leq 10^5$ integers ($\leq 10^9$) by any integer in $[1, 10^9]$ to maximise the gcd of all N numbers.

Pattern Recognition

“Everything except element i ” for an associative operation $\Rightarrow$ **prefix and suffix arrays**.

Approach and intuition

If we replace A_i , the best new value is the gcd g_i of all other numbers itself (the gcd can never exceed g_i , and choosing g_i attains it). So the answer is $\max_i g_i$ with $g_i = \gcd(\text{pre}_{i-1}, \text{suf}_{i+1})$, using $\gcd(0, x) = x$ for the empty sides.

Algorithm

- 1: $\text{pre}_0 \leftarrow 0, \text{pre}_i \leftarrow \gcd(\text{pre}_{i-1}, A_i); \text{suf}_{N+1} \leftarrow 0, \text{suf}_i \leftarrow \gcd(\text{suf}_{i+1}, A_i)$
- 2: **return** $\max_i \gcd(\text{pre}_{i-1}, \text{suf}_{i+1})$

Dry Run. $[7, 6, 8]$: drop 7 $\rightarrow \gcd(6, 8) = 2$, drop 6 $\rightarrow 1$, drop 8 $\rightarrow 1$. Answer 2. $[12, 15, 18]$: drop 15 $\rightarrow 6$. ✓

C++17 implementation

```

1 // AtCoder ABC125 C - GCD on Blackboard
2 // Replacing  $A_i$  by the gcd of all others is optimal, so the answer is
3 //  $\max_i \gcd(\text{prefix}[i-1], \text{suffix}[i+1])$ .
4 #include <algorithm>

```

```

5 #include <iostream>
6 #include <numeric>
7 #include <vector>
8 using namespace std;
9
10 int main() {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13     int n;
14     cin >> n;
15     vector<long long> a(n), pre(n + 1, 0), suf(n + 2, 0);
16     for (auto& x : a) cin >> x;
17     for (int i = 0; i < n; i++) pre[i + 1] = gcd(pre[i], a[i]); // gcd(0, x) = x
18     for (int i = n - 1; i >= 0; i--) suf[i] = gcd(suf[i + 1], a[i]);
19     long long best = 0;
20     for (int i = 0; i < n; i++) best = max(best, gcd(pre[i], suf[i + 1]));
21     cout << best << '\n';
22     return 0;
23 }

```

Complexity Analysis

Time: $O(N \log V)$. **Space:** $O(N)$.

Common Pitfalls and Edge Cases

- $N = 2$: answer $\max(A_1, A_2)$ — replace the smaller by the larger.
- Recomputing the gcd of the other $N - 1$ elements for each i is $O(N^2)$.

3.14 Prime Multiples

Source: CSES 2185 (Notion: Number Theory)

Level: Medium

Problem Statement. Given $k \leq 20$ distinct primes a_i and $n \leq 10^{18}$, count the integers in $[1, n]$ divisible by at least one a_i .

Pattern Recognition

“At least one of” over a small family $\Rightarrow$ **inclusion–exclusion** over the 2^k subsets.

Approach and intuition

The number of multiples of m in $[1, n]$ is $\lfloor n/m \rfloor$; for distinct primes the lcm of a subset is its product. Inclusion–exclusion:

$$\left| \bigcup_i M_{a_i} \right| = \sum_{\emptyset \neq S} (-1)^{|S|+1} \left\lfloor \frac{n}{\prod_{i \in S} a_i} \right\rfloor.$$

Products larger than n contribute 0; detect them *before* they overflow by checking $prod > n/a_i$.

Algorithm

- 1: **for** $mask = 1$ **to** $2^k - 1$ **do**
- 2: $prod \leftarrow$ product of chosen primes (abort if $> n$)
- 3: $total \leftarrow total \pm \lfloor n/prod \rfloor$ (sign by parity of $|S|$)

4: end for

Dry Run. $n = 20$, primes 2, 5: $10 + 4 - 2 = 12$. ✓

C++17 implementation

```

1 // CSES 2185 - Prime Multiples
2 // Inclusion-exclusion over the  $2^k$  subsets of primes. A subset whose product
3 // exceeds  $n$  contributes 0 (and stops the multiplication before overflow).
4 #include <iostream>
5 #include <vector>
6 using namespace std;
7
8 int main() {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11    long long n;
12    int k;
13    cin >> n >> k;
14    vector<long long> a(k);
15    for (auto& x : a) cin >> x;
16    long long total = 0;
17    for (int mask = 1; mask < (1 << k); mask++) {
18        long long prod = 1;
19        bool tooBig = false;
20        for (int i = 0; i < k; i++) {
21            if (!(mask >> i & 1)) continue;
22            if (prod > n / a[i]) { tooBig = true; break; }
23            prod *= a[i];
24        }
25        if (tooBig) continue;
26        long long cnt = n / prod;
27        total += (__builtin_popcount(mask) & 1) ? cnt : -cnt;
28    }
29    cout << total << '\n';
30    return 0;
31 }

```

Complexity Analysis

Time: $O(k 2^k) \approx 2 \cdot 10^7$. **Space:** $O(k)$.

Common Pitfalls and Edge Cases

- Overflow: $10^{18} \cdot 2$ already overflows signed 64-bit; compare with n/a_i instead of multiplying first.
- Gray-code order or DFS with pruning makes it $O(2^k)$ if needed.

3.15 Sum of Divisors

Source: CSES 1082 (Notion: Number Theory)

Level: Medium

Problem Statement. Compute $\sum_{i=1}^n \sigma(i) \bmod (10^9 + 7)$ for $n \leq 10^{12}$, where $\sigma(i)$ is the sum of divisors of i .

Pattern Recognition

Swap the order of summation (count how often each divisor appears), then group equal values of $\lfloor n/d \rfloor$ — **floor blocks**.

Approach and intuition

Divisor d divides exactly $\lfloor n/d \rfloor$ numbers in $[1, n]$, so

$$\sum_{i \leq n} \sigma(i) = \sum_{d=1}^n d \left\lfloor \frac{n}{d} \right\rfloor.$$

$\lfloor n/d \rfloor$ has at most $2\sqrt{n} = 2 \cdot 10^6$ distinct values. For a block $[\ell, r]$ where it equals q (with $r = \lfloor n/q \rfloor$), add $q \cdot (\ell + \dots + r)$, computing the arithmetic series modulo p with the inverse of 2.

Algorithm

```

1:  $\ell \leftarrow 1$ 
2: while  $\ell \leq n$  do
3:    $q \leftarrow \lfloor n/\ell \rfloor$ ;  $r \leftarrow \lfloor n/q \rfloor$ 
4:    $ans \leftarrow ans + q \cdot \frac{(r-\ell+1)(\ell+r)}{2} \pmod{p}$ ;  $\ell \leftarrow r + 1$ 
5: end while

```

Dry Run. $n = 5$: blocks $[1, 1]$ ($q = 5$): 5; $[2, 2]$ ($q = 2$): 4; $[3, 5]$ ($q = 1$): 12. Total $21 = 1 + 3 + 4 + 7 + 6$. ✓

C++17 implementation

```

1 // CSES 1082 - Sum of Divisors
2 // sum_{i<=n} sigma(i) = sum_{d<=n} d * floor(n/d). floor(n/d) takes only
3 // O(sqrt n) distinct values; for each block [l, r] with the same q add
4 // q * (l + ... + r).
5 #include <iostream>
6 using namespace std;
7
8 const long long MOD = 1'000'000'007LL;
9 const long long INV2 = (MOD + 1) / 2;
10
11 long long rangeSum(long long l, long long r) {           // (l + ... + r) mod p
12     long long cnt = (r - l + 1) % MOD, ends = (l + r) % MOD;
13     return cnt * ends % MOD * INV2 % MOD;
14 }
15
16 int main() {
17     ios::sync_with_stdio(false);
18     cin.tie(nullptr);
19     long long n;
20     cin >> n;
21     long long ans = 0;
22     for (long long l = 1; l <= n; ) {
23         long long q = n / l, r = n / q;                // largest r with n/r = q
24         ans = (ans + q % MOD * rangeSum(l, r)) % MOD;
25         l = r + 1;
26     }
27     cout << ans << '\n';
28     return 0;

```

29 }

Complexity Analysis**Time:** $O(\sqrt{n})$. **Space:** $O(1)$.**Common Pitfalls and Edge Cases**

- $(r - \ell + 1)(\ell + r)$ reaches 10^{24} : reduce each factor modulo p first, then multiply by $2^{-1} = 5000000004$.
- Reduce q modulo p too before multiplying.

3.16 Divisor Analysis

Source: CSES 2182 (Notion: Number Theory)**Level:** Medium-Hard

Problem Statement. $n = \prod_{i=1}^m x_i^{k_i}$ is given by $m \leq 10^5$ distinct primes $x_i \leq 10^6$ with exponents $k_i \leq 10^9$. Print the number, sum and product of the divisors of n modulo $p = 10^9 + 7$.

Pattern Recognition

Multiplicative functions from a factorisation; exponents go modulo $p-1$. The product needs the divisor count *before* the current prime — build it incrementally.

Approach and intuition

- Count: $\tau = \prod (k_i + 1)$.
- Sum: $\sigma = \prod (1 + x_i + \dots + x_i^{k_i}) = \prod \frac{x_i^{k_i+1} - 1}{x_i - 1}$.
- Product: suppose the primes processed so far give c divisors with product P . Adding x^k turns each old divisor d into $d, dx, \dots, dx^k$, so the new product is

$$P^{k+1} \cdot (x^{0+1+\dots+k})^c = P^{k+1} \cdot (x^{k(k+1)/2})^c.$$

Here c is an exponent, so it is kept modulo $p-1$ (Fermat), not p .

(The formula $n^{\tau/2}$ is awkward because τ may be odd and n is astronomically large.)

Algorithm

- 1: $c \leftarrow 1, c' \leftarrow 1$ ($c \bmod p, c' \bmod p-1$), $S \leftarrow 1, P \leftarrow 1$
- 2: **for** each (x, k) **do**
- 3: $P \leftarrow P^{(k+1) \bmod (p-1)} \cdot (x^{k(k+1)/2 \bmod (p-1)})^{c'}$
- 4: $c \leftarrow c(k+1), c' \leftarrow c'(k+1) \bmod (p-1)$
- 5: $S \leftarrow S \cdot (x^{k+1} - 1)(x - 1)^{-1}$
- 6: **end for**

Dry Run. $n = 2^2 \cdot 3$: after 2^2 : $c = 3, P = 2^3 = 8$ ($1 \cdot 2 \cdot 4$), $S = 7$. After 3^1 : $P = 8^2 \cdot 3^3 = 1728$, $c = 6, S = 7 \cdot 4 = 28$. Output 6 28 1728. ✓ (Stress-tested against explicit divisor lists.)

C++17 implementation

```

1 // CSES 2182 - Divisor Analysis
2 //  $n = \prod x_i^{k_i}$ .
3 //  $count = \prod (k_i + 1)$ 
4 //  $sum = \prod (x_i^{k_i+1} - 1) / (x_i - 1)$ 
5 //  $product =$  built incrementally: adding  $x^k$  to a number with  $c$  divisors
6 // and divisor-product  $P$  gives  $P^{(k+1)} * (x^{(k(k+1)/2)})^c$ .
7 // Exponents are reduced mod  $p - 1$  (Fermat), so  $c$  is kept mod  $p - 1$  as well.
8 #include <iostream>
9 using namespace std;
10
11 const long long MOD = 1'000'000'007LL;
12
13 long long power(long long b, long long e, long long m) {
14     long long r = 1 % m;
15     b %= m;
16     while (e > 0) {
17         if (e & 1) r = r * b % m;
18         b = b * b % m;
19         e >>= 1;
20     }
21     return r;
22 }
23
24 int main() {
25     ios::sync_with_stdio(false);
26     cin.tie(nullptr);
27     int n;
28     cin >> n;
29     long long cnt = 1, cntPhi = 1, sum = 1, prod = 1; // cntPhi = count mod (p-1)
30     for (int i = 0; i < n; i++) {
31         long long x, k;
32         cin >> x >> k;
33         // product first: it needs the divisor count *before* this prime
34         long long tri = (__int128)k * (k + 1) / 2 % (MOD - 1); //  $k(k+1)/2$ 
35         prod = power(prod, (k + 1) % (MOD - 1), MOD) *
36             power(power(x, tri, MOD), cntPhi, MOD) % MOD;
37         cnt = cnt * ((k + 1) % MOD) % MOD;
38         cntPhi = cntPhi * ((k + 1) % (MOD - 1)) % (MOD - 1);
39         long long geo = (power(x, k + 1, MOD) - 1 + MOD) % MOD *
40             power(x - 1, MOD - 2, MOD) % MOD;
41         sum = sum * geo % MOD;
42     }
43     cout << cnt << ' ' << sum << ' ' << prod << '\n';
44     return 0;
45 }

```

Complexity Analysis

Time: $O(m \log p)$. **Space:** $O(1)$.

Common Pitfalls and Edge Cases

- $k(k+1)/2$ with $k = 10^9$ is $5 \cdot 10^{17}$ — compute in `__int128` (or divide the even factor first), then reduce.
- Using $c \bmod p$ as an exponent is wrong; the exponent lives modulo $p - 1$.

- $x - 1$ is never divisible by p since $x \leq 10^6$.

3.17 A Simple GCD Problem (Easy Version)

Source: Codeforces 2210C1 (Notion: Number Theory, 1200)

Level: Medium

Problem Statement. Array a of $n \leq 2 \cdot 10^5$ values ($\leq 10^9$). Each a_i may be changed at most once to some $m \neq a_i$ with $1 \leq m \leq a_i$. After all changes, the gcd of every subarray of length ≥ 2 must be unchanged. Maximise the number of changed positions.

Pattern Recognition

Reduce a condition on *all* subarrays to a condition on **adjacent pairs**: the gcd of a range equals the gcd of the gcds of its neighbouring pairs.

Approach and intuition

$\gcd(a_\ell, \dots, a_r) = \gcd(g_\ell, \dots, g_{r-1})$ with $g_i = \gcd(a_i, a_{i+1})$, because every element appears in some pair. So the condition is exactly: every g_i is preserved. Then a'_i must be a multiple of $L_i = \text{lcm}(g_{i-1}, g_i)$ (only the existing neighbours).

Setting $a'_i = L_i$ *everywhere* keeps every g_i . Indeed, L_i and L_{i+1} are both multiples of g_i ; and since $L_i \mid a_i$ and $L_{i+1} \mid a_{i+1}$, their gcd divides $\gcd(a_i, a_{i+1}) = g_i$. So the gcd is exactly g_i . As $L_i \mid a_i$, a smaller valid value exists iff $L_i < a_i$.

$$\text{answer} = \#\{i : L_i < a_i\}.$$

Algorithm

- 1: $g_i \leftarrow \gcd(a_i, a_{i+1})$
- 2: **for** each i **do**
- 3: $L \leftarrow \text{lcm}$ of the existing g_{i-1}, g_i ; count if $L < a_i$
- 4: **end for**

Dry Run. $[8, 10, 10, 12, 12, 14]$: $g = (2, 10, 2, 12, 2)$, $L = (2, 10, 10, 12, 12, 2)$; $L_1 = 2 < 8$ and $L_6 = 2 < 14$: answer 2. $[2, 4, 8, \dots, 256]$: only the last ($128 < 256$) changes, answer 1. ✓ (Brute-forced on all small arrays.)

C++17 implementation

```

1 // Codeforces 2210C1 - A Simple GCD Problem (Easy Version)
2 // gcd of a subarray (length >= 2) = gcd of its adjacent-pair gcds, so the
3 // condition is: every  $g_i = \gcd(a_i, a_{i+1})$  is preserved.  $a'_i$  must be a
4 // multiple of  $L_i = \text{lcm}(g_{i-1}, g_i)$ ; setting  $a'_i = L_i$  everywhere keeps
5 // all  $g_i$ , so index  $i$  can change iff  $L_i < a_i$ .
6 #include <iostream>
7 #include <numeric>
8 #include <vector>
9 using namespace std;
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14     int t;
15     cin >> t;

```

```

16 while (t--) {
17     int n;
18     cin >> n;
19     vector<long long> a(n), b(n);
20     for (auto& x : a) cin >> x;
21     for (auto& x : b) cin >> x;           // b_i = a_i in this version
22     vector<long long> g(n - 1);
23     for (int i = 0; i + 1 < n; i++) g[i] = gcd(a[i], a[i + 1]);
24     int ops = 0;
25     for (int i = 0; i < n; i++) {
26         long long L = 1;
27         if (i > 0) L = lcm(L, g[i - 1]);
28         if (i + 1 < n) L = lcm(L, g[i]);
29         if (L < a[i]) ops++;           // L divides a_i, so L <= a_i
30     }
31     cout << ops << '\n';
32 }
33 return 0;
34 }

```

Complexity Analysis

Time: $O(n \log V)$. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- The endpoints have only one neighbour: $L_1 = g_1$, $L_n = g_{n-1}$.
- $\text{lcm}(g_{i-1}, g_i)$ divides $a_i \leq 10^9$, so no overflow — but compute it as $g_{i-1} / \text{gcd} \cdot g_i$.

3.18 Maximum GCD on Whiteboard

Source: Codeforces 2156C (Notion: Number Theory, 1400)

Level: Medium–Hard

Problem Statement. $n \leq 2 \cdot 10^5$ numbers $1 \leq a_i \leq n$. You may erase at most k numbers, and any number of times split $x \geq 3$ into $x_1 \leq x_2 \leq x_3$ with $x_1 + x_2 + x_3 = x$, keeping only x_1 and x_3 . Maximise the gcd of the final board.

Pattern Recognition

Fix the answer d and count what *cannot* be fixed for it; values are $\leq n$, so every candidate d can be checked with prefix counts in $O(n/d)$.

Approach and intuition

Fix d . Multiples of d are already fine. A non-multiple x must be split into pieces that are multiples of d (or be erased). With $x_1 = d$, $x_3 = td$ and $d \leq x_2 = x - d - td \leq td$ we can reach every x with $d(t+2) \leq x \leq d(2t+1)$ for some $t \geq 1$; these intervals cover $\{3d\}$ and all of $[4d, \infty)$. Smaller x (below $4d$, not a multiple) cannot work: $x_1 \leq x/3 < d$ and pieces never grow. (Verified by exhaustive search for $d \leq 6$, $x < 60$.)

Criterion

d is achievable iff $\#\{i : a_i < 4d, d \nmid a_i\} \leq k$. Take the largest such d ($d = 1$ always works).

With a prefix count array, the number of values $< 4d$ is $\text{pre}[\min(4d - 1, n)]$ and the good ones among them are the counts of $d, 2d, 3d$.

Algorithm

```

1:  $cnt[v], pre[v]$  over  $1..n$ 
2: for  $d = n$  down to 1 do
3:    $bad \leftarrow pre[\min(4d - 1, n)] - cnt[d] - cnt[2d] - cnt[3d]$  (existing indices)
4:   if  $bad \leq k$  then return  $d$ 
5:   end if
6: end for

```

Dry Run. $[4, 9, 6, 8, 2, 6, 7, 8, 2]$, $k = 1$: $d = 2$: values < 8 that are odd: only 7, so $bad = 1 \leq 1$; larger d fail. Answer 2. With a second 7 added, $bad = 2 > 1$ and the answer drops to 1. ✓

C++17 implementation

```

1 // Codeforces 2156C - Maximum GCD on Whiteboard
2 // For a candidate d: a multiple of d is fine; a non-multiple x can be split
3 // into multiples of d iff  $x \geq 4d$  ( $x = d + x_2 + t*d$  with  $d \leq x_2 \leq t*d$ ).
4 // So d needs  $\#\{x < 4d, d \nmid x\} \leq k$  erasures. Take the largest such d.
5 #include <algorithm>
6 #include <iostream>
7 #include <vector>
8 using namespace std;
9
10 int main() {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13     int t;
14     cin >> t;
15     while (t--) {
16         int n, k;
17         cin >> n >> k;
18         vector<int> cnt(n + 1, 0), pre(n + 1, 0);
19         for (int i = 0; i < n; i++) {
20             int x;
21             cin >> x;
22             cnt[x]++;
23         }
24         for (int v = 1; v <= n; v++) pre[v] = pre[v - 1] + cnt[v];
25         int ans = 1;
26         for (int d = n; d >= 1; d--) {
27             long long lim = min<long long>(4LL * d - 1, n);
28             int below = pre[lim]; // values < 4d
29             int good = 0;
30             for (int m = d; m <= lim; m += d) good += cnt[m]; // d, 2d, 3d
31             if (below - good <= k) { ans = d; break; }
32         }
33         cout << ans << '\n';
34     }
35     return 0;
36 }

```

Complexity Analysis

Time: $O(n)$ per test. **Space:** $O(n)$.

Common Pitfalls and Edge Cases

- The threshold is $4d$, not $3d$: $x = 7$, $d = 2$ cannot be split ($2 + 2 + 3$ leaves 3).
- Erase removes one copy only; count with multiplicity.

3.19 Substring Problem

Source: ICPC 2013–14 Kharagpur Online Round (PYQ)**Level:** Medium

Problem Statement. How many times does the decimal string of K occur (overlaps allowed) in $123456789101112\dots N$? ($N, K \leq 10^6$, sum of N over tests $\leq 10^6$.)

Pattern Recognition

Read the constraints: the *sum* of N is small, so the concatenated string has at most $\approx 6.9 \cdot 10^6$ characters in total — build it and run **KMP**. No digit-counting formula needed.

Approach and intuition

Numbers up to 10^6 have at most 7 digits, so the text is at most $7 \cdot 10^6$ characters over all tests. KMP finds all (overlapping) occurrences in linear time: after a full match, continue from the failure link instead of restarting.

Algorithm

- 1: $T \leftarrow$ concatenation of $1..N$; $P \leftarrow$ digits of K
- 2: build the KMP failure function of P ; scan T , counting full matches

Dry Run. $N = 11$, $K = 1$: 1234567891011 contains 1 at 1, in 10, and twice in 11: 4. $N = 10$:
 2. $N = 100$, $K = 333$: in $\dots 32\ 33\ 34\dots$ the digits of 33 followed by the first digit of 34 form 333; that is the only occurrence, answer 1. ✓

C++17 implementation

```

1 // ICPC 2013-14 Kharagpur Online - Substring Problem
2 // Sum of N <= 1e6, so build "123...N" (under 7e6 chars in total) and count
3 // (overlapping) occurrences of the digits of K with KMP.
4 #include <iostream>
5 #include <string>
6 #include <vector>
7 using namespace std;
8
9 long long countOccurrences(const string& text, const string& pat) {
10     int m = pat.size();
11     vector<int> fail(m, 0);
12     for (int i = 1, j = 0; i < m; i++) {
13         while (j > 0 && pat[i] != pat[j]) j = fail[j - 1];
14         if (pat[i] == pat[j]) j++;
15         fail[i] = j;
16     }
17     long long cnt = 0;
18     for (int i = 0, j = 0; i < (int)text.size(); i++) {
19         while (j > 0 && text[i] != pat[j]) j = fail[j - 1];
20         if (text[i] == pat[j]) j++;
21         if (j == m) { cnt++; j = fail[j - 1]; } // allow overlaps

```

```

22     }
23     return cnt;
24 }
25
26 int main() {
27     ios::sync_with_stdio(false);
28     cin.tie(nullptr);
29     int t;
30     cin >> t;
31     while (t--) {
32         int n, k;
33         cin >> n >> k;
34         string text;
35         for (int x = 1; x <= n; x++) text += to_string(x);
36         cout << countOccurrences(text, to_string(k)) << '\n';
37     }
38     return 0;
39 }

```

Complexity Analysis

Time: $O(|T| + |P|)$ per test. **Space:** $O(|T|)$.

Common Pitfalls and Edge Cases

- Overlapping matches count (11 occurs twice in 111).
- `string::find` in a loop with start +1 also works for such a short pattern.

3.20 Steel Bar Segment Durability

Source: ICPC 2014–15 Amritapuri Online Round (PYQ)

Level: Medium

Problem Statement. A digit string S ($|S| \leq 2 \cdot 10^4$) and $Q \leq 5$ queries (M, L) with $M < 500$. Count the substrings whose decimal value is $\equiv L \pmod{M}$.

Pattern Recognition

Counting substrings by remainder with a small modulus $\Rightarrow$ DP over the right end with a **histogram of remainders**.

Approach and intuition

Let $cnt[r]$ be the number of substrings ending at the current position with value $\equiv r$. Appending digit d maps each such substring's remainder r to $(10r + d) \bmod M$, and the new single-digit substring adds remainder $d \bmod M$. After each position add $cnt[L]$ to the answer.

(With general M the prefix-value trick $W(i, j) = (P_j - P_{i-1}10^{j-i+1})$ needs 10 to be invertible modulo M ; the histogram DP works for every M .)

Algorithm

- 1: **for** each digit d of S **do**
- 2: $next[(10r + d) \bmod M] += cnt[r]$ for all r ; $next[d \bmod M] += 1$
- 3: $cnt \leftarrow next$; $ans \leftarrow ans + cnt[L]$
- 4: **end for**

Dry Run. $S = 23128765$, $M = 9$, $L = 3$: the substrings 3, 12, 876, 31287 — 4 in total. ✓

C++17 implementation

```

1 // ICPC 2014-15 Amritapuri Online - Steel Bar Segment Durability
2 // cnt[r] = number of segments ending at the current block with value = r
3 // (mod M). Extending every segment by digit d maps r -> (10 r + d) mod M,
4 // and the new one-block segment adds d mod M. O(|S| * M) per query.
5 #include <iostream>
6 #include <string>
7 #include <vector>
8 using namespace std;
9
10 long long countSegments(const string& s, int M, int L) {
11     vector<long long> cnt(M, 0), nxt(M, 0);
12     long long total = 0;
13     for (char ch : s) {
14         int d = ch - '0';
15         fill(nxt.begin(), nxt.end(), 0);
16         for (int r = 0; r < M; r++)
17             if (cnt[r]) nxt[(r * 10 + d) % M] += cnt[r];
18         nxt[d % M]++;
19         cnt.swap(nxt);
20         total += cnt[L];
21     }
22     return total;
23 }
24
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28     string s;
29     int q;
30     cin >> s >> q;
31     while (q--) {
32         int M, L;
33         cin >> M >> L;
34         cout << countSegments(s, M, L) << '\n';
35     }
36     return 0;
37 }

```

Complexity Analysis

Time: $O(|S| \cdot M)$ per query $\approx 10^7$. **Space:** $O(M)$.

Common Pitfalls and Edge Cases

- Substrings with leading zeros count ("05" has value 5).
- The answer is up to $|S|^2/2 = 2 \cdot 10^8$: fits in `int` but use `long long` anyway.

3.21 ISRO Digit Sum

Source: ICPC 2014–15 Amritapuri Online Round (PYQ)

Level: Hard

Problem Statement. N is given run-length encoded: $M \leq 10^4$ blocks (len_i, d_i) meaning digit d_i repeated $len_i \leq 10^{18}$ times (total length $\leq 10^{18}$). Compute $\sum_{x=1}^N f(x) \bmod (10^9 + 7)$, where $f(x)$ is the digit sum of x .

Pattern Recognition

Classic “sum of digit sums up to N ” by scanning N ’s digits left to right — but with 10^{18} digits, each run of equal digits must be summed in **closed form** with geometric series.

Approach and intuition

Per digit. Scan N from the left. At a digit d with k digits to its right and prefix digit sum ps , the numbers that agree with N so far and then take a smaller digit $c < d$ are followed by all 10^k tails. Their total digit sum is

$$\sum_{c < d} \left(10^k (ps + c) + 45k 10^{k-1} \right) = d 10^k ps + 10^k \frac{d(d-1)}{2} + 45 d k 10^{k-1},$$

using that all k -digit tails have digit sum $45k 10^{k-1}$. Finally add $f(N)$ itself.

Per run. Inside a run of len digits d , the t -th digit has $k = k_{\max} - t$ and $ps = ps_0 + td$. Summing over the run needs only

$$G_0 = \sum_{j=k_{\min}}^{k_{\max}} 10^j, \quad G_1 = \sum_{j=k_{\min}}^{k_{\max}} j 10^j,$$

because $\sum t 10^k = k_{\max} G_0 - G_1$ and $\sum k 10^{k-1} = G_1/10$. Both have closed forms:

$$\sum_{i=0}^{n-1} 10^i = \frac{10^n - 1}{9}, \quad \sum_{i=0}^{n-1} i 10^i = \frac{10(1 - n 10^{n-1} + (n-1)10^n)}{81},$$

shifted by $10^{k_{\min}}$. Powers with exponents up to 10^{18} use fast power; factors like n and $k_{\max}$ are reduced modulo p .

Algorithm

```

1:  $L \leftarrow \sum len_i$ ;  $used \leftarrow 0$ ;  $ps \leftarrow 0$ ;  $ans \leftarrow 0$ 
2: for each run  $(n, d)$  do
3:    $k_{\max} \leftarrow L - used - 1$ ,  $k_{\min} \leftarrow k_{\max} - n + 1$ ; compute  $G_0, G_1$ 
4:    $ans \mathrel{+}= d(ps G_0 + d(k_{\max} G_0 - G_1)) + G_0 \frac{d(d-1)}{2} + 45 d G_1/10$ 
5:    $ps \mathrel{+}= d \cdot n$ ;  $used \mathrel{+}= n$ 
6: end for
7: return  $ans + ps$ 
```

Dry Run. $N = 10$ (runs $(1, 1), (1, 0)$): first run $k = 1$, $G_0 = G_1 = 10$, $ps = 0$: contributes $0 + 0 + 45 \cdot 1 \cdot 10/10 = 45$ (the numbers 0..9). Second run $d = 0$ contributes 0. Add $f(10) = 1$: 46. ✓ (Stress-tested against direct summation for all RLE numbers $\leq 2 \cdot 10^5$ and against the closed form $45k 10^{k-1}$ for $N = \underbrace{9 \cdots 9}_{10^{18}})$

C++17 implementation

```

1 // ICPC 2014-15 Amritapuri Online - ISRO Digit Sum (run-length encoded N)
2 // Standard digit sum-of-sums, scanning digits of N from the left. At a digit d
3 // with k digits after it and prefix digit-sum ps, the numbers that branch off
4 // with a smaller digit c < d contribute
5 //      d*10^k*ps + 10^k*d*(d-1)/2 + d*k*45*10^(k-1).
6 // A run of 'len' equal digits is summed in closed form with geometric series
```

```

7 //      GO = sum 10^j,  G1 = sum j*10^j  over j = k_min .. k_max.
8 // Finally add digitsum(N) itself.
9 #include <iostream>
10 #include <vector>
11 using namespace std;
12
13 using ll = long long;
14 using i128 = __int128;
15 const ll MOD = 1'000'000'007LL;
16
17 ll power(ll b, ll e) {                // e >= 0, may be ~1e18
18     ll r = 1;
19     b %= MOD;
20     if (b < 0) b += MOD;
21     while (e > 0) {
22         if (e & 1) r = (i128)r * b % MOD;
23         b = (i128)b * b % MOD;
24         e >>= 1;
25     }
26     return r;
27 }
28 ll inv(ll x) { return power(x, MOD - 2); }
29
30 // sum_{i=0}^{n-1} 10^i  and  sum_{i=0}^{n-1} i*10^i  (mod p)
31 ll geo0(ll n) { return (power(10, n) - 1 + MOD) % MOD * inv(9) % MOD; }
32 ll geo1(ll n) {
33     // r (1 - n r^{n-1} + (n-1) r^n) / (1-r)^2 with r = 10
34     ll nm = n % MOD, rn1 = power(10, n - 1), rn = power(10, n);
35     ll inner = (1 - (i128)nm * rn1 % MOD + (i128)((nm - 1 + MOD) % MOD) * rn % MOD) %
36     MOD;
37     inner = (inner + 2 * MOD) % MOD;
38     return (i128)10 * inner % MOD * inv(81) % MOD;
39 }
40
41 int main() {
42     ios::sync_with_stdio(false);
43     cin.tie(nullptr);
44     const ll INV10 = inv(10);
45     int T;
46     cin >> T;
47     while (T--) {
48         int m;
49         cin >> m;
50         vector<ll> len(m);
51         vector<int> dig(m);
52         ll total = 0;                // total number of digits of N
53         for (int i = 0; i < m; i++) { cin >> len[i] >> dig[i]; total += len[i]; }
54         ll ans = 0, ps = 0, used = 0;    // ps = digit sum of prefix (mod p)
55         for (int i = 0; i < m; i++) {
56             ll d = dig[i], n = len[i];
57             ll kmax = total - used - 1, kmin = kmax - n + 1;
58             // GO = sum_{j=kmin}^{kmax} 10^j, G1 = sum j 10^j
59             ll base = power(10, kmin), g0n = geo0(n), g1n = geo1(n);
60             ll G0 = (i128)base * g0n % MOD;
61             ll G1 = (i128)base * (((i128)(kmin % MOD) * g0n + g1n) % MOD) % MOD;
62             // sum over t of t*10^k with k = kmax - t : kmax*G0 - G1
63             ll T1 = ((i128)(kmax % MOD) * G0 % MOD - G1 + MOD) % MOD;
64             ll part = ((i128)ps * G0 + (i128)d * T1) % MOD;    // prefix sums

```

```

64     part = (i128)part * d % MOD;
65     part = (part + (i128)G0 * (d * (d - 1) / 2)) % MOD;    // smaller digits
66     part = (part + (i128)d * 45 % MOD * G1 % MOD * INV10) % MOD; // free tail
67     ans = (ans + part) % MOD;
68     ps = (ps + (i128)d * (n % MOD)) % MOD;
69     used += n;
70 }
71 ans = (ans + ps) % MOD;           // N itself
72 cout << ans << '\n';
73 }
74 return 0;
75 }

```

Complexity Analysis

Time: $O(M \log L)$ per test. **Space:** $O(M)$.

Common Pitfalls and Edge Cases

- Never expand the runs: 10^{18} digits.
- Exponents may be reduced modulo $p - 1$ only because $\gcd(10, p) = 1$; multiplicative factors like n are reduced modulo p .
- Division by 9, 81 and 10 means multiplying by modular inverses.

Appendix A

Contest Cheat Sheet

Complexity budget

Roughly 10^8 simple operations per second.

n	acceptable complexity
≤ 20	$O(2^n \cdot n)$
≤ 500	$O(n^3)$
≤ 5000	$O(n^2)$
$\leq 2 \cdot 10^5$	$O(n \log n)$
$\leq 10^{12}$	$O(\sqrt{n})$
$\leq 10^{18}$	$O(\log n)$

Template

```
1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4 using namespace std;
5 using ll = long long;
6
7 int main() {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10    int t;
11    cin >> t;
12    while (t--) {
13        // read, solve, print with '\n' (never endl in loops)
14    }
15 }
```

Modular arithmetic

```
1 const ll MOD = 1'000'000'007LL;
2 ll power(ll b, ll e, ll m = MOD) {      // b^e mod m, O(log e)
3     ll r = 1 % m; b %= m; if (b < 0) b += m;
4     for (; e > 0; e >>= 1, b = b * b % m)
5         if (e & 1) r = r * b % m;
6     return r;
7 }
8 ll inv(ll a) { return power(a, MOD - 2); } // MOD prime, a % MOD != 0
```

Sieve (smallest prime factor)

```

1 vector<int> spf(N + 1, 0);
2 for (int i = 2; i <= N; i++)
3     if (!spf[i])
4         for (int j = i; j <= N; j += i)
5             if (!spf[j]) spf[j] = i;
6 // factorise x: while (x > 1) { int p = spf[x]; while (x % p == 0) x /= p; }

```

Xor basis

```

1 unsigned long long b[61] = {};
2 bool insert(unsigned long long v) { // true if v was independent
3     for (int i = 60; i >= 0; i--) if (v >> i & 1) {
4         if (!b[i]) { b[i] = v; return true; }
5         v ^= b[i];
6     }
7     return false; // v is representable
8 }

```

Identities

$x + y = (x \oplus y) + 2(x \& y)$	$\gcd(a, b) \cdot \text{lcm}(a, b) = ab$
$a \oplus b = c \iff a \oplus c = b$	$a^{p-1} \equiv 1 \pmod{p}, p \nmid a$
$\bigoplus_{i=0}^{2^M-1} i = 0 \ (M \geq 2)$	$\sum_{d \leq n} \lfloor n/d \rfloor \approx n \ln n$
$x \& (x - 1)$: drop lowest bit	$\tau(n) = \prod (k_i + 1)$
$i \oplus (i \gg 1)$: Gray code	$\sigma(n) = \prod \frac{p_i^{k_i+1} - 1}{p_i - 1}$

Greedy proof patterns

- **Exchange:** swap the first disagreement of an optimal solution into the greedy choice; show nothing gets worse.
- **Stays ahead:** after every step greedy is at least as good by some measure.
- **Lower bound = construction:** find a quantity every solution must pay; show greedy pays exactly that.

Last-hour checklist

1. long long for sums, products, and anything times n .
2. Multi-test: reset every array/set/counter per test.
3. Try $n = 1$, all equal, already sorted, maximum values.
4. Strict vs non-strict comparisons in sorts and sweeps.
5. For a WA on a greedy, write a brute force for $n \leq 8$ and diff.